CUA-Gym: 面向计算机使用智能体的可验证训练环境与任务的规模化构建

Bowen Wang[1,2] Dunjie Lu[1,2] Junli Wang[3] Tianyi Bai[2] Shixuan Liu[2]
Zhipeng Zhang[2] Haiquan Wang[2] Hao Hu[4] Tianbao Xie[1] Shuai Bai[2]
Dayiheng Liu[2] Que Shen[2] Junyang Lin[2] Tao Yu[1†]

[1]香港大学 [2]阿里巴巴通义千问团队
[3]加利福尼亚大学圣迭戈分校 [4]清华大学

项目主页: https://cua-gym.xlang.ai
数据集: https://huggingface.co/datasets/xlangai/CUA-Gym
代码库: https://github.com/xlang-ai/CUA-Gym
环境: https://github.com/xlang-ai/CUA-Gym-Hub

摘要

具有可验证奖励的强化学习（RLVR）已推动数学、工具使用和软件工程等领域取得突破，但将其扩展到计算机使用智能体（CUA）时，却受限于带有确定性奖励、可规模化构建的训练数据匮乏。为CUA 构建此类数据，需要任务指令、可执行环境与可验证奖励彼此一致。然而，人工整理的基准虽然奖励保真度高，却只覆盖少量应用；基于大语言模型评判的数据集虽然易于扩展，却缺乏可靠验证。我们提出CUA-Gym，一条可规模化的流水线，从共同的主题规范出发，联合生成任务指令、环境状态和奖励函数。具体而言，生成器智能体构建初始状态与标准完成状态，独立的判别器智能体仅依据任务规范编写奖励函数。编排器智能体推动两者多轮迭代，直到执行奖励函数时能够区分初始环境与标准完成环境。生成的元组随后通过结合大语言模型多数投票与智能体轨迹采样的最终过滤器，在单任务对抗循环之外进一步保障质量。为解决训练环境匮乏的问题，我们进一步使用相同的智能体流水线合成CUA-Gym-Hub：一套以真实世界软件使用分布为依据、覆盖广泛的高保真网页仿真应用，使CUA RLVR 数据可达到的规模提升一个数量级。利用该流水线，我们构建了CUA-Gym 数据集，包含基于**110** 个环境的**32,112** 个经过验证的RLVR 训练元组。在CUA-Gym 上使用GSPO 训练后，我们的CUA-Gym-A3B 和CUA-Gym-A17B 在OSWorld-Verified 上分别达到**62.1%** 和**72.6%**，优于此前同等规模的开源CUA；性能随数据量和环境多样性增加而平稳提升。同一批模型检查点在留出的WebArena 浏览器基准上也有所提升，表明其迁移能力超出了训练环境。我们将开源完整的合成流水线、数据集、CUA-Gym-Hub 环境与模型。

1 引言

具有可验证奖励的强化学习（RLVR）已成为前沿模型的主流后训练范式，推动了数学 [9, 18]、软件工程 [13, 33, 8, 6] 以及终端使用 [7, 14] 领域的突破。其方法已经较为清晰：以程序化方式合成大规模训练任务并配套确定性奖励信号，再使用GRPO [18] 等算法依据这些信号优化策略。在这些领域中，可验证训练数据的供给已多次被证明是制约智能体能力的关键因素；在当前数据量下，数据与性能的缩放曲线仍未饱和。鉴于计算机使用智能体（CUA）有望自动完成广泛的数字化知识工作，将这一方法扩展到CUA 是自然而迫切的下一步。然而，尽管CUA 基础模型 [2, 22, 23] 进展迅速，该领域的RLVR 训练数据仍然稀缺且零散，远未达到推动其他领域突破的数据规模。

瓶颈来自结构层面，而非算法层面。数学或代码领域的训练实例可以归结为问题描述和可检查的答案；相比之下，CUA RLVR 实例是由任务指令、可执行环境状态和奖励函数构成的元组 (t, s, r)，其中每个组成部分都是不可小觑的工程产物，且必须相互配合。人工编写一个这样的元组就需要专家投入数小时；随着应用多样性增加，成本还会叠加，因为每个新应用都有自己的设置流程和验证接口。因此，CUA RLVR 数据集比数学、软件工程及终端操作等相邻纯文本领域的RLVR 数据集小数个数量级。然而，现有方法尚未同时满足可规模化CUA RLVR 所需的三项属性：确定性的可验证奖励、广泛的应用覆盖，以及可扩展的任务多样性。监督数据集 [23, 30] 覆盖多种应用，但提供的是轨迹层面的模仿目标，而非结果奖励，因而无法用于RL。以视觉语言模型为评判器的框架 [32] 能够为任意任务评分，却会引入使策略优化不稳定的奖励噪声。代码原生流水线 [5, 36] 能够在浏览器仿真环境中生成确定性奖励，但迄今规模有限，且局限于浏览器环境。因此，CUA RLVR 一直未能复现数学和代码领域中数据规模与性能同步提升的趋势。

我们提出CUA-Gym，一条为CUA 规模化合成可验证RLVR 训练数据的智能体流水线。给定主题规范，CUA-Gym 将任务指令、环境设置和奖励函数联合生成为一个经过验证的元组，借助编程智能体完成过去

*通讯作者。

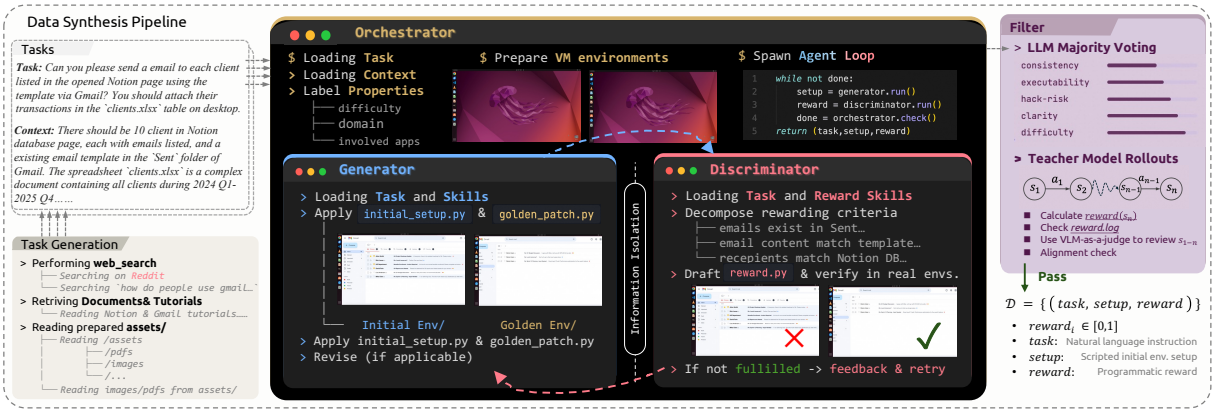


图 1: **CUA-Gym 数据合成流水线概览**。从任务指令及其有依据的上下文出发，**编排器**配置成对虚拟机，并启动两个以对抗方式耦合的智能体。**生成器**编写`initial_setup.py`与`golden_patch.py`，构建初始状态与标准完成状态。**判别器**通过*信息隔离屏障*与生成器脚本隔离，仅依据任务独立编写`reward.py`。两个智能体在编排器调度下迭代，直到满足五项一致性条件。被接受的元组随后通过结合大语言模型多数投票与教师模型轨迹采样的**过滤器**，形成最终数据集 $\mathcal{D}$。

需要人类专家承担的工程工作。流水线运行三个相互协调的智能体：生成器构建初始环境状态与标准完成环境状态；判别器仅依据任务描述编写奖励函数；编排器推动两者多轮迭代，直到奖励函数在执行中能够区分这两个环境。我们进一步采用严格的信息隔离来防止奖励投机，并通过结合大语言模型多数投票与智能体轨迹采样的最终过滤阶段，剔除虽通过单任务验证、却在真实智能体行为下失败的元组。除单任务合成之外，CUA-Gym 还将环境视为一个规模扩展维度。真实世界CUA 训练不仅受任务数量制约，也受环境覆盖广度制约；现有基准只覆盖知识工作者实际使用的软件版图中的很小一部分。为此，我们首先依据O*NET 的职业分类体系 [38]与Anthropic 经济指数中的软件使用分布 [3]选择目标应用。对于每个选定应用，我们再使用多智能体合成流水线实现带有统一状态API 的独立网页仿真应用，并通过基于浏览器的自动化测试验证交互功能。所得环境可复现、可编程，且在结构上忠实于真实应用；借助单任务联合生成流水线，每个环境都能支持数千种不同任务，使CUA RLVR 数据可达到的规模提升数个数量级。我们将这些可复用的合成网页环境打包为CUA-Gym-Hub。这一环境基础层将环境规模扩展与任务合成解耦，使新的可验证任务能够基于共享、可重置的应用池生成。

我们使用该流水线构建CUA-Gym 数据集，包含跨越110 个环境的32,112 个经过验证的RLVR 训练元组，其中包括16 个桌面应用以及来自CUA-Gym-Hub 的94 个合成网页仿真应用。利用这一数据集，我们在两个开源混合专家模型主干Qwen3.5-35B-A3B 和Qwen3.5-397B-A17B 上，通过GSPO [37] 实施RLVR。在OSWorld-Verified [28] 上，训练后的模型分别达到**62.1%** 和**72.6%**；较小的A3B 模型以约少 10× 的总参数量，达到未经此训练的A17B 基础模型的性能。同一批检查点在留出的WebArena 浏览器基准上也优于各自基础模型，展现了从合成网页仿真应用向真实浏览器基准的跨平台迁移。性能随数据量与环境池规模增加而平稳提升；将环境池从10 个扩展到80 个所带来的增益，仅增加轨迹数量无法弥补，这表明环境多样性是与数据量互补的扩展维度。我们还观察到，RL 训练会自发诱导多动作工具调用：在任务性能相当时，轨迹长度缩短33–45%。这一涌现出的效率行为，与面向推理的RL 中自发出现验证和自我反思的现象类似。

我们的贡献包括三个方面。**(1)** 提出CUA-Gym，一条通过构造方式缓解奖励投机、联合生成可验证RLVR 训练元组的智能体流水线，并提供迄今规模最大的开放CUA RLVR 数据集。**(2)** 将同一智能体范式扩展到环境合成，发布可复用的可控网页环境集合CUA-Gym-Hub，并表明环境多样性是CUA RL 的独立扩展维度。**(3)** 首次展示了推动数学与代码领域突破的RLVR 数据方法能够规模化迁移到CUA；训练所得的CUA-Gym-A3B 和CUA-Gym-A17B 在各自规模的开源CUA 中，取得了最先进的OSWorld-Verified 结果。我们将开源完整合成流水线、数据集、CUA-Gym-Hub 环境与模型，以支持可复现的CUA RLVR 研究。

2 方法

CUA-Gym 通过联合生成任务指令、环境状态与奖励函数的流水线，以可规模化的方式自动构建计算机使用智能体的RLVR 训练数据。流水线沿两个互补维度扩展：数据合成（§2.1）与环境规模扩展（§2.2）。

2.1 数据合成

我们将用于CUA 训练的RLVR 数据形式化为元组 (t, s, r) 的集合，其中 t 是自然语言任务指令，s 表示可复现的初始环境状态，$r : s \to [0, 1]$ 是可验证奖励函数。如图 1 所示，CUA-Gym 使用联合生成流水线解决这一问题：三个组成部分均从同一个查询规范推导而来，通过构造方式确保一致性。

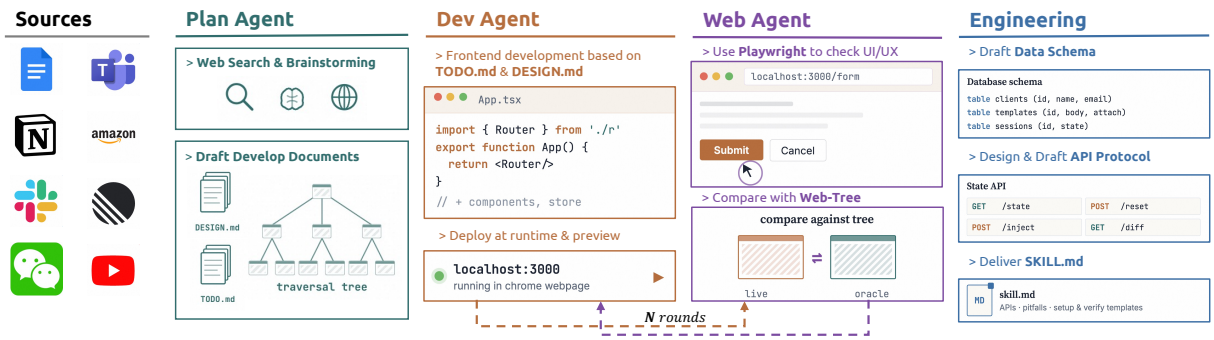


图 2: 用于仿真环境合成的多智能体流水线。从目标应用种子出发，规划智能体编写DESIGN.md 与TODO.md，规定功能、数据模式、API 协议和UI 布局。开发智能体依据这些规范实现单页应用。网页智能体随后通过Playwright 操作每个交互元素，对照规范检查实时DOM，并将差异反馈给开发智能体，直至收敛。最终产物包括状态API，以及面向下游任务生成、记录API、易错点和验证模板的SKILL.md。

任务生成 单有指令无法确定其执行所需的环境。因此，我们将每个任务生成为指令与上下文对 (t, c)，其中 c 枚举初始环境所需的实体、文件与应用状态。上下文来自三类互补的事实依据：对真实使用模式的网络调研、软件文档以及预先准备的素材文件。每个任务都标注难度、领域与涉及的应用，以支持第 4 节所述的均衡采样。详细信息见附录 A.2。

环境设置与奖励的对抗式联合生成 手工编写设置脚本和奖励脚本无法规模化：每个任务都需要针对不同应用、文件格式和验证原语定制逻辑。我们将这项工作交给编程智能体，它们可以跨任务迁移领域知识，并快速适应新应用。然而，不能信任单个智能体同时负责问题的两侧。如果同一智能体既构建标准完成状态，又编写奖励，奖励往往会重复检查构建过程，而非衡量任务完成情况，所得元组在RL 训练中提供的信号便十分有限。因此，我们将这两个角色拆分给以对抗方式耦合的子智能体，并用严格的*信息隔离屏障*隔开，使奖励合成必须依据任务本身，而非环境设置的构建方式。

图 1 展示了所得流水线。对于每个任务，编排器配置两台虚拟机并启动两个子智能体。生成器读取任务及相关领域技能，随后编写`initial_setup.py` 和`golden_patch.py`，并分别在两台虚拟机上执行，以产生 s_{init} 与 s_{gold}。判别器在沙箱进程中运行，无法访问生成器脚本或工作目录；它只能看到 t 与两个生成后的环境。在此基础上，判别器将任务拆解为细粒度子标准，并编写`reward.py`，将这些标准汇总为渐进式的 $[0, 1]$ 分数。编排器在每轮后检查两个智能体的输出，并触发下一轮迭代，直到所有一致性条件同时满足。

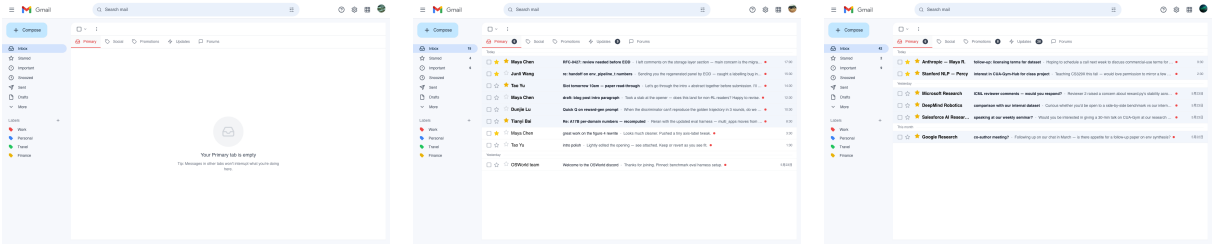
循环层面的收敛能保证元组内部一致，却无法暴露指令含糊或任何策略都无法满足的规范等更高层问题；这些问题只有在轨迹采样中才会显现。因此，我们让每个被接受的元组通过包含两个独立阶段的数据集级过滤器。第一阶段通过大语言模型多数投票，从一致性、可执行性、投机风险、清晰度和难度校准五个方面为元组评分。第二阶段利用教师模型轨迹采样，验证任务可解且奖励信号能够反映任务成功。通过两个阶段的元组进入最终数据集。这一过滤器弥合了循环层面一致性与端到端训练实用性之间的差距，剔除内部循环无法检测的含糊或不可行任务。

2.2 环境规模扩展

CUA-Gym 的任务多样性受可用环境多样性的上限约束。现有基准只覆盖少量桌面应用；真实网站则由于身份认证、速率限制以及不可复现的状态，无法以程序方式注入、检查或重置，因此不适合作为RL 训练环境。为此，我们合成了CUA-Gym-Hub：一套独立的网页仿真应用，既保持与真实应用相近的交互保真度，又支持对状态进行完整的程序化控制。该集合覆盖94 个应用，来自两类来源：涵盖通信、生产力、电商和分析等领域的常用产品，以及为拓展领域覆盖而选取的长尾应用。每个仿真应用都是单页应用，由统一的HTTP API 支持；该API 暴露会话状态，以供检查、注入和重置。这种设计使数据合成流水线能够将每个会话视为沙箱单元，并将状态转移恢复为结构化差异，为奖励计算提供明确的基础。从这一意义上说，CUA-Gym-Hub 本身不是一个基准，而是可复用的环境层：同一组可重置应用可以支持众多生成任务、奖励函数和后训练方案。

规模化合成高保真仿真应用，既需要忠实建模目标应用，也需要系统验证交互正确性；为此我们采用多智能体合成流水线（图 2）。规划智能体通过网络调研和参考截图，将每个仿真应用与真实目标对齐，并产出完整的开发规范，其中包含作为验证判据的UI 布局树。开发智能体依据规范实现应用。网页智能体随后操作每个交互元素，将实时行为与判据比较，再将差异反馈给开发智能体修复。开发智能体与网页智能体迭代直至收敛，从而无需人工干预，并使所有合成环境达到一致的基本保真水平。生成的仿真应用与记录其API 和验证模板的`SKILL.md` 一同，直接接入第 2.1 节的数据合成流水线，作为生成来源。

两个设计使这些仿真应用成为动态训练环境，而非固定的网站副本。*状态注入*从结构化初始状态实例化任务专属世界，因此单个仿真应用实现无需改动代码，就能支持多种不同任务。*会话隔离*将每次修改、上传和重置限制在单个会话中，使分布式RL 工作进程能够并发使用共享的仿真后端池，同时看不到彼此的



(a) 整洁的收件箱。 (b) 项目截止日期密集的状态。 (c) 离开一段时间后的邮件积压。

图 3: **CUA-Gym-Hub 中注入不同状态的环境**。同一个邮件仿真应用可以实例化为不同的任务专属初始状态：空收件箱、包含待协调会话的项目截止日期状态，以及大量邮件积压状态。由于所有修改和重置均限定于各自会话状态，并行轨迹采样工作进程能够共享同一应用实现，同时观察彼此隔离的世界状态。

修改。图 3 通过邮件环境展示了这一属性：界面和应用代码保持不变，但注入状态改变了可用邮件、标签与未读数量，进而改变可行任务的分布。

3 实验

我们在旨在反映真实世界软件使用情况的环境池上合成CUA-Gym 训练数据。环境池将网页仿真应用与具有代表性的桌面应用结合起来：前者的目标依据O*NET 职业任务分类 [38]和Anthropic 经济指数中的软件使用分布 [3]选取；后者涵盖办公生产力、网页浏览、图像编辑、开发工具与媒体。所得数据集的定量构成见第 4 节。

我们从两个维度评估CUA-Gym：一是生成训练数据的质量，以使用这些数据训练的智能体性能衡量；二是流水线的可扩展性，以数据规模对下游智能体性能的影响衡量。

3.1 实验设置

数据集。 我们从CUA-Gym 中抽取10,858 个经过验证的元组作为RLVR 训练集，覆盖 80+ 个环境中的任务。此外，我们在同一任务集上使用Claude-Sonnet-4-6 [4]进行轨迹采样，仅保留成功完成的轨迹，整理出3,578 条轨迹用于SFT 预热。

模型。 我们在Qwen3.5 系列 [15]的两种参数规模上开展实验：Qwen3.5-35B-A3B 和Qwen3.5-397B-A17B。

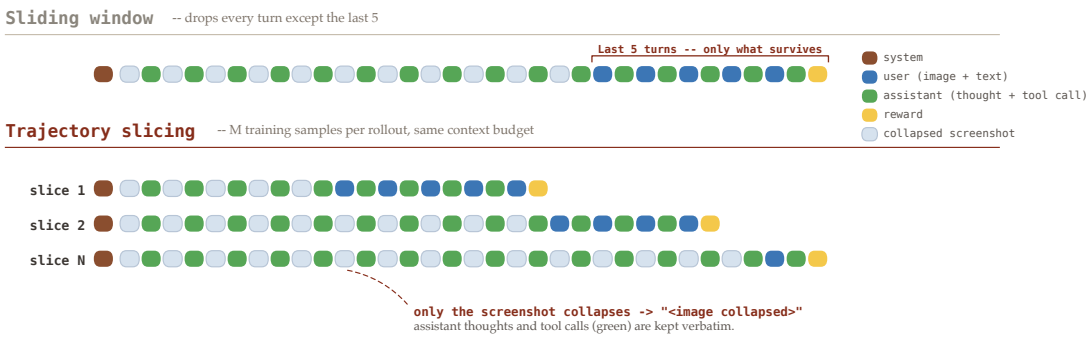


图 4: **面向长时程CUA 轨迹的训练运行框架**。轨迹切片使近期多模态观测保持可用，同时将较早截图折叠为确定性占位符，而非使用会丢弃较早交互状态的单一滑动窗口进行训练。这在固定上下文预算下保留了轨迹后期的监督，并形成稳定前缀，便于在计算策略对数概率时复用KV 缓存。

运行框架。 智能体观察原始截图，并输出包裹在工具调用标签中的动作。长时程CUA 轨迹采样带来两项实际压力：对包含大量图像的历史进行上下文管理，以及针对众多重叠轨迹前缀进行高效训练。因此，我们使用图 4 所示的运行框架：一种确定性的*轨迹切片*方案，在固定上下文预算下为每条采样轨迹构建多个训练样本。传统滑动窗口只是在对话上移动，并丢弃解释后续动作所需的较早状态；相比之下，我们的切片保留系统与任务前缀，将过时截图折叠为紧凑占位符，并完整保留近期观测和助手动作的多模态形式。这种格式保留了后期轮次的监督，避免有损的学习式摘要，并显式保留可复用前缀，从而在计算相邻切片的策略对数概率时复用KV 缓存。完整构建细节见附录 C.1.2。

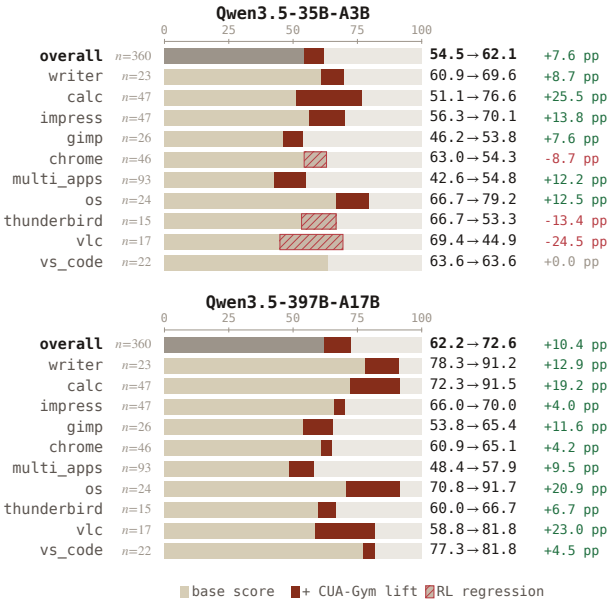
算法。 我们使用组序列策略优化（GSPO）[37]训练所有模型，它为混合专家模型的RL 训练提供更高稳定性。给定任务指令 t 与初始状态 s，策略 π_θ 生成 G 条轨迹 $\{\tau_1, \ldots, \tau_G\}$，每条轨迹获得可验证奖励 $r_i = r(s, \tau_i) \in [0, 1]$。GSPO 使用序列级重要性比率，最大化以组内归一化优势 $\hat{A}_i = (r_i - \mu)/\sigma$ 加权的截断替代目标，其中 μ 和 σ 分别是 $\{r_j\}_{j=1}^{G}$ 的均值与标准差：

$$\mathcal{L}_{\text{GSPO}}(\theta) = \mathbb{E}_{t,s\sim\mathcal{D},\,\tau_i\sim\pi_{\theta_{\text{old}}}}\left[\frac{1}{G}\sum_{i=1}^{G}\min\Big(\rho_i\,\hat{A}_i,\ \text{clip}(\rho_i, 1-\varepsilon, 1+\varepsilon)\,\hat{A}_i\Big) - \beta\, D_{\text{KL}}[\pi_\theta \| \pi_{\text{ref}}]\right], \tag{1}$$

其中，序列级重要性比率为

$$\rho_i = \left(\frac{\pi_\theta(\tau_i \mid t, s)}{\pi_{\theta_{\text{old}}}(\tau_i \mid t, s)}\right)^{1/|\tau_i|}, \tag{2}$$

其中 $|\tau_i|$ 是轨迹长度，ε 和 β 分别控制截断阈值，以及相对于参考策略 π_{ref} 的KL 惩罚。



(a) OSWorld-Verified 上各领域的成功率。

Model	OSWorld-V.	WebArena
Proprietary Models		
Claude Sonnet 4.6	72.9	65.6
Claude Opus 4.7	78.0	–
GPT-5.5	78.7	–
Open-source Models		
EvoCUA-8B	46.1	–
EvoCUA-32B	56.7	–
OpenCUA-32B	34.8	–
OpenCUA-72B	45.0	–
Step-GUI-8B	40.2	–
Kimi-K2.6	73.1	–
Our Models		
Qwen3.5-35B-A3B	54.5	40.8
Qwen3.5-397B-A17B	62.2	54.0
CUA-Gym-A3B	**62.1**	**44.5**
CUA-Gym-A17B	**72.6**	**56.0**

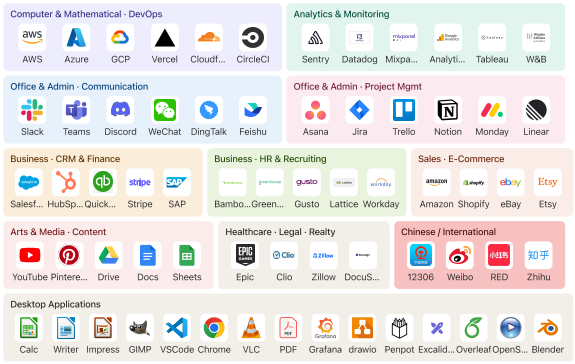
(b) OSWorld-Verified 与WebArena 上的主要结果。

图 5: 主要结果。（a）CUA-Gym-A3B（红色）与经过SFT 初始化的基础模型（灰色虚线）在OSWorld-Verified 上各领域的成功率；n 表示各领域任务数，训练后各领域分数使用第4 次运行的检查点明细。最大增益出现在`libreoffice_calc`、`multi_apps` 与`vs_code`。（b）OSWorld-Verified 与WebArena 上的成功率。在CUA-Gym 数据上进行RLVR，使Qwen3.5-35B-A3B 与Qwen3.5-397B-A17B 两个基础模型均有提升；CUA-Gym-A3B 以约少 10× 的总参数量，达到Qwen3.5-397B-A17B 基础模型的水平。

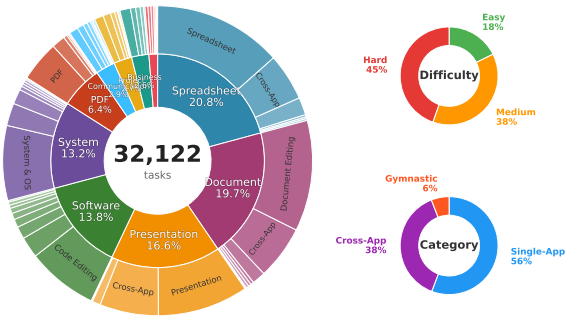
主要结果 表 5b 报告了OSWorld-Verified [28]（OSWorld [26] 的经过验证版本）和WebArena 上的性能。在CUA-Gym 数据上进行RLVR，使两种参数规模的基础模型均有提升：CUA-Gym-A3B 将Qwen3.5-35B-A3B 基础模型从54.5 提升至62.1（+7.6 个百分点），CUA-Gym-A17B 将Qwen3.5-397B-A17B 基础模型从62.2 提升至72.6（+10.4 个百分点）。在更大模型规模下，增益依然保持而未减弱；在这一规模区间，GUI 智能体的RL 收益以往往往被观察到趋于平台。同一批检查点在WebArena 上也有所提升。WebArena 是留出的浏览器基准，其网站克隆与训练所用的94 个网页仿真应用互不重合。CUA-Gym-A3B 将Qwen3.5-35B-A3B 基础模型从40.8 提升至44.5（+3.7 个百分点），CUA-Gym-A17B 将Qwen3.5-397B-A17B 基础模型从54.0 提升至56.0（+2.0 个百分点）。这种迁移表明，第 2.2 节的网页仿真流水线产生了能够跨浏览器环境泛化的技能，而非仅拟合合成仿真应用，支持了CUA-Gym 同时面向桌面与网页的定位。

4 分析

CUA-Gym 数据集包含覆盖110 个环境的32,112 个经过验证的RLVR 训练元组。环境池（图 6a）以两个外部来源为依据：O*NET 职业任务分类确定与SOC 主要职业组对齐的94 个网页应用类别，Anthropic 经济指数中的软件使用分布则影响类别内部的选择。互补的桌面应用集合覆盖网页环境无法表达的操作系统级工作流。我们将合成的网页部分发布为CUA-Gym-Hub，一套可重置的环境集合，旨在使环境覆盖能力能够在特定的CUA-Gym 任务语料之外复用。任务构成（图 6b）在类别之间保持均衡，没有单一类别超过21%，并侧重较难区间：45% 的任务标记为困难，38% 涉及跨应用工作流。表 6c 将CUA-Gym 与现



(a) 各职业类别的环境覆盖。



(b) 任务在类别、难度和跨应用复杂性上的分布。

Dataset	Platform	Data Size	Env. Size	Reward	Open
GUI-Genesis [5]	Mobile	969	1	Programmatic	No
WebArena-Infinity [24]	Web	1,260	10	Programmatic	Yes
InfiniteWeb [36]	Web	600	–	Programmatic	No*
UltraCUA [34]	Desktop	17,000	9	Programmatic	No*
Gym-Anything [1]	Desktop	7,277	193	VLM	Yes
CUA-Gym (Ours)	**Desktop+Web**	**32,112**	**110**	**Programmatic**	**Yes**

(c) 与现有GUI 智能体训练数据集的比较。*Programmatic* 表示代码原生断言；*VLM* 表示以视觉语言模型为评判器；* 表示已承诺发布但尚未发布。

图 6: **CUA-Gym 的数据与环境。(a)** CUA-Gym 覆盖110 个环境，包括与O*NET 职业类别 [38]对齐的CUA-Gym-Hub 网页环境。**(b)** 32,112 个经过验证的元组均衡覆盖各应用类别（没有类别超过21%），并偏向较难任务（45%）与跨应用任务（38%）。**(c)** 与现有RLVR 数据集相比，CUA-Gym 是规模最大的公开发布集合，并且独特地以程序化验证同时覆盖桌面和网页平台。

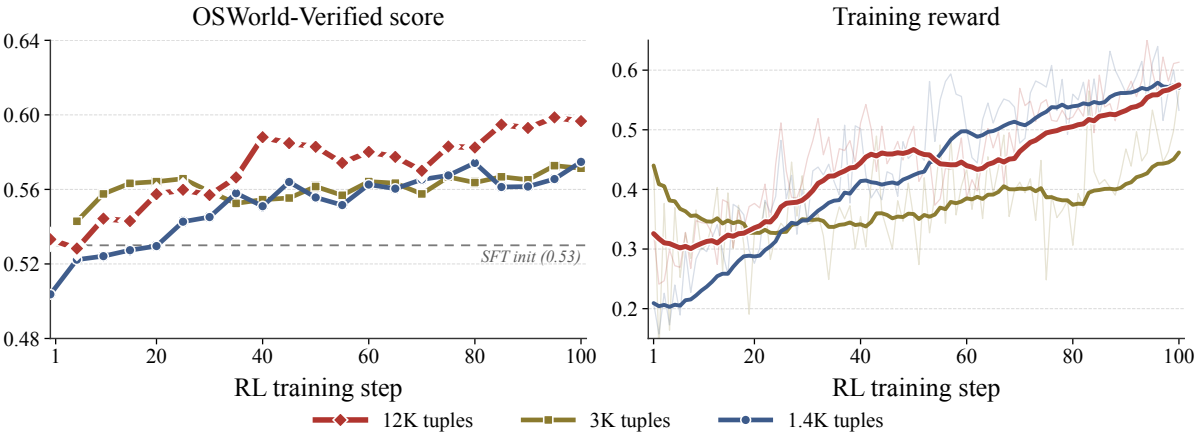


图 7: **在CUA-Gym 上进行RL 训练的数据规模扩展。**在Qwen3.5-35B-A3B 上使用三个CUA-Gym 训练子集（1.4K、3K、12K 个经过验证的元组）时，OSWorld-Verified 得分（左）与训练奖励（右）随RL 训练步数的变化。所有运行均从相同的SFT 检查点初始化（0.53 处的灰色虚线）。浅色线表示逐步原始值，粗线表示指数平滑值。

有GUI 智能体训练数据集进行比较；据我们所知，它是将程序化验证与桌面、网页双平台覆盖相结合的最大开源语料库。

4.1 数据规模扩展

CUA-Gym 的一个核心前提是：扩大经过验证的RLVR 数据规模，能在下游RL 训练中产生可衡量的增益。我们通过在CUA-Gym 的三个经验证子集（1.4K、3K 和12K 个元组）上，使用GSPO 训练Qwen3.5-35B-A3B 来检验这一前提；各次运行的SFT 初始化和所有训练超参数均保持不变。所得训练曲线还可用于诊断第 2.1 节的信息隔离屏障设计，因为可被利用的奖励函数往往会在RL 优化中诱发具有特征性的不稳定现象。

扩大RL 数据规模提高了RL 训练的性能上限。 图 7 中的三次运行随数据规模形成分层，且在整个训练过程中保持相对排序。12K 运行达到最高峰值，比两个较小规模的运行更早脱离SFT 基线，并在所探索区间内持续保持明显更高的性能范围；3K 与1.4K 运行则都在更靠近基线的位置进入平台。因此，新增CUA-Gym 数据的边际价值同时体现在两个方面：渐近性能上限与训练曲线的整体形态。这表明在本文考察的范围内，规模扩展产生了实际RL 收益，尚未表现为收益递减。12K 曲线也没有出现趋向饱和的拐点，表明该流水线所能支持的数据规模扩展区间尚未耗尽。我们还注意到，三次运行均从SFT 基线单调提升，没有出现RLVR 训练中常见的振荡、崩溃，或奖励与成功率脱钩现象，说明合成奖励足够稳定，能够支持我们考察的各数据规模下的训练。

4.2 环境规模扩展

第 4.1 节的数据规模实验表明，轨迹数量是CUA-Gym 数据推动RL 性能提升的一个维度。第二个维度隐含于数据集构建方式中：第 2.2 节的CUA-Gym-Hub 网页仿真合成流水线，为扩大环境覆盖投入了大量工程工作，其前提是多样性能带来仅靠增加轨迹量无法获得的下游性能收益。这一前提是否成立，是我们在此考察的实证问题。环境级消融所需的细粒度划分，使基于RL 的评估成本高得难以承受，因此我们采用教师蒸馏设置开展实验。这还带来额外收益：展示CUA-Gym 数据也可用于不依赖RL 基础设施的后训练方案。

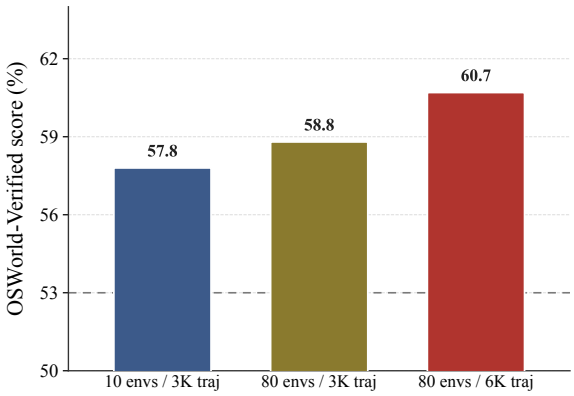


图 8: **教师蒸馏下的环境规模扩展。**在100 步评测预算下，OSWorld-Verified 得分随训练环境数量的变化。广覆盖设置（80 个环境，每个75 条轨迹）优于窄覆盖设置（10 个环境，每个300 条），尽管前者每个环境使用的轨迹少 4×。

蒸馏流水线。 我们使用Claude-Sonnet-4-6 [4] 在CUA-Gym 任务上进行轨迹采样，仅保留满足 $r(s,\tau) = 1$ 的轨迹，从而得到过滤后的教师示范池。学生模型是第 4.1 节所用的同一个Qwen3.5-35B-A3B 基础模型，使用标准SFT 在过滤后的轨迹上微调。我们从CUA-Gym-Hub 池中构建三种环境规模设置：*窄覆盖*（10 个环境，3K 条轨迹，每环境300 条）、*中等覆盖*（80 个环境，3K 条轨迹，每环境38 条）和*广覆盖*（80 个环境，6K 条轨迹，每环境75 条）。窄覆盖与中等覆盖的对比，在总数据量固定时隔离环境多样性的作用；中等覆盖与广覆盖的对比，在环境覆盖固定时隔离轨迹数量的作用。三个学生模型使用完全相同的SFT 超参数，唯一差异是训练集构成。

环境多样性与数据量作为互补维度发挥作用。 图 8 沿这两个消融维度分解了规模扩展效应。固定数据量时增加环境，带来适度提升；在拓宽后的环境池中将轨迹数量翻倍（从中等覆盖到广覆盖），则产生明显更大的提升。这一解读为第 2.2 节的合成流水线提供了直接实证支持：扩大环境覆盖的工程投入之所以有价值，恰在于轨迹数量本身无法替代多样化经验。换言之，CUA-Gym-Hub 提供可复用的环境多样性，CUA-Gym 则将这一基础层实例化为经过验证的RLVR 数据。

4.3 涌现的多动作工具调用

在CUA-Gym 的RL 训练中，出现了一种我们未显式优化的行为：策略自发学会在每轮发出多个工具调用，将最初每轮一个动作的轨迹压缩为更密集的多动作轮次。图 9 显示，每步调用数从SFT 初始化时约一次，持续提升至RL 训练期间稳定的 1.4–1.9 区间；在任务性能相当时，轨迹长度缩短33–45%。我们将其归因于组内归一化优势估计下的步数预算压力：在固定单任务预算内完成的轨迹，比超时轨迹获得更高相对奖励，因此GSPO 隐式偏好将菜单遍历、表单填写和键盘剪贴板操作链等确定性子序列装入单轮的策略。对后训练轨迹的检查证实了这一模式。诸如`click(File)`→`click(Export)`→`click(PDF)`的确定性动作序列会在一轮内发出；而结果依赖非确定性状态的动作，包括网络响应以及需要视觉确认的对话框，则明显不会出现在批处理组中。策略已内化出一种粗粒度模型，用于判断哪些动作可以安全地批处理。

这一涌现现象有两方面意义。轨迹缩短约 40%，直接降低了训练期间的轨迹采样成本与部署时的推理延迟；

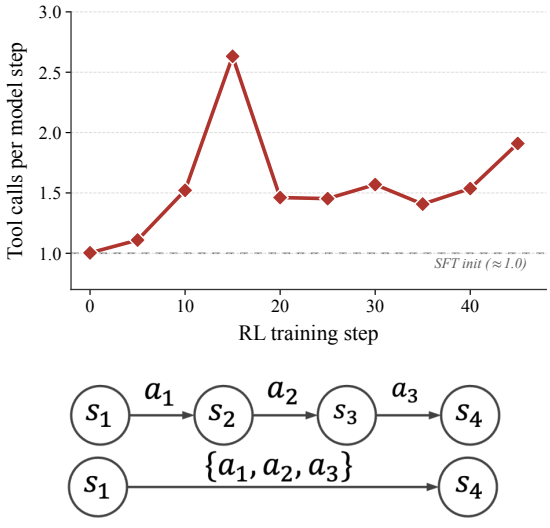


图 9: **RL 过程中涌现的动作批处理。**CUA-Gym-A3B 每个模型步的平均工具调用数。经过SFT 初始化的策略每步约发出一次调用；RL 将其提升至稳定的 1.4–1.9 区间。

这一效率收益完全来自RL，没有架构改动或奖励塑形干预。更广泛地说，这一发现表明，仅凭具有可验证奖励的RL，除了任务层面能力，还能涌现结构性的效率行为。这与面向推理的RL 中自发出现验证和自我反思相呼应，而此次观察发生在动作执行领域。

5 相关工作

数字智能体的数据合成与后训练 在可执行环境中，通过扰动或逆向分析产物来合成任务，再以程序化检查验证结果，已成为数字智能体各领域规模化生成RLVR 数据的主流方法 [7, 33, 6, 14, 10, 11]。RLVE [35] 研究了利用自适应可验证环境扩展RL 的互补方向，进一步凸显环境生成作为RLVR 基础层的重要性。在软件工程领域，SWE-smith [33] 通过语言模型引导的函数重写与抽象语法树层面的变异，在真实代码库中合成缺陷；R2E-Gym [10] 借助执行辅助的反向翻译从提交记录中提取任务；SWE-Gym [13] 则利用预先构建的执行环境，将这一范式扩展到数千个代码库。这些方法均将单元测试作为二元奖励信号。在终端操作领域，Endless Terminals [7] 与Terminal-Task-Gen [14] 通过程序化方式生成容器化命令行任务，并使用双重验证协议：前置测试必须在初始状态失败，且只有正确执行后才能通过。这些流水线在实证上的收益，正是其工程投入的依据。SWE-smith 与SWE-Gym 均报告，性能随合成任务量呈对数线性提升，即使任务达到数万条也尚未饱和 [33, 13]；Endless Terminals [7] 在终端操作上观察到相同趋势。尽管这些领域各不相同，却呈现一致的缩放行为，这意味着RLVR 数据合成尚未进入饱和阶段，也促使我们为基于图形用户界面的计算机使用智能体构建类似基础设施：该领域的数据瓶颈更为突出，且尚无同等规模的缩放研究报告。

GUI 智能体的任务与环境合成 将RLVR 数据合成扩展到GUI 智能体，面临奖励可验证性与环境覆盖范围之间的根本张力。一类工作采用基于视觉语言模型的奖励估计，无需针对环境专门插桩即可跨应用泛化 [32, 16]；ZeroGUI [32] 自身的消融实验也证实，由此产生的假阳性奖励会使RL 训练不稳定。另一类工作合成具有代码原生奖励判定器的可控网页环境：GUI-Genesis [5] 为每个任务构建Flask 应用，InfiniteWeb [36] 采用以任务为中心的测试驱动开发，AutoWebWorld [25] 则利用有限状态机的状态转移模型。然而，这些方法均局限于浏览器交互，无法覆盖操作系统层面的桌面任务或跨应用工作流。近期尝试兼顾桌面环境与可验证性的方法，要么退回基于检查清单的视觉语言模型评估 [1]，要么应用覆盖仍然有限 [19, 31, 29, 12, 27, 21]。OSWorld [26] 提供了面向真实桌面环境的互补评测基准。现有工作尚未在GUI 智能体RLVR 训练中，同时实现确定性的可验证奖励、广泛的操作系统级应用覆盖，以及可扩展的任务多样性。

6 局限性

CUA-Gym 的奖励验证终止环境状态，而非智能体到达该状态的完整过程。这使奖励编写易于规模化，却无法区分规范的编辑操作与先破坏、再重建相同最终状态的操作序列。同样，信息隔离屏障和禁用模式扫描能够减少奖励投机，却不能从形式上排除残余模式，例如过于宽松的语义检查，或仅与任务成功相关的状态代理指标。CUA-Gym-Hub 仿真环境拓宽了训练覆盖范围，但仍是真实应用的近似：身份认证流程、第三方集成、网络延迟、速率限制以及罕见的服务端故障状态，均不在当前仿真范围内。最后，由于计算成本高昂，我们规模最大的RL 运行只报告了单个随机种子的结果。因此，我们将这些结果解释为：在所研究区间内，可验证任务规模与环境规模具有价值，而不是对流水线能力上限或部署鲁棒性的最终刻画。

7 结论

我们提出CUA-Gym，一条为计算机使用智能体的RLVR 训练联合生成任务指令、环境状态与奖励函数的自动化流水线。该流水线在进程级信息隔离屏障下耦合生成器与判别器，通过迭代循环约束元组在初始和完成两个端点的一致性，并利用数据集级过滤器捕捉仅在轨迹采样中显现的失败。为拓宽流水线可用的环境，我们以O*NET 职业分类与Anthropic 经济指数为依据，合成包含94 个网页仿真应用的CUA-Gym-Hub。在OSWorld-Verified 上，使用CUA-Gym 数据训练的模型随数据量增加而提升，在我们考察的范围内尚未饱和。同一批检查点在留出的WebArena 浏览器基准上也有所提升。我们将这些结果视为任务与奖励多样性确实构成当前GUI 智能体重要瓶颈的证据，而非对流水线最终能力上限的断言。我们发布完整合成流水线、CUA-Gym-Hub 环境与生成数据，以供社区研究任务规模扩展与环境规模扩展。

参考文献

[1] Pranjal Aggarwal, Graham Neubig, 和Sean Welleck. Gym-anything：将任意软件转化为智能体环境, 2026. URL https://arxiv.org/abs/2604.06126.

[2] Anthropic. 计算机使用功能、全新Claude 3.5 Sonnet 与Claude 3.5 Haiku 简介. https://www.anthropic.com/news/3-5-models-and-computer-use, 2024.

[3] Anthropic. Anthropic 经济指数. https://www.anthropic.com/economic-index, 2025.

[4] Anthropic. Claude sonnet 4.6. https://www.anthropic.com/news/claude-sonnet-4-6, 2026.

[5] Yuan Cao, Dezhi Ran, Mengzhou Wu, Yuzhe Guo, Xin Chen, Ang Li, Gang Cao, Gong Zhi, Hao Yu, Linyi Li, Wei Yang, 和Tao Xie. Gui-genesis：为gui 智能体后训练自动合成具有可验证奖励的高效环境, 2026. URL https://arxiv.org/abs/2602.14093.

[6] Mouxiang Chen, Lei Zhang, Yunlong Feng, Xuwu Wang, Wenting Zhao, Ruisheng Cao, Jiaxi Yang, Jiawei Chen, Mingze Li, Zeyao Ma, Hao Ge, Zongmeng Zhang, Zeyu Cui, Dayiheng Liu, Jingren Zhou, Jianling Sun, Junyang Lin, 和Binyuan Hui. Swe-universe：将真实世界可验证环境扩展至百万级, 2026. URL https://arxiv.org/abs/2602.02361.

[7] Kanishk Gandhi, Shivam Garg, Noah D. Goodman, 和Dimitris Papailiopoulos. Endless terminals：扩展终端智能体的强化学习环境, 2026. URL https://arxiv.org/abs/2601.16443.

[8] Alexander Golubev, Maria Trofimova, Sergei Polezhaev, Ibragim Badertdinov, Maksim Nekrashevich, Anton Shevtsov, Simon Karasik, Sergey Abramov, Andrei Andriushchenko, Filipp Fisin, Sergei Skvortsov, 和Boris Yangel. 通过强化学习训练长上下文、多轮软件工程智能体, 2025. URL https://arxiv.org/abs/2508.03501.

[9] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Hanwei Xu, Honghui Ding, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jiashi Li, Jingchang Chen, Jingyang Yuan, Jinhao Tu, Junjie Qiu, Junlong Li, J. L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaichao You, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingxu Zhou, Meng Li, Miaojun Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiushi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, S. S. Li, Shuang Zhou, Shaoqing Wu, Tao Yun, Tian Pei, Tianyu Sun, T. Wang, Wangding Zeng, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, W. L. Xiao, Wei An, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, X. Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha Chen, Xiaowen Sun, Xiaoxiang Wang, Xinnan Song, Xinyi Zhou, Xianzu Wang, Xinxia Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Yang Zhang, Yanhong Xu, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Yu, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yuan Ou, Yuduan Wang, Yue Gong, Yuheng Zou, Yujia He, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Y. X. Zhu, Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Ying Tang, Yukun Zha, Yuting Yan, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhicheng Ma, Zhigang Yan, Zhiyu Wu, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Zizheng Pan, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, 和Zhen Zhang. Deepseek-r1：通过强化学习激励大语言模型推理. *自然*, 645(8081):633–638, 9 2025. ISSN 1476-4687. doi: 10.1038/s41586-025-09422-z. URL http://dx.doi.org/10.1038/s41586-025-09422-z.

[10] Naman Jain, Jaskirat Singh, Manish Shetty, Liang Zheng, Koushik Sen, 和Ion Stoica. R2e-gym：借助程序化环境与混合验证器扩展开源权重软件工程智能体, 2025. URL https://arxiv.org/abs/2504.07164.

[11] Jiarong Liang, Zhiheng Lyu, Zijie Liu, Xiangchao Chen, Ping Nie, Kai Zou, 和Wenhu Chen. Swenext：面向智能体的可规模化真实世界软件工程任务, 2026. URL https://arxiv.org/abs/2603.20691.

[12] Dunjie Lu, Yiheng Xu, Junli Wang, Haoyuan Wu, Xinyuan Wang, Zekun Wang, Junlin Yang, Hongjin Su, Jixuan Chen, Junda Chen, Yuchen Mao, Jingren Zhou, Junyang Lin, Binyuan Hui, 和 Tao Yu. VideoAgentTrek：从无标注视频中进行计算机使用预训练, 2025. URL https://arxiv.org/abs/2510.19488.

[13] Jiayi Pan, Xingyao Wang, Graham Neubig, Navdeep Jaitly, Heng Ji, Alane Suhr, 和Yizhe Zhang. 使用swe-gym 训练软件工程智能体与验证器, 2025. URL https://arxiv.org/abs/2412.21139.

[14] Renjie Pi, Grace Lam, Mohammad Shoeybi, Pooya Jannaty, Bryan Catanzaro, 和Wei Ping. 关于扩展大语言模型终端能力的数据工程, 2026. URL https://arxiv.org/abs/2602.21193.

[15] Qwen 团队. Qwen3.5 模型集合. https://huggingface.co/collections/Qwen/qwen35, 2026.

[16] Ram Ramrakhya, Andrew Szot, Omar Attia, Yuhao Yang, Anh Nguyen, Bogdan Mazoure, Zhe Gan, Harsh Agrawal, 和Alexander Toshev. 通过探索扩展智能体合成任务生成规模, 2025. URL https://arxiv.org/abs/2509.25047.

[17] SGLang Project. SGLang：高效执行结构化语言模型程序. https://github.com/sgl-project/sglang, 2024.

[18] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, 和Daya Guo. Deepseekmath：拓展开放语言模型的数学推理极限, 2024. URL https://arxiv.org/abs/2402.03300.

[19] Zeyi Sun, Ziyu Liu, Yuhang Zang, Yuhang Cao, Xiaoyi Dong, Tong Wu, Dahua Lin, 和Jiaqi Wang. Seagent：从经验中自主学习的自演化计算机使用智能体, 2025. URL https://arxiv.org/abs/2508.04700.

[20] verl Project. verl：面向大语言模型的火山引擎强化学习框架. https://github.com/verl-project/verl, 2024.

[21] Bowen Wang, Xinyuan Wang, Jiaqi Deng, Tianbao Xie, Ryan Li, Yanzhe Zhang, Junli Wang, Dunjie Lu, Zicheng Gong, Gavin Li, Toh Jing Hua, Wei-Lin Chiang, Ion Stoica, Diyi Yang, Yu Su, Yi Zhang, Zhiguo Wang, Victor Zhong, 和Tao Yu. Computer agent arena：迈向以人为中心的计算机使用智能体评估与分析. 载于*国际学习表征会议（ICLR）*, 2026. URL https://openreview.net/forum?id=3x4SDbXbgl.

[22] Haoming Wang, Haoyang Zou, Huatong Song, Jiazhan Feng, Junjie Fang, Junting Lu, Longxiang Liu, Qinyu Luo, Shihao Liang, Shijue Huang, Wanjun Zhong, Yining Ye, Yujia Qin, Yuwen Xiong, Yuxin Song, Zhiyong Wu, Aoyan Li, Bo Li, Chen Dun, Chong Liu, Daoguang Zan, Fuxing Leng, Hanbin Wang, Hao Yu, Haobin Chen, Hongyi Guo, Jing Su, Jingjia Huang, Kai Shen, Kaiyu Shi, Lin Yan, Peiyao Zhao, Pengfei Liu, Qinghao Ye, Renjie Zheng, Shulin Xin, Wayne Xin Zhao, Wen Heng, Wenhao Huang, Wenqian Wang, Xiaobo Qin, Yi Lin, Youbin Wu, Zehui Chen, Zihao Wang, Baoquan Zhong, Xinchun Zhang, Xujing Li, Yuanfan Li, Zhongkai Zhao, Chengquan Jiang, Faming Wu, Haotian Zhou, Jinlin Pang, Li Han, Qi Liu, Qianli Ma, Siyao Liu, Songhua Cai, Wenqi Fu, Xin Liu, Yaohui Wang, Zhi Zhang, Bo Zhou, Guoliang Li, Jiajun Shi, Jiale Yang, Jie Tang, Li Li, Qihua Han, Taoran Lu, Woyu Lin, Xiaokang Tong, Xinyao Li, Yichi Zhang, Yu Miao, Zhengxuan Jiang, Zili Li, Ziyuan Zhao, Chenxin Li, Dehua Ma, Feng Lin, Ge Zhang, Haihua Yang, Hangyu Guo, Hongda Zhu, Jiaheng Liu, Junda Du, Kai Cai, Kuanye Li, Lichen Yuan, Meilan Han, Minchao Wang, Shuyue Guo, Tianhao Cheng, Xiaobo Ma, Xiaojun Xiao, Xiaolong Huang, Xinjie Chen, Yidi Du, Yilin Chen, Yiwen Wang, Zhaojian Li, Zhenzhu Yang, Zhiyuan Zeng, Chaolin Jin, Chen Li, Hao Chen, Haoli Chen, Jian Chen, Qinghao Zhao, 和Guang Shi. Ui-tars-2 技术报告：通过多轮强化学习提升gui 智能体, 2025. URL https://arxiv.org/abs/2509.02544.

[23] Xinyuan Wang, Bowen Wang, Dunjie Lu, Junlin Yang, Tianbao Xie, Junli Wang, Jiaqi Deng, Xiaole Guo, Yiheng Xu, Chen Henry Wu, Zhennan Shen, Zhuokai Li, Ryan Li, Xiaochuan Li, Junda Chen, Boyuan Zheng, Peihang Li, Fangyu Lei, Ruisheng Cao, Yeqiao Fu, Dongchan Shin, Martin Shin, Jiarui Hu, Yuyan Wang, Jixuan Chen, Yuxiao Ye, Danyang Zhang, Dikang Du, Hao Hu, Huarong Chen, Zaida Zhou, Haotian Yao, Ziwei Chen, Qizheng Gu, Yipu Wang, Heng Wang, Diyi Yang, Victor Zhong, Flood Sung, Y. Charles, Zhilin Yang, 和Tao Yu. Opencua：计算机使用智能体的开放基础, 2025. URL https://arxiv.org/abs/2508.09123.

[24] WebArena Team. WebArena-infinity. https://webarena.dev/webarena-infinity/, 2025.

[25] Yifan Wu, Yiran Peng, Yiyu Chen, Jianhao Ruan, Zijie Zhuang, Cheng Yang, Jiayi Zhang, Man Chen, Yenchi Tseng, Zhaoyang Yu, Liang Chen, Yuyao Zhai, Bang Liu, Chenglin Wu, 和Yuyu Luo. Autowebworld：通过有限状态机合成无限可验证网页环境, 2026. URL https://arxiv.org/abs/2602.14296.

[26] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, 和Tao Yu. OSWorld：在真实计算机环境中评测多模态智能体的开放式任务能力, 2024. URL https://arxiv.org/abs/2404.07972.

[27] Tianbao Xie, Jiaqi Deng, Xiaochuan Li, Junlin Yang, Haoyuan Wu, Jixuan Chen, Wenjing Hu, Xinyuan Wang, Yuhui Xu, Zekun Wang, Yiheng Xu, Junli Wang, Doyen Sahoo, Tao Yu, 和Caiming Xiong. 通过用户界面分解与合成扩展计算机使用定位能力, 2025. URL https://arxiv.org/abs/2505.13227.

[28] Tianbao Xie, Mengqi Yuan, Danyang Zhang, Xinzhuang Xiong, Zhennan Shen, Zilong Zhou, Xinyuan Wang, Yanxu Chen, Jiaqi Deng, Junda Chen, Bowen Wang, Haoyuan Wu, Jixuan Chen, Junli Wang, Dunjie Lu, Hao Hu, 和Tao Yu. OSWorld-verified 简介. *xlang.ai*, July 2025. URL https://xlang.ai/blog/osworld-verified.

[29] Yiheng Xu, Dunjie Lu, Zhennan Shen, Junli Wang, Zekun Wang, Yuchen Mao, Caiming Xiong, 和Tao Yu. Agenttrek：利用网页教程引导回放来合成智能体轨迹, 2025. URL https://arxiv.org/abs/2412.09605.

[30] Yiheng Xu, Zekun Wang, Junli Wang, Dunjie Lu, Tianbao Xie, Amrita Saha, Doyen Sahoo, Tao Yu, 和Caiming Xiong. Aguvis：用于自主gui 交互的统一纯视觉智能体, 2025. URL https://arxiv.org/abs/2412.04454.

[31] Taofeng Xue, Chong Peng, Mianqiu Huang, Linsen Guo, Tiancheng Han, Haozhe Wang, Jianing Wang, Xiaocheng Zhang, Xin Yang, Dengchang Zhao, Jinrui Ding, Xiandi Ma, Yuchen Xie, Peng Pei, Xunliang Cai, 和Xipeng Qiu. Evocua：通过学习可规模化合成经验来演化计算机使用智能体, 2026. URL https://arxiv.org/abs/2601.15876.

[32] Chenyu Yang, Shiqian Su, Shi Liu, Xuan Dong, Yue Yu, Weijie Su, Xuehui Wang, Zhaoyang Liu, Jinguo Zhu, Hao Li, Wenhai Wang, Yu Qiao, Xizhou Zhu, 和Jifeng Dai. Zerogui：以零人力成本自动开展在线gui 学习, 2025. URL https://arxiv.org/abs/2505.23762.

[33] John Yang, Kilian Lieret, Carlos E. Jimenez, Alexander Wettig, Kabir Khandpur, Yanzhe Zhang, Binyuan Hui, Ofir Press, Ludwig Schmidt, 和Diyi Yang. Swe-smith：扩展软件工程智能体的数据规模, 2025. URL https://arxiv.org/abs/2504.21798.

[34] Yuhao Yang, Zhen Yang, Zi-Yi Dou, Anh Nguyen, Keen You, Omar Attia, Andrew Szot, Michael Feng, Ram Ramrakhya, Alexander Toshev, Chao Huang, Yinfei Yang, 和Zhe Gan. UltraCUA：采用混合动作的计算机使用智能体基础模型, 2025. URL https://arxiv.org/abs/2510.17790.

[35] Zhiyuan Zeng, Hamish Ivison, Yiping Wang, Lifan Yuan, Shuyue Stella Li, Zhuorui Ye, Siting Li, Jacqueline He, Runlong Zhou, Tong Chen, Chenyang Zhao, Yulia Tsvetkov, Simon Shaolei Du, Natasha Jaques, Hao Peng, Pang Wei Koh, 和Hannaneh Hajishirzi. RLVE：利用自适应可验证环境扩展语言模型强化学习, 2025. URL https://arxiv.org/abs/2511.07317.

[36] Ziyun Zhang, Zezhou Wang, Xiaoyi Zhang, Zongyu Guo, Jiahao Li, Bin Li, 和Yan Lu. Infiniteweb：面向gui 智能体训练的可规模化网页环境合成, 2026. URL https://arxiv.org/abs/2601.04126.

[37] Chujie Zheng, Shixuan Liu, Mingze Li, Xiong-Hui Chen, Bowen Yu, Chang Gao, Kai Dang, Yuqiong Liu, Rui Men, An Yang, Jingren Zhou, 和Junyang Lin. 组序列策略优化, 2025. URL https://arxiv.org/abs/2507.18071.

[38] 美国劳工部就业与培训管理局. O*NET 在线平台. https://www.onetonline.org/, 2026. 访问日期：2026-05-06.

附录目录

A 数据合成流水线细节 **15**

A.1 流水线伪代码 15

A.2 任务生成 15

A.2.1 功能分类树构建 15

A.2.2 场景矩阵与采样 15

A.2.3 三轮生成：广度覆盖、缺口补齐与边界案例 . . . 15
A.2.4 防重复规则 . . . 15
A.2.5 覆盖保证与定量目标 . . . 16
A.2.6 完整示例：LibreOffice Calc 格式设置功能分类树 . . . 16
A.3 生成器—判别器对抗循环 . . . 17
A.3.1 五项一致性条件（形式化定义） . . . 17
A.3.2 信息隔离屏障：沙箱与访问矩阵 . . . 17
A.3.3 反投机静态扫描的禁用模式列表 . . . 18
A.3.4 循环终止、最大轮次与反馈协议 . . . 18
A.4 过滤模块 . . . 18
A.4.1 LLM 多数投票：投票者、评判准则与聚合 . . . 18
A.4.2 使用视觉语言模型评判器的教师模型轨迹验证 . . . 18
A.4.3 各阶段过滤保留情况 . . . 19
A.5 领域技能文件 . . . 19
A.5.1 SKILL.md 标准结构 . . . 19
A.5.2 SKILL.md 自动生成流水线 . . . 19
A.5.3 示例：领域SKILL.md 摘录 . . . 19
A.6 输出包与OSWorld 兼容性 . . . 21
A.6.1 每个经验证任务的文件布局 . . . 21
A.6.2 config.json 模式与评估器契约 . . . 21

B 仿真网页环境合成 22
B.1 环境选择方法 . . . 22
B.1.1 将O*NET SOC 职业大类映射到网页应用类别 . . . 22
B.1.2 Anthropic 经济指数加权 . . . 22
B.1.3 覆盖截断标准 . . . 22
B.2 合成仿真应用完整列表 . . . 22
B.2.1 逐应用清单 . . . 22
B.2.2 真实应用参考映射与保真度说明 . . . 24
B.2.3 商标、商业外观与素材合规 . . . 24
B.2.4 逐仿真应用代码行数与端点统计 . . . 25
B.3 统一状态API 规格 . . . 25
B.3.1 端点契约 . . . 25
B.3.2 通过sid 实现会话隔离 . . . 26
B.3.3 状态差异计算规则 . . . 26
B.3.4 与reward.py 集成 . . . 26
B.4 多代理合成流水线 . . . 26
B.4.1 规划代理：网络调研与DESIGN.md . . . 26
B.4.2 开发代理：实现协议 . . . 26
B.4.3 网页代理：使用Playwright 对照UI 树验证 . . . 27
B.4.4 迭代收敛循环 . . . 27
B.5 代表性仿真应用案例研究 . . . 27

B.5.1 Slack 仿真应用：状态模式与功能清单 . . . 27
B.5.2 Jira 仿真应用：跨实体状态转换 . . . 27
B.5.3 Salesforce 仿真应用：表单密集型工作流 . . . 27

C 训练细节 **27**
C.1 智能体运行框架 . . . 27
C.1.1 观测与动作空间 . . . 27
C.1.2 轨迹切片方案 . . . 28
C.1.3 推理时配置 . . . 29
C.1.4 系统提示词与工具定义 . . . 29
C.2 SFT 预热 . . . 30
C.2.1 教师轨迹筛选 . . . 30
C.2.2 SFT 超参数 . . . 30
C.3 GSPO 训练 . . . 30
C.3.1 超参数 . . . 30
C.3.2 参考策略更新安排 . . . 30
C.3.3 优势归一化细节 . . . 31
C.4 基础设施 . . . 31
C.4.1 硬件与集群配置 . . . 31
C.4.2 轨迹采样的虚拟机供给 . . . 31
C.4.3 每次训练的实际耗时与成本 . . . 31
C.5 可复现性检查清单 . . . 31

D 扩展实验 **31**
D.1 涌现的多动作行为 . . . 31
D.1.1 最常见的K 类批量动作模式 . . . 31
D.1.2 避免批量执行的非确定性动作类别 . . . 32

E 定性示例 **32**
E.1 端到端任务详解 . . . 32
E.1.1 桌面任务——LibreOffice Calc：编号补零公式 . . . 32
E.1.2 桌面任务——VS Code：自定义文件关联 . . . 37
E.1.3 跨应用任务——Slack + Impress + PDF：优先级排序演示文稿 . . . 41
E.1.4 跨应用任务——PDF → Calc：多币种费用分析 . . . 48
E.2 完整采样轨迹：Shopify 供应商整合 . . . 56
E.3 信息隔离屏障拦截的奖励投机案例 . . . 68
E.4 对抗循环迭代记录 . . . 69
E.5 奖励函数示例集 . . . 70

F 数据集统计 **71**
F.1 分布细分 . . . 71
F.1.1 各环境的任务数量 . . . 71
F.1.2 领域 × 难度矩阵 . . . 72

F.1.3 动作动词频率 72
F.1.4 跨应用对共现 72
F.1.5 指令长度分布 73
F.2 生成成本 73
F.2.1 每个元组的词元消耗 73
F.2.2 每个已验证元组的虚拟机时数 73

G 提示词目录 **73**
G.1 提示词工程说明 73
G.2 编排器系统提示词 74
G.3 生成器子智能体系统提示词 75
G.4 判别器子智能体系统提示词 78
G.5 五项一致性条件的验证器提示词 79
G.6 筛选评审器提示词 79
G.7 仿真应用合成智能体提示词 80

A 数据合成流水线细节

A.1 流水线伪代码

算法 1 概述了端到端的协同生成流水线。编排器配置成对的虚拟机，并在§2.1 所述的信息隔离屏障下调度生成器和判别器。循环持续迭代，直到五项一致性条件（§A.3.1）全部成立，或耗尽最大轮次预算；通过循环的元组还需接受§A.4 所述的数据集级过滤。

算法 1 CUA-Gym 协同生成流水线。

Input: task instruction t, context c, domain skill $\mathcal{S}_{\text{dom}}$, max rounds K
Output: verified tuple $(t, s_{\text{init}}, s_{\text{gold}}, r)$ or $\perp$
Provision isolated VMs $V_{\text{init}}, V_{\text{gold}}$
for $k = 1, \ldots, K$ **do**
  Generator reads $(t, c, \mathcal{S}_{\text{dom}})$ and writes `initial_setup.py`, `golden_patch.py`
  Execute `initial_setup.py` on V_{init}; execute `golden_patch.py` on V_{gold}
  Discriminator reads $(t, c, \mathcal{S}_{\text{dom}})$ **only** (*information barrier*) and writes `reward.py`
  Static scan `reward.py` against forbidden-pattern list (§A.3.3)
  Evaluate $r_{\text{init}} \leftarrow r(V_{\text{init}})$, $r_{\text{gold}} \leftarrow r(V_{\text{gold}})$
  if all five agreement conditions of §A.3.1 hold **then**
    break
  else
    Discriminator emits structured feedback $\mathcal{F}_k \rightarrow$ Generator
  end if
end for
if convergence not reached **then return** $\perp$
end if
Submit $(t, s_{\text{init}}, s_{\text{gold}}, r)$ to the dataset-level filter (§A.4)
return verified tuple if filter accepts, else $\perp$

A.2 任务生成

A.2.1 功能分类树构建

对于每个应用，我们构建一棵功能分类树：内部节点将相关能力归为一组（例如，`libreoffice_calc` → `formatting` → `conditional_formatting`），叶节点则对应原子的UI 能力。分类树以应用官方文档为起点，通过一轮由LLM 驱动的网络调研构建，使功能清单以真实使用模式为依据。为兼顾覆盖度和采样器的可处理性，树的深度限制为 ≤ 4，任一节点的分支数限制为 ≤ 12。每个叶节点均附有一行能力描述和涉及的UI 元素列表；任务合成时，这两项信息都会提供给生成器。

A.2.2 场景矩阵与采样

在分类体系之上，任务沿五维矩阵进行采样：*平台*（桌面 / 网页 / 跨平台）、*领域*（与O*NET 对齐的10 个类别）、*难度*（简单 / 中等 / 困难，其操作性定义为：简单 ≤ 3 个原子动作，中等 3–10 步，困难需要长时程规划）、*场景*（商业 / 教育 / 科学 / 个人），以及*任务类型*（单应用 / 跨应用）。采样器持续记录各单元格的计数，并使用逆频率加权，使后续采样倾向于覆盖不足的单元格。每个应用的最低数量作为硬约束执行，以避免小众应用构成的长尾得不到足够样本。

A.2.3 三轮生成：广度覆盖、缺口补齐与边界案例

生成过程按顺序遍历矩阵三轮。**第 1 轮（广度覆盖）**在分类体系上均匀采样，以最大化功能覆盖；调整产出规模，使每个叶节点至少获得 25 个任务。**第 2 轮（缺口补齐）**根据对第1 轮输出的事后覆盖分析，重新采样代表性不足的矩阵单元格。这一轮有意提高跨应用任务和困难任务的采样权重。**第 3 轮（边界案例）**针对§A.4 循环失败分析所识别的边界条件：处于单次尝试可解性极限的任务、以非常规顺序组合多个应用的任务，以及对罕见动作原语（拖拽、组合快捷键、多窗口导航）进行压力测试的任务。

A.2.4 防重复规则

在每轮内部及各轮之间，通过三项检查对生成的指令去重。首先，利用句子嵌入的余弦相似度阈值（≥ 0.85），相对于不断增长的语料库标记近重复内容。其次，采用词元级 4-gram 重叠检查，拒绝与任一先前任务共享超过 50% 的 4-gram 的任务指令。第三，槽位模板多样性规则将任一指令模板（例如“对列 Y 应用X 格式”）在每个应用中的实例数量限制为至多 3，迫使生成器变换表述形式。未通过任何一项检查的任务都会附带“增加多样性”的指令返回生成器重新合成。

A.2.5 覆盖保证与定量目标

采样器设置以下定量目标，并将其作为硬约束执行：

- 最终数据集中任一领域类别的占比不超过 21%（图 6b 中的经验结果与此相符）。
- 困难任务至少占语料库的 40%。
- 跨应用任务至少占语料库的 35%。
- 每个应用至少获得 1,000 个经验证的元组；任一应用的占比不超过语料库的 15%。

当所有矩阵单元格均超过各自的最低数量，且已验证元组总数超过项目预算时，采样器终止。

A.2.6 完整示例：LibreOffice Calc 格式设置功能分类树

生成器根据应用的文档索引和功能区清单构建`libreoffice_calc` 功能分类体系。清单 A.2.6 展示了聚焦于`formatting` 子树的精简版本；受篇幅所限，其余分类（数据导入 / 导出、公式、图表、数据透视表、工作表结构、保护、引用、脚本）已省略。叶节点的范围足够细，使单条任务指令可对应一到两个相邻原语；我们为每个叶节点列出两个示例任务，以说明§A.2.4 中的叶节点内多样性规则。

精简功能树：`libreoffice_calc` > formatting

```
libreoffice_calc/
|-- formatting/
|   |-- cell_appearance/
|   |   |-- number_format/
|   |   |   - "将 D 列设置为货币格式，使用 EUR 符号、
|   |   |      两位小数和千位分隔符。"
|   |   |   - "为 'Days Open' 列应用自定义数字格式，
|   |   |      使 >30 的数值以红色显示，
|   |   |      且不显示负号。"
|   |   |-- date_time_format/
|   |   |   - "将日期列的 ISO 'YYYY-MM-DD' 格式改为
|   |   |      'DD MMM YYYY'（例如 03 Mar 2026）。"
|   |   |   - "在 'Logs' 工作表中，设置 A 列的格式，
|   |   |      使时间戳显示星期名称及 24 小时制时间，
|   |   |      例如 'Mon 14:32'。"
|   |   |-- conditional_format/
|   |   |   - "使用浅红色填充，突出显示 'Q4 Revenue' 中
|   |   |      低于该列平均值的单元格。"
|   |   |   - "在 'Score' 列上添加红—黄—绿三色渐变，
|   |   |      按该列最小值 / 最大值缩放。"
|   |   |-- borders_and_fills/
|   |       - "为 B2:F18 表格应用细黑色外边框，
|   |          内部网格线使用点线。"
|   |       - "为 'Forecast' 表格的交替行填充
|   |          5\% 灰色，以提高可读性。"
|   |-- alignment_and_wrapping/
|   |   |-- horizontal_vertical/
|   |   |   - "将第 1 行所有表头单元格居中对齐，
|   |   |      并将所有数值列右对齐。"
|   |   |   - "将合并标题单元格 A1:F1 的垂直对齐
|   |   |      设置为 'middle'。"
|   |   |-- text_wrap_indent/
|   |       - "为 C 列启用自动换行，并将子类别行
|   |          缩进一级。"
|   |       - "将 B 列配置为缩小字体填充，使较长的
|   |          产品名称能够容纳在单元格中而不换行。"
|   |-- typography/
|   |   |-- font_face_size/
|   |   |   - "将 'Summary' 工作表整体改为 Liberation Sans
|   |   |      11pt，标题保持 16pt 加粗。"
|   |   |   - "为 'SQL' 列应用 9pt 等宽字体，
|   |   |      使查询文本在视觉上对齐。"
```

```
|   |   |-- emphasis/
|   |       - "加粗总计行，将差异行设为斜体，
|   |          并为列标题添加下划线。"
|   |       - "对于 A 列中状态单元格等于 'cancelled' 的
|   |          每条项目记录，添加删除线。"
|   |-- styles_and_themes/
|       |-- named_styles/
|       |   - "定义自定义单元格样式 'KPI-positive'（绿色文本、
|       |      加粗），并将其应用于 G 列所有正增量。"
|       |   - "将 'Q3' 工作表中所有使用 'Default' 样式的
|       |      位置替换为内置 'Heading-2' 样式。"
|       |-- table_styles/
|           - "将 A1:F50 区域转换为带样式的表格，
|              使用内置 'Tabular Light Blue' 样式。"
|           - "为 'Forecast' 表格改用条纹样式，
|              并冻结首行，使表头始终可见。"
```

完整的`libreoffice_calc`分类树共包含147个叶节点，分布于9个顶层子树；仅`formatting`就占约$\sim$21个叶节点。第1轮采样以每个叶节点两条任务指令为目标，满足逐叶节点的最低覆盖要求（§A.2.5）；随后，第2 / 3轮提高代表性不足的叶节点及困难边界条件的权重。

A.3 生成器—判别器对抗循环

A.3.1 五项一致性条件（形式化定义）

当且仅当以下五项条件同时成立时，循环接受元组$(t, s_{\text{init}}, s_{\text{gold}}, r)$：

C1. 初始状态可执行性：`initial_setup.py`在V_{init}上无异常地运行至结束。

C2. 标准完成状态可执行性：`golden_patch.py`在V_{gold}上无异常地运行至结束。

C3. 标准完成状态奖励：$r(V_{\text{gold}}) = 1.0$。

C4. 初始状态奖励：$r(V_{\text{init}}) = 0.0$。

C5. 奖励完整性：`reward.py`不包含禁用模式列表（§A.3.3）中的任何模式。

C1–C2确保环境产物结构正确；C3–C4确保奖励函数在两个端点状态上沿正确方向表现出敏感性；C5确保奖励验证的是任务语义，而非产物身份。五项条件的合取是循环层面接受元组的充要条件，由编排器根据判别器在结构化`REVIEW.md`中给出的判定进行检查。

A.3.2 信息隔离屏障：沙箱与访问矩阵

信息隔离屏障通过生成器进程与判别器进程之间的进程级隔离实现。每个进程在独立工作目录中执行，拥有彼此不相交的文件系统视图；进程间通信只能经由编排器中转。表1概述了所执行的访问权限。

Resource	Generator	Discriminator
Task instruction t, context c	✓	✓
Domain skill file $\mathcal{S}_{\text{dom}}$	✓	✓
`initial_setup.py` (Generator output)	write	**denied**
`golden_patch.py` (Generator output)	write	**denied**
Generator working directory	full	**denied**
V_{init} post-setup, via state-only API	read	read
V_{gold} post-patch, via state-only API	read	read
File-system contents of V_{init}, V_{gold}	write	**denied** (read via API only)
`reward.py` (Discriminator output)	**denied**	write
`REVIEW.md` (verdict + feedback)	read	write

表1: 用于执行生成器与判别器之间信息隔离屏障的访问矩阵。判别器仅能获得自然语言对(t, c)以及两台虚拟机的纯状态视图这两类任务特定信号，从而消除§2.1所述的逆向工程捷径。

A.3.3 反投机静态扫描的禁用模式列表

判别器生成的`reward.py`在写入时接受以下模式列表的扫描。任何匹配都会中止当前轮次，并以匹配到的模式为反馈，向判别器发送结构化的重新提示。

1. 不经计算就对验证标志**直接赋布尔值**，例如`chart_verified = True`。
2. **占位验证**：先将一个标志赋为常量，再有条件地将其计入分数，中间没有任何求值过程。
3. **硬编码成功**：函数在成功路径上返回 $\{0.5, 1.0\}$ 中的常量，却不检查环境。
4. **仅按存在性给分**：仅凭`os.path.exists(...)`就给予正分，不检查文件内容的任何属性。
5. **使用`subprocess`**：奖励调用外部进程，而这些进程不可复现且容易被伪造。
6. **仅注释式验证**：加分之前只有一条声称已检查的注释（`# 假定X 正确`），却没有执行检查的代码。

该列表通过正则表达式匹配和Python AST 遍历相结合的方式执行；完整源模式随代码一并发布。

A.3.4 循环终止、最大轮次与反馈协议

循环上限为 $K = 5$ 轮。每个失败轮次都会由判别器生成结构化`REVIEW.md`，列出未满足的条件、观测到的奖励值 $(r_{\text{init}}, r_{\text{gold}})$、匹配到的禁用模式（若有），以及自由文本建议。生成器将`REVIEW.md`作为唯一反馈信号，修订其脚本。如果第 K 轮仍失败，该元组将被拒绝并记录下来供离线分析。我们有意避免增大 K：经验上，到第 5 轮仍未收敛的元组主要由含糊任务指令和不可解规格造成，再多脚本修订也无法解决；将这些问题反馈给缺口补齐轮次（§A.2.3）比增加内循环迭代更具成本效益。

A.4 过滤模块

A.4.1 LLM 多数投票：投票者、评判准则与聚合

被循环接受的元组进入数据集级过滤器，该过滤器通过由 V 个异构评审器组成的集成进行LLM 多数投票。每个评审器接收四元组 $(t, s_{\text{init}}, s_{\text{gold}}, r)$，依据模式 ⟨`verdict, severity, can_fix_with_query_only, query_issues, setup_reward_risks, training_pool_fit, confidence, reasoning_summary, revised_query`⟩ 输出结构化JSON 判定，并遵循§G.6 提示词中默认保守的决策策略。集成中的投票者来自相互独立的基础模型家族，以降低特定家族偏差之间的相关性。

严重性评判准则（每位投票者）。

- **P0**（拒绝）：致命的初始化 / 奖励 / 环境问题；(t, s, r) 之间存在无法通过修订查询来修复的不匹配。
- **P1**（修改查询）：查询缺少必要上下文、存在有害歧义，或泄露流程信息而使评估产生偏差。
- **P2**（保留或修改）：非致命质量问题；元组可用，但不理想。
- **P3**（保留）：除表面风格外没有有意义的问题。

聚合规则。 当且仅当严格多数（$\lceil V/2 \rceil + 1$）投票者返回`verdict` $\in \{$`keep, modify_query`$\}$ 时，元组才会进入下一过滤阶段。对于获准进入的元组，依据投票者`verdict`的多数票选择规范指令：如果多数为`modify_query`，则发布指令采用提出修改的投票者中置信度最高的`revised_query`；如果多数为`keep`，则保留原始指令。平票时默认拒绝。集成对每个元组报告的严重性，取所有投票者所赋严重性中的最高值，从而在发布元数据中提供兼顾最坏情形的质量等级。

A.4.2 使用视觉语言模型评判器的教师模型轨迹验证

通过评审器筛选的元组还需进入教师轨迹验证阶段。我们针对每个元组使用强教师策略（Claude-Sonnet-4-6 [4]）执行 N_{teach} 次轨迹采样试验，对每条轨迹进行两次评分：一次使用`reward.py`实现的程序化奖励 $r(s, \tau) \in [0, 1]$，另一次使用独立视觉语言模型评判器给出的分数 $\hat{r}_{\text{vlm}}(s, \tau) \in \{0, 1\}$；该评判器接收最终状态截图、任务指令，以及从 t 提炼的验收标准清单。

为何使用两种分数。 两种分数衡量正确性的不同方面。r 以环境状态（文件内容、结构化状态差异、库内省）为依据，方差较低，但只能观察奖励编写者选择检测的内容。$\hat{r}_{\text{vlm}}$ 以最终视觉状态为依据，可以捕捉程序化奖励遗漏的表面失败（例如UI 加载了正确数据，但配色主题不对；或图表数值正确，但缺少坐标轴标签）。两者一致是奖励函数忠实于任务意图的强信号；持续不一致则表明奖励过于狭窄，或任务本身定义不充分。

纳入标准。 根据平均值 $(\bar{r}, \bar{\hat{r}}_{\text{vlm}})$ 对元组分类，该平均值在 N_{teach} 次试验上计算：

- 两个均值均位于 $(0,1)$ 内，且 $|\bar{r} - \bar{\hat{r}}_{\text{vlm}}| < \delta$：*接受*。
- 两个均值均等于 0：*可能不可解*；从语料库中移除。
- 首次试验中两个均值均等于 1：*可能过于简单*；进行下采样以控制简单任务尾部。
- 均值差异 $\geq \delta$：*标记*；将元组送回判别器，并突出显示分歧区域，作为改进奖励的反馈。

接受集合构成发布的数据集。被标记为存在分歧的元组，在通过判别器改进后重新进入教师轨迹采样；两轮改进后仍未收敛的元组将被放入离线分析集合。

A.4.3 各阶段过滤保留情况

两个过滤阶段拒绝不同的失败模式，对总过滤损失的贡献比例约为70/30。从循环接受的样本池出发，LLM 多数投票阶段（§A.4.1）因P0 初始化 / 奖励缺陷、P1 查询问题或持续的投票分歧，拒绝约 3,100 个元组；这一阶段主要捕捉虽通过循环一致性条件、却未通过端到端审查的静态规格错误。后续教师轨迹采样阶段（§A.4.2）再拒绝 1,278 个元组：教师在 N_{teach} 次试验中无法解决它们，或它们在首次试验中就轻易获得满分；这一阶段主要捕捉只有在现实代理行为下才会显现的不可行性与过度简单性。最终语料库包含正文报告的32,112 个经验证元组。

A.5 领域技能文件

A.5.1 SKILL.md 标准结构

每个应用领域都附带一份`SKILL.md` 文档，供生成器和判别器在合成时加载。标准结构包含六节：（1）领域特定概念和Python 库（例如，`libreoffice_writer` 使用`python-docx`，`libreoffice_calc` 使用`openpyxl`）；（2）状态与文件系统布局，包括文档和配置的规范路径；（3）按最常见任务上下文参数化的`initial_setup.py` 模板；（4）具有对应参数槽位的`golden_patch.py` 模板；（5）展示渐进式部分得分分解的`reward.py` 评分模式；（6）“惨痛教训”，即开发过程中观察到的陷阱精选列表（例如，仿真应用API 中对set 与set_current 的混淆，文档库中的样式继承陷阱）。

A.5.2 SKILL.md 自动生成流水线

SKILL.md 的初稿通过对应用文档的一轮单次调研生成，再通过检查第一轮生成的拒绝日志进行完善，将反复出现的失败模式提炼为新的“惨痛教训”条目。SKILL.md 文件与合成流水线一同进行版本管理；本文所有实验结果均使用发布时冻结的版本。

A.5.3 示例：领域SKILL.md 摘录

我们给出`libreoffice_calc` SKILL.md 的摘录，以具体说明这一格式。其中完整重现了前置元数据、GUI 启动模板（第0 节）和*惨痛教训*列表（第6 节）；受篇幅所限，文件创建、文件读取和奖励模式部分（第1、4、5 节）经过了缩写。

前置元数据与角色。

libreoffice_calc SKILL.md 前置元数据

```
---
name: libreoffice-calc
description: 如何使用 Python openpyxl 以编程方式创建、修改和验证
             LibreOffice Calc (.xlsx) 文件，
             供 setup-gen 和 reward-gen 使用。
---

# LibreOffice Calc -- Python 操作指南

本技能教 setup-gen (创建 / 修改 xlsx) 和 reward-gen
(读取 / 验证 xlsx) 如何通过纯 Python 代码
处理电子表格文件。

  - 库：openpyxl (批量数据处理另用 pandas)
  - 安装：pip3 install openpyxl pandas
```

第0 节——GUI 启动模板。 生成初始工作簿后，initial_setup.py 必须使任务处于GUI 就绪状态。由于在无头虚拟机上，不显式指定显示器而启动GUI 应用会静默失败，因此每次启动都设置DISPLAY=:0，并使用非阻塞的Popen，使脚本能够正常退出：

第0 节——GUI 启动辅助函数

```
import os, shlex, subprocess, time

def launch_gui(command: str, delay_sec: float = 1.0):
    env = os.environ.copy()
    env["DISPLAY"] = ":0"
    subprocess.Popen(
        shlex.split(command),
        stdout=subprocess.DEVNULL,
        stderr=subprocess.DEVNULL,
        env=env,
    )
    time.sleep(delay_sec)

# 为 GUI 代理打开初始工作簿。
launch_gui(f'libreoffice --calc "{output_path}"', delay_sec=2.0)

# 可选的多应用启动，例如用于跨应用任务。
# launch_gui('nautilus "/home/user"', delay_sec=1.0)
```

第1–5 节（缩写）。 第1 节列出initial_setup.py 和golden_patch.py 使用的openpyxl API：工作簿构建、单元格级写入、批量行写入、工作表创建与重排、字体 / 填充 / 对齐 / 边框样式、数字格式、合并单元格、公式、图表（柱形、折线、饼图、散点）、数据验证、自动筛选区域和条件格式。第2 节介绍reward.py 使用的读取API：分别以启用和不启用data_only=True 的方式加载工作簿、遍历单元格、验证样式、读取条件格式规则，以及检测合并单元格区域。第4 节包含按任务类型（数据编辑、格式、公式、图表、布局）参数化的渐进式评分奖励模板；第5 节包含针对常见验证习惯用法的辅助函数（带类型转换的列提取、部分相等性检查、数值容差比较）。全文随发布的流水线提供。

第6 节——惨痛教训。

第6 节——部署过程中吸取的惨痛教训

1. openpyxl 不会计算公式值。cell.value 返回公式字符串 "=SUM(A1:A10)"，而非计算结果。使用 data_only=True 获取最近缓存的值（要求文件至少曾在 Calc/Excel 中打开过一次）。

2. 颜色始终使用 8 字符 ARGB。PatternFill(start_color="4472C4") 会静默变成 "004472C4"（alpha=00，透明）。应写成 "FF4472C4"。读取时，按 8 字符形式比较。

3. 可见背景使用 fgColor，而非 bgColor。cell.fill.fgColor.rgb 给出你看到的背景色；bgColor 通常不是你想要的属性。

4. 合并单元格：只有左上角单元格有数据。在 merge_cells("A1:D1") 后，B1/C1/D1 变为 MergedCell，其 value=None。只为左上角单元格设置样式。

5. 标准完成状态文件应先复制再修改。只要初始文件存在，就不要从头重建。先 shutil.copy(initial, golden)，再打开修改。这样可保留元数据、打印设置和其他不可见属性。

6. openpyxl 无法创建数据透视表，只能读取 / 保留。使用已构建好数据透视表的模板文件。

7. 筛选器仅定义规则。ws.auto_filter.ref = "A1:D20"

```
    设置区域，但实际上不会隐藏行。只有在 LibreOffice
    中打开文件时，筛选才会生效。

8.  showDropDown=False 表示显示下拉菜单。在 DataValidation
    中，此布尔值的含义相反。设为 False 才会显示下拉箭头。

9.  图表类型命名。BarChart(type="col") 为竖向柱形图；
    BarChart(type="bar") 为横向条形图。这与 LibreOffice
    的菜单名称并不一致。

10. 样式赋值后不可变。不能执行 cell.font.bold = True。
    应创建新的 Font 对象：cell.font = Font(bold=True, ...)。
    Fill、Alignment、Border 同理。

11. 主题色的 .rgb 可能返回 None。如果单元格使用主题色
    而非显式 RGB，cell.font.color.rgb 可能是 None 或主题索引。
    读取颜色时应使用 try/except 包裹。

12. data_only=True 会丢失公式。以 data_only=True 加载时，
    公式会被其缓存值替换。若两者都需要，就加载文件两次：
    一次正常加载，一次设置 data_only=True。
```

“惨痛教训”部分是SKILL.md 格式最鲜明的特点。每个条目都源于早期开发中的一次具体调试经历；若没有这些条目，LLM 生成器或判别器往往会浪费一轮对抗迭代，再次遇到同样的陷阱。“惨痛教训”与流水线一同进行版本管理；每当拒绝日志中出现反复发生的失败模式，就会增加新条目。

A.6 输出包与OSWorld 兼容性

A.6.1 每个经验证任务的文件布局

每个经验证的元组均以独立目录的形式输出到`output/final/<task_id>/` 下，布局如下：

单任务输出包布局

```
output/final/<task_id>/
  config.json          # OSWorld 评估器契约
  meta.json            # 任务分类字段、难度、来源轮次
  initial_setup.py     # 在全新虚拟机上生成 s_init
  golden_patch.py      # 在全新虚拟机上生成 s_gold
  reward.py            # 实现 r: state -> [0, 1]
  REVIEW.md            # 最终接受轮次中的判别器判定
  task_config.json     # 自然语言任务指令与上下文
```

该布局无需修改即可供OSWorld 评估器 [26]、我们的SFT 教师轨迹采样流水线，以及§C 的RL 训练器使用。

A.6.2 config.json 模式与评估器契约

`config.json` 完全遵循OSWorld 评估器契约，包含虚拟机镜像、快照标识符、动作观测类型（`a11y_tree` 或`screenshot`）、步数预算，以及指向我们`reward.py` 的执行后评估器入口列表。我们未增加任何模式扩展；与上游OSWorld 评估器 [26] 完全兼容是明确的设计目标，以便将CUA-Gym 元组直接用作OSWorld 基准条目。

B 仿真网页环境合成

B.1 环境选择方法

B.1.1 将O*NET SOC 职业大类映射到网页应用类别

仿真应用套件以O*NET [38] 维护的标准职业分类（SOC）大类为基础。每个SOC 大类对应一组不同的数字工作流；我们将这些工作流簇映射到应用类别，覆盖各职业最常接触的软件。表 2 概述了该映射。这是一个多对多映射：一个SOC 大类可能涉及多个应用类别（例如，管理类职业同时使用CRM 与项目管理工具），而一个应用类别也常服务于多个SOC 大类（例如，通信应用几乎出现在所有知识工作类职业中）。

O*NET SOC major group	Mock application categories represented
Management Occupations	CRM, project management, communication, analytics
Business and Financial Operations	Accounting, payments, marketing analytics, CRM
Computer and Mathematical	Code hosting, CI/CD, monitoring, cloud consoles, infra
Architecture and Engineering	Cloud infrastructure, CAD/diagram, project management
Life, Physical, and Social Science	Document collaboration, paper review, analytics
Community and Social Service	Communication, scheduling, document collaboration
Legal Occupations	Document signing, case management, legal research
Educational Instruction and Library	Learning management, document collaboration, scheduling
Arts, Design, Entertainment, Sports	Design tools, social media, content publishing
Healthcare Practitioners and Technical	EHR systems, medical imaging, scheduling
Sales and Related	CRM, e-commerce admin, marketing analytics
Office and Administrative Support	Email, scheduling, document collaboration, HRIS
Transportation and Material Moving	Booking, travel planning, e-commerce
Government and Civil Service	Government portals, tax, identity verification

表 2: 从O*NET SOC 职业大类到CUA-Gym 所涵盖的仿真应用类别的映射。知识工作密集的职业（管理、商业、计算机、销售）按比例获得更多环境，反映了AEI 加权中的相对软件使用强度（§B.1.2）。

B.1.2 Anthropic 经济指数加权

在每个SOC 组内，应用优先级由Anthropic 经济指数 [3] 加权；该指数提供根据大规模代理流量日志估计的经验软件使用频率。我们提取每个类别的使用频率 f_c，在选定纳入的类别间进行归一化，再将得到的权重 $w_c = f_c / \sum_{c'} f_{c'}$ 用作各类别的合成预算。因此，仿真应用的分配近似于知识工作者在各应用类别上花费的相对时间，而非任一单独职业的应用多样性。

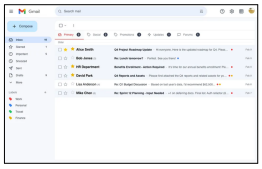
B.1.3 覆盖截断标准

只有当类别 c 的权重 w_c 超过纳入阈值 $w_{\min}$ 时，我们才为其合成仿真应用。阈值经过校准，以避免低权重类别构成的长尾（占加权使用量 $< 1\%$）消耗本可更有效地用于提升类别深度的工程预算。在已纳入的类别中，按使用频率从高到低选择具体应用，直到耗尽该类别预算。由此得到一种在定性意义上较为保守的长尾截断：发布的套件以高保真度覆盖分布头部，而非以低保真度覆盖完整尾部。

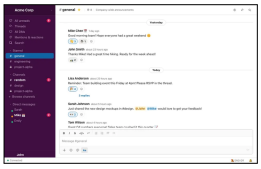
B.2 合成仿真应用完整列表

B.2.1 逐应用清单

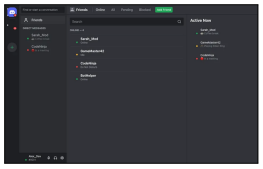
表 3 按类别列出了94 个合成仿真应用。每个仿真应用保留对应真实应用的规范名称作为标识符；实现为独立的单页应用，不依赖对应的真实服务。图 10 展示了跨主要场景类别的 32 个仿真应用的代表性首页截图，说明发布套件的视觉保真度范围。



gmail



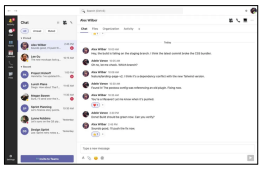
slack



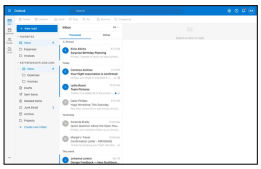
discord



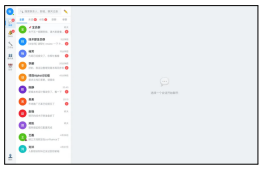
zoom-web



microsoft-teams



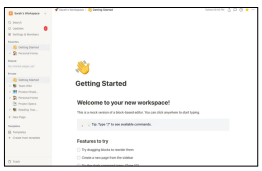
outlook-web



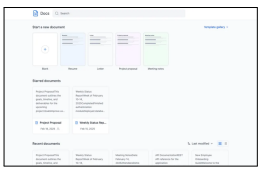
dingtalk



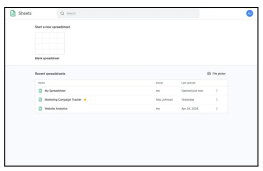
feishu



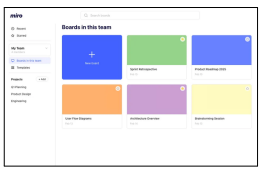
notion



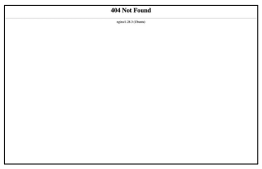
google-docs



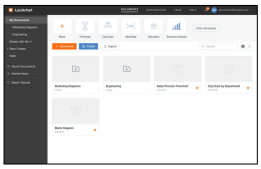
google-sheets



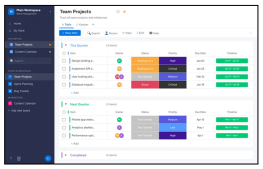
miro



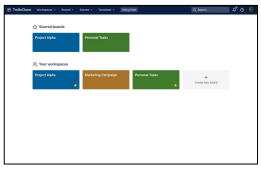
figma



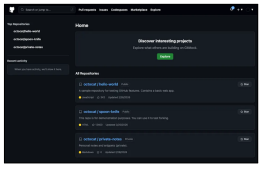
lucidchart



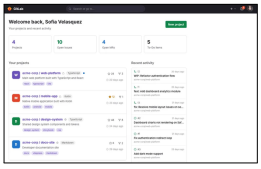
monday



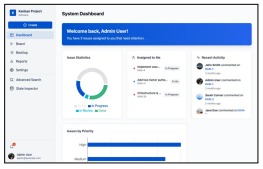
trello



github



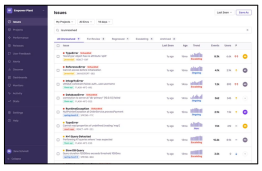
gitlab



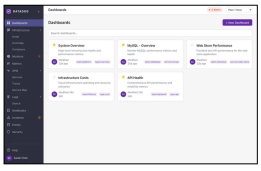
jira



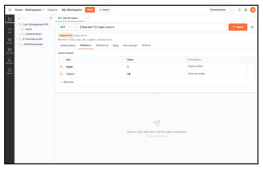
linear



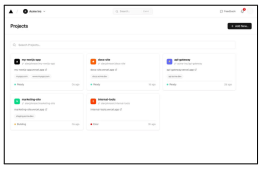
sentry



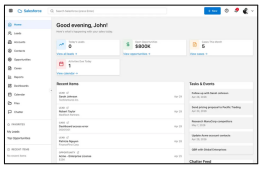
datadog



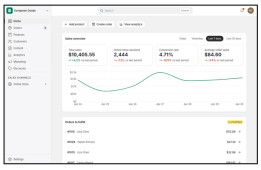
postman



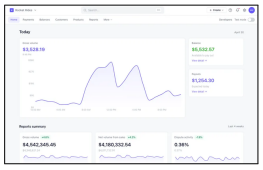
vercel



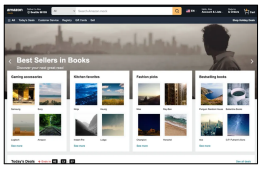
salesforce



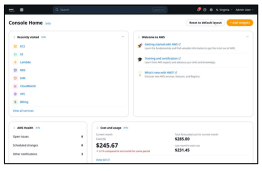
shopify-admin



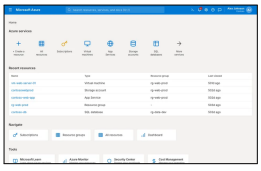
stripe-dashboard



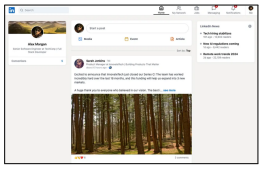
amazon



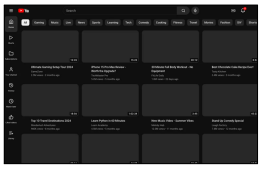
aws-console



azure



linkedin



youtube

图 10: 已发布的 32 个代表性仿真网页应用的首页截图，按主题每两行组织：通信与会议（第 1–2 行）、生产力与协作（第 3–4 行）、工程与开发者工具（第 5–6 行），以及商业 / 云 / 社交（第 7–8 行）。每个仿真应用重现其所命名的参考应用的表面视觉布局、导航树和主要功能清单，不包含认证、第三方集成或外部网络调用（§B.2.2）。已发布的94 个仿真应用的完整清单见表 3。

表 3: 按类别分组的合成仿真网页应用完整清单。

Category	Mock applications
Communication / messaging	slack, microsoft_teams, discord, dingtalk, feishu, wechat, zoom_web, gmail, outlook_web
Social / publishing	twitter, instagram, facebook, linkedin, reddit, weibo, xiaohongshu, zhihu, pinterest, youtube
Project management	jira, asana, trello, monday, linear, notion
Document collaboration	google_docs, google_sheets, google_drive, google_calendar, airtable, miro, lucidchart, openreview
Code hosting / DevOps	github, gitlab, vercel, circleci, sentry, postman, wandb
Cloud / infrastructure	aws_console, azure, aliyun, cloudflare, datadog
E-commerce / marketplace	amazon, ebay, taobao_seller, amazon_seller, shopify_admin, woocommerce, instacart, uber_eats
Travel	booking_com, expedia, tripadvisor, google_flights, 12306
Finance / payments	paypal, robinhood, coinbase, stripe_dashboard, quickbooks, TradingView
HR / hiring	workday, bamboohr, greenhouse, gusto, lattice, adp
CRM / customer service	salesforce, hubspot, Zendesk, ServiceNow, SAP
Marketing / ads	hubspot_marketing, klaviyo, mailchimp, meta_ads, google_ads
Analytics	google_analytics, mixpanel, amplitude, hotjar, looker_studio, tableau
Healthcare / legal	epic-health, PACS-viewer, clio, westlaw, contractbook, docusign, Expensify
Government / education	california_tax, usa-gov, visa_portal_ds160, Canvas-LMS
Real estate / utilities	zillow

B.2.2 真实应用参考映射与保真度说明

每个仿真应用都保留其所命名的参考应用的表面视觉布局、导航树与主要功能清单。仿真应用在三个方面有意区别于参考应用：(i) 移除认证（无登录流程、无OAuth、无SSO）；(ii) 所有数据均位于会话本地，并通过合成方式初始化；(iii) 外部网络调用被替换为进程内状态修改。如果参考应用的能力依赖这些被去除的层（例如，实时推送通知、跨账户共享、支付处理），仿真应用就提供一个确定性桩实现，以在不依赖网络的情况下支持任务验证。

B.2.3 商标、商业外观与素材合规

表 3 和图 10 中的仿真应用标识符（例如，`slack`、`gmail`、`shopify_admin`）是*内部*开发标签，用于将每个合成目标绑定到真实世界的参考对象，为规划代理与网页代理提供依据。公开发布的产物采用以下合规措施，以避免商标、商业外观与素材授权方面的顾虑：

- **重命名为通用标识符**。发布时，每个仿真应用都改为不易引起混淆的通用标签，以描述其功能类别而非参考产品（例如，`slack` → `team-chat-mock`，`gmail` → `mail-mock`，`shopify_admin` → `ecommerce-admin-mock`）。发布包附带内部名称与公开标识符的映射表，所有任务指令、文件路径和`config.json` 条目都仅引用公开标识符。
- **不使用第三方徽标或专有品牌素材**。发布的仿真应用中不嵌入真实徽标、品牌专属图标集或专有插画素材。规划代理调研过程中查看的参考徽标仅作为LLM 的临时输入，不存储在仿真应用源码树中。能够鲜明识别品牌的配色（例如，标识某个真实产品的特定紫色、橙色或蓝色）会替换为从精选的12 色集合中选取的中性配色。
- **独立撰写的UI 文案与插画**。营销文案、微文案、占位插画和营销图像由开发代理从零编写，而非从参考应用复制。导航树和功能清单在功能上受到参考应用启发，但不会逐字复现可能构成商业外观的文本字符串或视觉元素。
- **不再分发参考截图**。规划代理在网络调研中收集的参考截图仅在合成期间保存在本地，不包含在任何发布产物中。图 10 展示的是已发布*仿真应用*的截图，均由我们的开发代理生成。
- **对服务条款敏感的仿真应用标记为需要额外审查**。对于服务条款限制较严的应用（金融服务、医疗、政府门户），其仿真版本会在发布清单中被标记，要求在公开发布前接受额外人工审查；如果参考应用的服务条款甚至禁止功能复现，则该仿真应用不会公开发布，而会替换为同类别的通用替代版本。

- **发布时提供合规清单。**我们将随数据集一并发布逐应用披露文档，列出每个已发布仿真应用的（a） 公开标识符、（b） 功能类别、（c） 所采用的合规措施，以及（d） 服务条款敏感性审查结果。该清单将包含在发布包中，并在数据集卡中跟踪。

本文保留内部命名惯例，以维持清单的可读性，并使§B.4、§B.5 及已发布任务标识符中的交叉引用可追溯；读者应将所有真实产品名称视为相应通用仿真应用类别的内部标签。

B.2.4 逐仿真应用代码行数与端点统计

我们沿三个结构维度衡量每个已发布仿真应用：源码规模（LOC，汇总`src/` 下JavaScript/TypeScript/CSS/HTML 文件的代码行数，排除`node_modules` 和构建产物）、路由数量（单页应用React Router 配置中的`<Route>` 声明），以及实体数量（在`SCHEMA.md` 或`src/utils/dataManager.js` 中声明的内存数据模型顶层实体）。发布的生产套件包含94 个仿真应用；源码树还保留了 5 个模板运行框架（用作新仿真应用的起点），它们不计入已发布任务语料库。表 4 报告了完整 99 目录测量集合的汇总统计；LOC 分桶直方图见表 5。仅统计已发布的94 个仿真应用时，每个维度的均值变化均小于 3%（这些运行框架位于极低 LOC 桶中），因此为保持透明，我们报告包含这些框架的统计数值。

Metric	mean	p10	p25	p50	p75	p90	p99	max
Source LOC	6,127	4,237	4,828	5,663	7,196	9,180	13,095	13,095
Route components	15.0	4	8	14	20	27	60	60
Data-model entities	5.9	3	3	5	7	10	23	23

表 4: 所测量的 99 个仿真应用的结构统计。

Source LOC	Mocks	%
$<$1K	1	1.0
1K–2.5K	1	1.0
2.5K–5K	31	31.3
5K–10K	59	59.6
10K–20K	7	7.1
$\geq$20K	0	0.0

表 5: 仿真应用源码行数分布。大多数仿真应用（91%）位于 2.5K–10K LOC 区间，与设计约束一致：一个独立的单页应用实现约 10–20 个可导航视图，并使用小型、有类型的内存模式。超过 10K LOC 的离群应用（例如，`aws_console_mock`、`Canvas-LMS_mock`、`expedia_mock`）覆盖更广的功能面；两个最小的仿真应用是为保留派生来源而保留的模板运行框架，并非任务目标。

B.3 统一状态API 规格

B.3.1 端点契约

每个仿真应用都实现四端点状态API，支持状态注入、状态检查与重置。该API 实现为与单页应用共驻的 Vite 中间件插件，无需独立进程，并可通过与单页应用相同的源访问。

`POST /post?sid=<sid>` —— 状态注入与生命周期控制。请求体：`{ "action": <"set" | "set_current" | "reset" | "merge">, "state": <object>, "merge": <bool> }`。动作语义：

- `set`：将`state` 写入会话的*初始*快照。由`initial_setup.py` 使用。
- `set_current`：将`state` 写入会话的*当前*快照，保持初始快照不变。由`golden_patch.py` 使用；少数需要进行中状态的情形下，`initial_setup.py` 也会使用。
- `merge`：与`set_current` 相同，但对现有当前快照进行深度合并，而非替换。
- `reset`：清空初始快照和当前快照，并重新加载默认种子数据。

响应：`{ "success": true, "sid": <sid>, "state_id": <hash> }`。

`GET /go?sid=<sid>` ——结构化状态检查。响应：`{ "initial_state": <object>, "current_state": <object>, "state_diff": <object> }`。`state_diff` 是一个扁平键路径映射，将已更改字段映射为`{ "old": <v>, "new": <v> }` 对，具体计算方式见§B.3.3。这是`reward.py` 使用的主要接口。

`GET /state?sid=<sid>` ——原始当前状态。
响应：`{ "stored_state": <object>, "has_custom_state": <bool>, "sid": <sid> }`。当不需要差异抽象时，提供此端点作为更底层的检查接口。

`POST /upload?sid=<sid>` ——文件上传，供要求用户提供附件的任务使用。
请求体：`multipart/form-data`；响应包含逐文件URL，供单页应用解析。文件作用域限定于`sid`，会话重置时会被丢弃。

B.3.2 通过sid 实现会话隔离

每次API 调用都通过查询参数携带会话标识符`sid`，将所有状态读写限定在各自会话的存储命名空间中。训练器为每次轨迹采样生成新的`sid`（UUIDv4），并将其持久化到虚拟机内一个已知文件路径，使后续所有`initial_setup.py` 调用、代理发出的调用和`reward.py` 调用共享同一会话。各会话相互独立：在同一仿真应用实例上并发进行的轨迹采样互不干扰。会话状态的生存时间为最后一次访问后的一小时；过期会话会自动被垃圾回收，无需干预。

B.3.3 状态差异计算规则

`/go` 返回的`state_diff` 通过比较初始快照与当前快照的键路径计算，采用三条面向奖励编写优化的规则。对象字段按结构比较（对嵌套对象递归下降）。数组视为有序序列，任一索引处出现差异都会将整个数组标记为已改变；这样可避免代理只是重排而非编辑时产生错误的部分相等匹配。列入白名单的易变字段（例如，`lastViewedAt`、计算出的UI 状态缓存）在求差异前被屏蔽，防止被动浏览动作污染差异。差异表示有意采用扁平结构（键路径字符串），而非嵌套结构，因为奖励编写者会针对各个字段编写线性排列的断言来使用它。

B.3.4 与reward.py 集成

网页仿真任务中`reward.py` 的规范模式是获取一次 `/go`，然后对`state_diff` 执行一系列部分得分断言，累计得到一个 $[0,1]$ 分数：

网页仿真任务的规范reward.py 模式

```python
import requests

def verify_task():
    sid = open('/tmp/task_web_sid').read().strip()
    response = requests.get(f'http://localhost:8080/go?sid={sid}').json()
    diff = response['state_diff']
    score = 0.0
    if 'channels[0].name' in diff and diff['channels[0].name']['new'] == 'engineering':
        score += 0.25
    if 'messages.engineering[0].content' in diff:
        score += 0.25
    # ... 其他断言 ...
    print(f'REWARD: {score}')
    return score
```

B.4 多代理合成流水线

B.4.1 规划代理：网络调研与DESIGN.md

规划代理负责在编写任何代码之前理解目标应用。给定应用名称，它执行结构化网络调研（文档抓取、截图收集、针对不同用户角色画像的功能清单整理），并输出四项产物：`DESIGN.md`（配色、字体、间距设计值、组件样式）、`assets/README.md`（UI 布局描述与主要用户工作流）、`assets/data_model.md`（内存状态中的实体定义），以及`TODO.md`（面向开发代理、按P0/P1/P2 优先级排列的工作队列）。角色规格明确排除认证流程、真实网络通信，以及超出`localStorage` 的持久化；仿真应用的契约是成为忠实的交互沙箱，而非参考服务的完整功能克隆。

B.4.2 开发代理：实现协议

开发代理读取规划代理的产物，按照固定项目布局将仿真应用实现为Vite + React 单页应用（`src/App.jsx` 负责路由，`src/context/AppContext.jsx` 负责全局状态，`src/utils/dataManager.js` 负责状态初始化与`localStorage` 持久化，`src/utils/stateTracker.js` 负责差异计算，`vite.config.js` 负责状态API 插件）。它与规划代理、网页代理之间的协调完全基于文件，通过§B.4.1 的产物和§B.4.3 的测试报告进行；不允许直接消息通信。任一工作周期的优先级按以下顺序执行：AUDIT P0 问题 > TEST P0 问题 > AUDIT P1 问题 > TODO P0 条目 > TODO P1 条目 > TODO P2 条目，确保在开发新功能前先解决代码正确性回归问题。

B.4.3 网页代理：使用Playwright 对照UI 树验证

网页代理通过无头Playwright 浏览器操作已部署的仿真应用，并对照DESIGN.md 和assets/README.md 验证UI 树。它测试TODO.md 中列出的每个交互元素，在有参考截图时执行视觉差异比较，并生成TEST.md（功能与视觉缺陷报告）和AUDIT.md（代码层面问题，例如失效的事件处理器、未跟踪的状态修改，以及差异API 缺少条目）。报告采用与开发代理优先级顺序一致的P0/P1/P2 严重性分级，确保修复按确定性顺序推进。

B.4.4 迭代收敛循环

规划、开发和网页代理持续迭代，直到TEST.md 与AUDIT.md 均报告P0 和P1 问题数量为零，或耗尽逐应用轮次预算。每轮包含：网页代理测试 → 开发代理修复 → 网页代理复测。只有出现根本性规格缺口时（例如，真实截图中出现某功能，但TODO.md 未包含该功能），规划代理才重新介入。未能在轮次预算内收敛的仿真应用会被标记为需要人工审查，并在问题解决前排除在发布套件之外。

B.5 代表性仿真应用案例研究

B.5.1 Slack 仿真应用：状态模式与功能清单

Slack 仿真应用展示了通信应用的典型状态模式：由频道构成的树，每个频道拥有一个包含元数据的有序消息列表（发送者、时间戳、父线程、表情回应、附件、编辑标记）。状态对象带有顶层currentUser 引用和扁平users 列表，在无需认证运行框架的情况下提供用户切换任务（例如，“以用户X 身份发送消息”）。可验证交互包括频道创建与归档、消息发送与编辑、线程回复、表情回应添加 / 移除，以及私信创建。仿真应用与真实应用的显著差异在于：前者没有在线状态、推送通知或工作区级管理策略的概念；依赖这些能力的任务留给其他类别处理。

B.5.2 Jira 仿真应用：跨实体状态转换

Jira 仿真应用展示了关联实体间状态转换的保真度。状态模式涵盖项目、冲刺、看板、问题及问题状态工作流（To Do → In Progress → In Review → Done）。数据模型强制维持关键的跨实体不变量：问题所属的冲刺必须与该冲刺的项目保持一致；状态转换必须遵循已配置的工作流图；看板列归属是问题状态的派生视图，而非独立字段。多步任务（例如，“将当前冲刺中所有未分配的缺陷移入待办列表”）通过检查问题集合的差异来验证，而非追踪各个UI 事件。

B.5.3 Salesforce 仿真应用：表单密集型工作流

Salesforce 仿真应用展示了表单密集型CRM 工作流模式。状态模式围绕潜在客户、联系人、客户和商机展开，规范的潜在客户转商机流程包含涉及 ∼25 个字段的多步表单（资质评估阶段、利益相关者识别、商机规模估计、成交日期预测）。这些任务的奖励验证针对字段值，而非操作路径：检查差异中是否包含正确的已填写字段集合，以及正确的类型和值约束，但代理可以自由使用任意UI 路径填写。这种解耦解释了为何即使代理在视觉上不熟悉应用，只要正确识别字段，也能解决表单密集型任务。

C 训练细节

C.1 智能体运行框架

C.1.1 观测与动作空间

智能体观测桌面或浏览器视口的原始截图，并以XML 封装的工具调用形式输出动作。每个轨迹采样步骤将一幅图像观测与一个<tool_call> 块配对；策略可以在一个块内输出多个原始动作，将确定性的动作链压缩到一轮中（§4.3）。

截图预处理。 截图按底层虚拟机的原生分辨率采集（OSWorld 桌面任务为 1000×1000；网页任务使用原生视口）。视觉编码器采用的词元预算对应于min_pixels $= 64 \cdot 32^2 = 65{,}536$ 至max_pixels $= 2048 \cdot 32^2 = 2{,}097{,}152$ 个像素，图像处理器在该范围内选择最接近且保持宽高比的缩放尺寸。不进行额外裁剪或显著性遮罩处理。

工具调用模式。 策略可调用一个computer_use 函数。每个动作原语作为<function> 块的<parameter=action> 子元素调用，位置参数（coordinate、text、key、seconds）通过附加参数提供：

工具调用的XML 格式

```
<tool_call>
<function=computer_use>
<parameter=action>left_click</parameter>
<parameter=coordinate>[500, 250]</parameter>
</function>
</tool_call>
```

动作原语。 动作集合涵盖指针、键盘、导航与控制原语：

- *指针*：`left_click`、`right_click`、`middle_click`、`double_click`、`triple_click`、`left_click_drag`、`mouse_move`、`left_mouse_down`、`left_mouse_up`。
- *键盘*：`type`、`key`、`key_down`、`key_up`。
- *导航*：`scroll`、`hscroll`、`screenshot`、`wait`。
- *控制*：`terminate`（声明回合成功或失败）、`call_user`（请求澄清，仅用于评估，从不用于RL 期间）。

多动作格式。 XML 格式允许在单个`<tool_call>` 内连续包含多个`<function>` 块，按顺序执行且中间不获取截图。策略不受必须使用该格式的约束；RL 训练中涌现的批量执行行为（§4.3）完全由策略自身驱动。

C.1.2 轨迹切片方案

长时程CUA 轨迹经常超出模型的上下文预算。在我们的设置下，一个回合包含 $\texttt{max_turns} = 100$ 轮，每轮最多 $\texttt{max_response} = 2048$ 个词元，再加上每轮截图，总长度可增长至约 200K 个词元，而上下文硬上限为 144K。溢出时直接截断，要么丢弃后续轮次的监督信号，要么在无提示的情况下产生形状不匹配的批次。轨迹切片在固定上下文预算下为*每条轨迹输出多个训练样本*来解决这一问题，每个样本从对话历史中不同的起点观察同一条轨迹。

切片构造。 每经过$\texttt{traj_slice_interval} = 10$ 组轮次对，管理器便输出一个以整数`collapsed_length` 为索引的切片。切片的*提示词*部分由系统消息及其后的前`collapsed_length` 组轮次对组成，其中的截图被简短的占位文本（`"<image collapsed>"`）替代；切片的*响应*部分则包含此后直至当前采样位置的所有轮次，并保留完整的多模态观测。第一个切片的$\texttt{collapsed_length} = 0$，包含原始形式的轨迹。后续切片通过文本占位符表示越来越多的较早轮次，从而为新的观测释放上下文预算。

损失掩码。 对于切片中的每条消息，我们设置：

- 提示词部分的每个词元均设为$\texttt{loss_mask} = \texttt{False}$（不产生梯度）。
- 响应部分的每条`user` 或`tool` 消息均设为$\texttt{loss_mask} = \texttt{False}$（它们是环境观测，而非策略输出）。
- 响应部分的每条`assistant` 消息均设为$\texttt{loss_mask} = \texttt{True}$；但若缺少缓存的对数概率（表明该消息由过时策略版本采样得到），或消息包含超过$\texttt{MAX_TOOL_CALLS_PER_TURN} = 10$ 次工具调用（用于防止格式异常输出），则除外。

奖励归属。 每个切片都获得所属轨迹的*完整回合奖励*，以完全相同的数值复制。我们不在切片之间拆分奖励或对奖励折扣。理由有二：奖励是最终环境状态的属性，而非某个单独轮次的属性；此外，组归一化优势估计（§3.1）按提示词应用，因此各切片相同的奖励只是为同一个优势信号提供更多视角。

上下文溢出。 如果切片在折叠后累计词元数仍超出上下文预算，则将该切片替换为*占位切片*：由填充词元构成，且`loss_mask` 全部为`False`。占位切片不贡献梯度，但保留数据加载器所要求的批次形状。实际中这种情况很少出现（在我们的运行中占切片数的 $< 1\%$），因为切片安排使活跃上下文始终远低于上限。

与其他方案的比较。 在我们的设置中，切片严格优于两种自然的基线方案。*截断*（上下文溢出时丢弃后续轨迹）恰好丢弃了决定成功或失败的轨迹后期轮次的监督信号，使策略偏向早期轮次的行为。*摘要*（用语言模型生成的摘要替换前缀）比截断保留更多信号，但会为每个训练样本增加一次语言模型调用，将摘要质量的方差引入梯度，并丢失从截图像素到定位坐标的确定性映射。切片能像摘要一样缓解上下文压力，同时无需额外推理成本或有损压缩：文本占位符完全确定，而所有切片的并集覆盖完整轨迹，梯度即在这一完整轨迹上计算。

C.1.3 推理时配置

步数与历史预算。 每次轨迹采样在达到 `max_turns` = 100 个模型步骤或执行`terminate` 动作时结束，以先发生者为准。智能体在活跃上下文中保留最近的 `history_n` = 100 轮（在步数上限下，实际上就是全部轮次）；若存在更早的轮次，则按上述方式进行切片处理。

解码。 训练轨迹采样与验证轨迹采样共享同一解码栈（启用多词元预测的`sglang` 服务，`enable_mtp` = `True`，`speculative_num_steps` = 3，`speculative_num_draft_tokens` = 4，注意力后端为FA3）。验证解码使用温度 0.6、top-p 0.95，不限制top-k，重复惩罚为 1.0。训练轨迹采样使用继承自采样配置的 GSPO 采样温度（默认 1.0）。每轮最多生成 `max_new_tokens` = 2048 个输出词元。

故障恢复。 编排器处理三类故障而不中止轨迹采样：（i）工具调用XML 格式异常时，在下一条`user` 消息中加入解析器错误信息，重新提示模型；（ii）虚拟机侧动作失败（例如未找到目标元素）时，返回错误字符串，供模型作为普通环境响应观测；（iii）采样器侧异常（超时、服务端断连）时，按指数退避最多重试 `retry_times` = 5 次，之后才将轨迹标记为失败，并由过采样过滤器丢弃。

C.1.4 系统提示词与工具定义

智能体的系统提示词在所有轨迹采样与任务中保持不变。它定义工具调用语法、响应结构和终止语义，但不包含任务特定信息；任务指令作为第一条`user` 消息提供。

推理时智能体系统提示词

```
你是一名多用途智能助手。根据我的请求，
你可以使用工具帮助我完成各种任务。

# 工具

你可以使用以下函数：

{
  "name": "computer_use",
  "description": "使用鼠标和键盘与计算机交互，并截取屏幕。",
  "parameters": { ... action / coordinate / text / key / seconds ... }
}

如果选择调用函数，只能按以下格式回复，末尾不得附加任何内容：

<tool_call>
<function=example_function_name>
<parameter=example_parameter_1>value_1</parameter>
<parameter=example_parameter_2>value_2</parameter>
</function>
</tool_call>

# 响应格式

对于普通的界面交互步骤：
1) Action：一句简短的祈使句，描述要在界面中执行的操作。
2) 单个 <tool_call>...</tool_call> 块。

规则：
- 对于非终止的界面步骤，严格依次输出：Action，然后 <tool_call>。
- 保持简洁：Action 只用一句话。
- 工具调用之后不要输出任何内容。
- 需要用户提供信息或确认时，使用 call_user。
- 想要明确以成功或失败状态结束任务时，使用 terminate。
```

完整的`computer_use` 工具模式（此处为简洁起见省略）列出§C.1.1 中的每个原语及其接受的参数类型，并在发布代码中原样提供。

C.2 SFT 预热

C.2.1 教师轨迹筛选

SFT 初始化数据通过在与RL 相同的任务池上使用强教师策略进行轨迹采样来筛选。我们在温度 1.0 下为每个任务采样4 条轨迹，仅保留终止奖励为 $r(s,\tau)=1$ 的轨迹。这一仅保留成功样本的过滤步骤丢弃了约50% 的教师轨迹，最终得到包含3,578 条轨迹、覆盖完整任务分布的SFT 语料。SFT 阶段不进行额外的重加权或领域再平衡。

C.2.2 SFT 超参数

SFT 在助手词元上使用标准交叉熵目标，并对用户、系统和工具消息进行掩码处理。我们使用AdamW，峰值学习率为 7×10^{-6}，在整个训练过程中按余弦计划衰减至 7×10^{-7}；权重衰减为 0.01，批大小为512 个提示词，对筛选后的语料训练1 轮。序列长度为 256K 个词元；同样应用轨迹切片（§C.1.2），以保持SFT 与RL 数据形状兼容。SFT 在 512 块GPU 上采用`bf16` 混合精度运行，矩阵乘法路径使用FP8 权重。

C.3 GSPO 训练

C.3.1 超参数

表 6 列出了本文所有RL 运行使用的超参数值。除特别注明外，默认设置遵循§3.1 中的GSPO 表述。

Symbol	Value	Description
G	16	Group size: rollouts per prompt for GSPO advantage estimation.
$N_{\text{oversample}}$	20	Over-sampling target; the first $G=16$ valid rollouts per prompt are retained, invalid ones (timeouts, malformed outputs) discarded.
ε	0.2	Clipping range for the importance ratio (positive and negative clip equal).
β	0	KL penalty against the reference policy. We disable the KL term and the reference model entirely in our reported runs; ablations with $\beta>0$ did not yield improvements at this data scale.
LR	1×10^{-6}	AdamW learning rate, constant schedule, no warmup.
$(\beta_1,\beta_2,\epsilon_{\text{Adam}})$	$(0.9, 0.999, 10^{-8})$	AdamW optimizer parameters.
Weight decay	0.01	Applied to all non-bias non-norm parameters.
Batch size	128 prompts	Outer GSPO batch (collected via the over-sampled rollouts).
Mini-batch size	32 prompts	Inner gradient-accumulation step.
PPO epochs	1	Single optimizer pass per outer batch.
Total steps	1000	Outer-batch updates over the full training run.
Rollout temperature	1.0	Sampling temperature during training rollouts.
Eval temperature	0.6	Sampling temperature during validation rollouts.
Top-p / top-k	0.95 / unconstrained	Nucleus sampling parameters (eval).
Max new tokens / turn	2048	Per-turn response cap.
Max context length	144K	Hard context cap; triggers trajectory slicing (§C.1.2).
Max prompt length	8K	Initial prompt budget per slice.

表 6: §3.1 和§4.1 所报告运行的GSPO 训练超参数。

辅助损失。 我们额外施加中文词元惩罚（系数为 0.5），以降低低质量双语输出的权重，同时禁用格式、语言与无休止输出惩罚，并禁用组过滤。熵奖励始终保持为零。

C.3.2 参考策略更新安排

当 $\beta=0$ 时，训练期间不将参考策略加载至内存，也不对其进行评估。这为每个节点释放了GPU 显存，以容纳更大的轨迹采样批次，并允许轨迹采样实例（§C.4.1）以完整的 24 实例规模运行而不发生资源争用。我们在早期原型阶段重新评估了 $\beta\in\{10^{-3},10^{-2}\}$ 的设置，在本文所报告的数据规模下，既未观察到训练稳定性提升，也未观察到任务成功率提升。

C.3.3 优势归一化细节

优势按提示词逐组计算：$\hat{A}_i = r_i - \mu_g$，其中 μ_g 是该提示词 $G = 16$ 条轨迹的平均奖励。我们有意禁用标准差归一化（`divide_by_std = False`）；渐进式的 $[0,1]$ 值奖励（§2.1）本身已处于固定尺度，在组内结果近乎一致（所有轨迹都成功或都失败）的提示词上，除以 σ_g 会放大噪声。对于零方差组，我们保留去均值后的优势，其值为零，不贡献梯度。

C.4 基础设施

C.4.1 硬件与集群配置

RL 运行使用NVIDIA H200 SXM GPU（每块具有 141 GB HBM3e 显存），每节点 8 块GPU，通过高带宽NVLink/NVSwitch 互连。我们报告两种与模型规模匹配的配置：用于Qwen3.5-35B-A3B 规模RL 的 *turbo* 集群，包含 192 块GPU（24 个节点）；以及用于Qwen3.5-397B-A17B 规模RL 的*plus* 集群，包含 512 块GPU（64 个节点）。借助`verl` [20] 的训练器解耦基础设施，每个集群均分为训练与轨迹采样子集群：plus 使用 32 个训练节点与 32 个采样节点配对，turbo 则在其 24 节点规模下采用相应的均衡划分。两个子集群通过定制调度层通信，调度比为 2.35，采样与训练之间的策略版本最多偏差 4 步。两种配置的训练并行设置均为 TP = 2、EP = 8、CP = 4；plus 的流水线深度为 PP = 8（更深的划分用于容纳更大的骨干模型），turbo 为 PP = 2。轨迹采样服务在推理集群上使用 TP = 8、数据并行复制因子 4，以及`fa3`注意力后端。整个过程中均启用推测解码（`enable_mtp`、3 个推测步骤、4 个草稿词元）。轨迹采样服务使用`sglang` [17]。

C.4.2 轨迹采样的虚拟机供给

每次轨迹采样连接至内部路由器后方提供服务的2,000 个并行OSWorld 虚拟机实例之一。采样请求提交任务指令与智能体步数上限；路由器选择一个空闲实例，恢复任务`config.json` 中指定的快照，并向训练器返回逐步截图与奖励流。回合终止后（执行`terminate` 动作或耗尽步数预算），虚拟机先重置为干净快照，再释放回池中。轨迹采样子集群向大小为 2,000 的池供给任务时，每个训练步骤的平均虚拟机利用率超过 75%，轨迹采样延迟并非训练瓶颈。

C.4.3 每次训练的实际耗时与成本

在Qwen3.5-35B-A3B 骨干模型上完成一次完整的 1000 步RL 训练（turbo，192 块GPU），实际耗时约 5 天，总成本约为 ∼23,040 GPU 小时。Qwen3.5-397B-A17B 骨干模型上的对应运行（plus，512 块GPU）同样约需 5 天，总成本约为 ∼61,440 GPU 小时。成本主要来自轨迹采样，训练器在等待回合完成时相对空闲。1.4K 和 3K 规模的较小数据消融实验（§4.1）完成得相应更快，但每步计算量相同，因为不同运行之间变化的是轨迹数量，而非步数。

C.5 可复现性检查清单

本附录之后完整附上NeurIPS 论文检查清单，其中我们的`Yes/No/NA` 回答及理由均交叉引用相应章节（合成流水线见§B.4，超参数见§C.3.1，基础设施见§C.4.1，局限性见§6）。

D 扩展实验

D.1 涌现的多动作行为

我们通过解析 1,105 条轨迹中的原始`<tool_call>` 流，对§4.3 报告的多动作涌现进行刻画。这些轨迹由Qwen3.5-397B-A17B 规模RL 检查点在训练第 30 步时于OSWorld-Verified 测试划分上采样得到，合计 22,361 个模型步骤。该检查点在这一分布上每个模型步骤平均调用工具 1.41 次，逐步批量调用分布如下：69.6% 的步骤调用一次，26.7% 调用两次，2.4% 调用三次，0.8% 调用四次，0.5% 调用五次及以上。因此，轨迹缩短效果主要来自两次调用的批量执行；对于策略已内化为局部确定性的动作链，才会选择性地使用更长的批次。

D.1.1 最常见的K 类批量动作模式

表 7 列出了单个模型步骤内最常见的长度为 ≥ 2 的工具调用序列。两种最主要的模式是确定性输入链（`type` → `key`，3,942 步）和连续滚动的前置动作（`mouse_move` → `scroll`，1,052 步）；两者合计占所有批量调用步骤的 ∼ 75%。其次是动作与观测配对（`wait` → `screenshot`、`left_click` → `screenshot`），策略在同一轮内将确定性的环境操作与一次新观测交织起来，节省一次往返。三次调用的表单填写模式（`type` → `key` → `screenshot`、`key_down` → `left_click` → `key_up`）和多步宏操作（`type` → `key` → `wait` → `screenshot`、`left_click` → `type` → `key` → `screenshot`）则出现在长尾部分。

|seq|	Count	Sequence
2	3,942	`type` → `key`
2	1,052	`mouse_move` → `scroll`
2	315	`wait` → `screenshot`
2	303	`left_click` → `screenshot`
3	158	`type` → `key` → `screenshot`
3	119	`key_down` → `left_click` → `key_up`
2	99	`mouse_move` → `left_click_drag`
2	96	`key` → `screenshot`
3	54	`mouse_move` → `scroll` → `screenshot`
4	51	`type` → `key` → `wait` → `screenshot`
4	35	`left_click` → `type` → `key` → `screenshot`
3	33	`left_click` → `wait` → `screenshot`

表 7: Qwen3.5-397B-A17B 规模RL 检查点每个模型步骤中最常见的批量工具调用序列（长度为 ≥ 2），汇总自 1,105 条测试轨迹。

D.1.2 避免批量执行的非确定性动作类别

一个互补的观察角度是各类动作的*单独执行率*，即该动作的全部出现次数中，单独占据一个模型步骤的比例（表 8）。指针点击变体（`right_click`、`double_click`、`triple_click`）在 94–98% 的情况下单独输出，这与如下假设一致：这些动作会显示上下文菜单或选择状态，其操作后的界面难以预测。`left_click` 的单独执行率也很高（90.6%）：单次点击通常会触发视图变化，策略需要重新观测后才能决定下一步。相反，`scroll`、`key_down`、`key_up` 和 `left_click_drag` 从不单独输出；它们是较大手势的机械性组成部分（例如`mouse_move` → `scroll`、组合键修饰的点击模式、拖动选择），孤立时没有语义。因此，策略的批量执行行为符合一个明确原则：对机械性手势的组成部分和确定性动作链进行批量执行；对于结果依赖随机界面状态的动作，暂缓跨动作批量执行。

Action	Solo	Total	Solo %
right_click	144	147	98.0
double_click	328	346	94.8
triple_click	630	666	94.6
left_click	10,132	11,187	90.6
screenshot	1,023	2,319	44.1
key	2,351	7,356	32.0
wait	160	615	26.0
type	738	5,199	14.2
mouse_move	55	1,279	4.3
scroll	0	1,112	0.0
key_down	0	153	0.0
key_up	0	153	0.0
left_click_drag	0	104	0.0

表 8: 各工具动作的单独执行率：动作单独出现在一个模型步骤中的次数，相对于其在所有批量与非批量步骤中的总出现次数。靠近表格底部的动作从不单独出现，表明它们仅作为较大手势的组成部分批量执行。

E 定性示例

E.1 端到端任务详解

我们展示从发布数据集中抽取的四个代表性元组，涵盖桌面、网页和跨应用领域。每个详解均原样引用任务指令，概述奖励分解，并展示`reward.py` 中最具特色的评分组件。完整的设置与奖励脚本随数据一并发布。

E.1.1 桌面任务——LibreOffice Calc：编号补零公式

这是一个紧凑的单领域示例。任务要求智能体使用基于TEXT 的补零公式填充一列中的多行；奖励分别针对标准首单元格、向下传播的公式以及跨单元格引用的正确性分配得分。

calc_afm_082——任务指令

```
使用 TEXT 构建公式，在员工编号前补零，
使编号始终为 5 位数字。
```

奖励分解（3 个组件，总计 1.0）：

- **C1 (0.40)**：B2 包含格式字符串为"00000" 的TEXT 公式。
- **C2 (0.30)**：B3:B6 均包含使用相同格式字符串的TEXT 公式。
- **C3 (0.30)**：每个公式都引用A 列中对应的单元格。

calc_afm_082/initial_setup.py

```python
"""
初始设置：使用 TEXT 公式在员工编号前补零
任务 ID：  calc_afm_082
领域：  libreoffice_calc
"""

import os
import shlex
import subprocess
import time
import openpyxl

WORKDIR = '/home/user'
TASK_ID = 'calc_afm_082'
OUTPUT = f'{WORKDIR}/{TASK_ID}.xlsx'


def launch_gui(command: str, delay_sec: float = 1.0):
    """在虚拟机显示器上启动 GUI 应用，不阻塞脚本退出。"""
    env = os.environ.copy()
    env["DISPLAY"] = ":0"
    subprocess.Popen(
        shlex.split(command),
        stdout=subprocess.DEVNULL,
        stderr=subprocess.DEVNULL,
        env=env,
    )
    time.sleep(delay_sec)


def create_initial():
    wb = openpyxl.Workbook()

    # --- 工作表：IDs ---
    ws = wb.active
    ws.title = 'IDs'

    # 表头
    ws['A1'] = 'Raw ID'
    ws['B1'] = 'Formatted ID'

    # 数据——原始员工编号（整数）
    raw_ids = [42, 7, 1356, 890, 15]
    for r, val in enumerate(raw_ids, 2):
        ws.cell(row=r, column=1, value=val)

    # B2:B6 有意留空——这正是智能体需要完成的任务

    # 设置合理的列宽
    ws.column_dimensions['A'].width = 12
```

```
    ws.column_dimensions['B'].width = 16

    wb.save(OUTPUT)
    print(f'Initial file created: {OUTPUT}')

    # 启动并准备好 GUI
    launch_gui(f'libreoffice --calc "{OUTPUT}"', delay_sec=2.0)
    print('GUI_READY: launched LibreOffice Calc with DISPLAY=:0')


create_initial()
```

calc_afm_082/golden_patch.py

```
"""
标准完成状态补丁：使用 TEXT 公式在员工编号前补零
任务 ID：  calc_afm_082
领域：  libreoffice_calc
变更：在 B2:B6 中添加 =TEXT(A#,"00000") 公式
"""

import openpyxl

WORKDIR = '/home/user'
TASK_ID = 'calc_afm_082'
OUTPUT = f'{WORKDIR}/{TASK_ID}.xlsx'


def create_golden():
    wb = openpyxl.Workbook()

    # --- 工作表：IDs ---
    ws = wb.active
    ws.title = 'IDs'

    # 表头
    ws['A1'] = 'Raw ID'
    ws['B1'] = 'Formatted ID'

    # 数据——与初始状态相同的原始编号
    raw_ids = [42, 7, 1356, 890, 15]
    for r, val in enumerate(raw_ids, 2):
        ws.cell(row=r, column=1, value=val)

    # 标准完成状态：B2:B6 中包含 TEXT 公式
    for r in range(2, 7):
        ws.cell(row=r, column=2, value=f'=TEXT(A{r},"00000")')

    # 列宽与初始状态相同
    ws.column_dimensions['A'].width = 12
    ws.column_dimensions['B'].width = 16

    wb.save(OUTPUT)
    print(f'Golden-state file created: {OUTPUT}')


create_golden()
```

calc_afm_082/reward.py

```
"""
奖励脚本：使用 TEXT 公式在员工编号前补零
任务 ID：  calc_afm_082
领域：  libreoffice_calc
评分：
  组件 1 (0.4)：B2 包含格式为 "00000" 的 TEXT 公式
  组件 2 (0.3)：B3:B6 均包含格式为 "00000" 的 TEXT 公式
  组件 3 (0.3)：所有公式均引用正确的 A 列单元格
"""

import os
import re
import openpyxl

WORKDIR = '/home/user'
TASK_ID = 'calc_afm_082'


def persist_app_state(domain: str):
    """在验证前保存尚未保存的 GUI 状态。"""
    import time
    os.environ["DISPLAY"] = ":0"
    if domain in {"libreoffice_calc", "libreoffice_writer", "libreoffice_impress"}:
        try:
            import pyautogui
            pyautogui.hotkey("ctrl", "s")
            time.sleep(0.8)
            print(f"PERSIST: ctrl+s sent for {domain}")
        except Exception as e:
            print(f"PERSIST_WARN: save hook failed: {e}")


def verify_task(file_path):
    """
    使用渐进式评分验证任务完成情况。
    返回值：0.0 至 1.0 之间的浮点数
    """
    total_score = 0.0

    try:
        wb = openpyxl.load_workbook(file_path)
    except Exception as e:
        print(f"CRITICAL: Cannot load file {file_path}: {e}")
        print("REWARD: 0.0")
        return 0.0

    # 前置条件：'IDs' 工作表必须存在
    if 'IDs' not in wb.sheetnames:
        print("FAIL: Sheet 'IDs' not found")
        print("REWARD: 0.0")
        return 0.0

    ws = wb['IDs']

    # 组件 1：B2 包含格式为 "00000" 的 TEXT 公式 (0.4 分)
    try:
        b2_val = ws['B2'].value
        if b2_val and isinstance(b2_val, str):
            b2_upper = b2_val.upper().replace(" ", "")
            if b2_upper.startswith("=TEXT(") and "00000" in b2_val and "A2" in b2_upper:
                print(f"PASS: C1 - B2 has TEXT formula with 00000 format: {b2_val} (0.4 pts)
")
                total_score += 0.4
            else:
```

```
                print(f"FAIL: C1 - B2 has formula but not correct TEXT/00000/A2 pattern: {
    b2_val}")
            else:
                print(f"FAIL: C1 - B2 does not contain a formula, found: {repr(b2_val)}")
        except Exception as e:
            print(f"ERROR: C1 - {e}")

        # 组件 2: B3:B6 均包含格式为 "00000" 的 TEXT 公式 (0.3 分)
        # 每个单元格得 0.075 分 (= 0.3 / 4 个单元格)
        try:
            comp2_score = 0.0
            for row in range(3, 7):
                cell_ref = f'B{row}'
                val = ws.cell(row=row, column=2).value
                if val and isinstance(val, str):
                    val_upper = val.upper().replace(" ", "")
                    if val_upper.startswith("=TEXT(") and "00000" in val:
                        comp2_score += 0.075
                        print(f"  PASS: {cell_ref} has TEXT/00000 formula: {val}")
                    else:
                        print(f"  FAIL: {cell_ref} has formula but not TEXT/00000: {val}")
                else:
                    print(f"  FAIL: {cell_ref} does not contain a formula, found: {repr(val)}")
            if comp2_score > 0:
                print(f"PASS: C2 - TEXT formulas in B3:B6 ({comp2_score:.3f} pts)")
                total_score += comp2_score
            else:
                print(f"FAIL: C2 - No TEXT formulas found in B3:B6")
        except Exception as e:
            print(f"ERROR: C2 - {e}")

        # 组件 3: 每个公式均引用正确的 A 列单元格 (0.3 分)
        # 每个正确引用得 0.06 分 (= 0.3 / 5 个单元格)
        try:
            comp3_score = 0.0
            for row in range(2, 7):
                cell_ref = f'B{row}'
                expected_ref = f'A{row}'
                val = ws.cell(row=row, column=2).value
                if val and isinstance(val, str):
                    val_upper = val.upper().replace(" ", "")
                    if expected_ref in val_upper:
                        comp3_score += 0.06
                        print(f"  PASS: {cell_ref} references {expected_ref}")
                    else:
                        print(f"  FAIL: {cell_ref} does not reference {expected_ref}: {val}")
                else:
                    print(f"  FAIL: {cell_ref} not a formula, cannot check reference")
            if comp3_score > 0:
                print(f"PASS: C3 - Correct A-column references ({comp3_score:.3f} pts)")
                total_score += comp3_score
            else:
                print(f"FAIL: C3 - No correct A-column references found")
        except Exception as e:
            print(f"ERROR: C3 - {e}")

        final_score = round(min(total_score, 1.0), 2)
        print(f"\nScore: {total_score:.3f}/1.0")
        print(f"REWARD: {final_score}")
        return final_score


    # 入口
    persist_app_state("libreoffice_calc")

    file_path = f'{WORKDIR}/{TASK_ID}.xlsx'
    if not os.path.exists(file_path):
```

```
    print(f"File not found: {file_path}")
    print("REWARD: 0.0")
else:
    verify_task(file_path)
```

E.1.2 桌面任务——VS Code：自定义文件关联

这是一个代码编辑器配置任务。奖励解析VS Code 的`settings.json`（去除JSONC 注释），并验证单个配置键的存在性与内容。验证完全基于文件，无需GUI 内部检查或截图解析。设置脚本创建一个包含混合扩展名文件的真实感工作区，让智能体在终止前可以直观确认语法高亮的变化。

vscode_lp_081——任务指令

```
配置自定义文件关联，使所有 *.config 文件按 JSON 处理，
所有 *.tmpl 文件按 HTML 处理，以进行语法高亮。
```

奖励分解（3 个组件，总计 1.0）：

- **C1 (0.35)**：`settings.json` 中存在`files.associations` 键，且其值为非空对象。
- **C2 (0.35)**：`*.config` 映射到`json`。
- **C3 (0.30)**：`*.tmpl` 映射到`html`。

vscode_lp_081/initial_setup.py

```
"""
初始设置：为 *.config 和 *.tmpl 文件配置自定义文件关联
任务 ID:  vscode_lp_081
领域:  vscode
"""

import json
import os
import shlex
import subprocess
import time

WORKDIR = '/home/user'
TASK_ID = 'vscode_lp_081'
WORKSPACE = f'{WORKDIR}/workspace'
VSCODE_USER = os.path.join(WORKDIR, '.config', 'Code', 'User')
SETTINGS_PATH = os.path.join(VSCODE_USER, 'settings.json')


def launch_gui(command: str, delay_sec: float = 1.0):
    env = os.environ.copy()
    env["DISPLAY"] = ":0"
    subprocess.Popen(
        shlex.split(command),
        stdout=subprocess.DEVNULL,
        stderr=subprocess.DEVNULL,
        env=env,
    )
    time.sleep(delay_sec)


def create_initial():
    os.makedirs(WORKSPACE, exist_ok=True)

    # --- 初始数据：两个 .config 文件（类似 JSON 的内容，尚无高亮）---
    db_config = """{
```

```
  "host": "db.internal.acme.com",
  "port": 5432,
  "database": "inventory_prod",
  "username": "app_service",
  "max_connections": 25,
  "ssl_enabled": true,
  "timeout_ms": 3000,
  "retry_policy": {"max_retries": 3, "backoff_ms": 500}
}"""
    with open(os.path.join(WORKSPACE, 'database.config'), 'w') as f:
        f.write(db_config)

    app_config = """{
  "app_name": "InventoryTracker",
  "version": "2.4.1",
  "environment": "production",
  "logging": {"level": "info", "rotate_size_mb": 50},
  "features": {"enable_caching": true, "cache_ttl_seconds": 300},
  "api_base_url": "https://api.acme.com/v3"
}"""
    with open(os.path.join(WORKSPACE, 'app.config'), 'w') as f:
        f.write(app_config)

    # --- 初始数据：两个 .tmpl 文件（类似 HTML 的内容）---
    email_tmpl = """<!DOCTYPE html>
<html lang="en">
<head><meta charset="UTF-8"><title>Order {{order_id}}</title></head>
<body>
  <div class="header">
    <h1>Order Confirmation</h1>
    <p>Thank you, {{customer_name}}!</p>
  </div>
  <div class="order-details">
    <h2>Order #{{order_id}}</h2>
    <table>
      <tr><th>Item</th><th>Qty</th><th>Price</th></tr>
      {{#items}}
      <tr><td>{{name}}</td><td>{{quantity}}</td><td>${{price}}</td></tr>
      {{/items}}
    </table>
    <p class="total">Total: ${{total_amount}}</p>
  </div>
</body>
</html>"""
    with open(os.path.join(WORKSPACE, 'email_notification.tmpl'), 'w') as f:
        f.write(email_tmpl)

    dashboard_tmpl = """<!DOCTYPE html>
<html lang="en">
<head><title>{{dashboard_title}}</title></head>
<body>
  <nav class="sidebar">
    <ul>
      {{#menu_items}}
      <li><a href="{{url}}">{{label}}</a></li>
      {{/menu_items}}
    </ul>
  </nav>
  <main class="content">
    <h1>{{page_title}}</h1>
    <div class="metrics-grid">
      {{#metrics}}
      <div class="metric-card">
        <span class="metric-value">{{value}}</span>
        <span class="metric-label">{{label}}</span>
      </div>
      {{/metrics}}
```

```
    </div>
  </main>
</body>
</html>"""
    with open(os.path.join(WORKSPACE, 'dashboard.tmpl'), 'w') as f:
        f.write(dashboard_tmpl)

    # --- 初始数据：一个 .py 文件，用于增加多样性 ---
    main_py = """#!/usr/bin/env python3
\"\"\"库存跟踪器——主应用入口。\"\"\"
import logging, json, os
CONFIG_PATH = os.path.join(os.path.dirname(__file__), 'app.config')

def load_config(path):
    with open(path) as f: return json.load(f)

def main():
    cfg = load_config(CONFIG_PATH)
    logging.basicConfig(level=getattr(logging, cfg['logging']['level'].upper()))
    logger = logging.getLogger(cfg['app_name'])
    logger.info(f"Starting {cfg['app_name']} v{cfg['version']}")

if __name__ == '__main__':
    main()
"""
    with open(os.path.join(WORKSPACE, 'main.py'), 'w') as f:
        f.write(main_py)

    # --- 确保 VSCode 设置存在，但不包含 files.associations ---
    os.makedirs(VSCODE_USER, exist_ok=True)
    settings = {}
    if os.path.exists(SETTINGS_PATH):
        try:
            with open(SETTINGS_PATH, 'r') as f:
                settings = json.load(f)
        except (json.JSONDecodeError, FileNotFoundError):
            settings = {}
    settings.pop('files.associations', None)   # 若存在则移除
    with open(SETTINGS_PATH, 'w') as f:
        json.dump(settings, f, indent=4)

    print(f'Workspace created at: {WORKSPACE}')
    launch_gui(f'code "{WORKSPACE}"', delay_sec=2.0)
    print('GUI_READY: launched VSCode with DISPLAY=:0')


create_initial()
```

vscode_lp_081/golden_patch.py

```
"""
标准完成状态补丁：为 *.config 和 *.tmpl 文件配置自定义文件关联
任务 ID：  vscode_lp_081
领域：  vscode
变更：
  - 使用相同的初始文件重建工作区（标准完成状态环境独立构建）。
  - 向 settings.json 添加 files.associations：
      *.config -> json,  *.tmpl -> html
"""

import json
import os

WORKDIR = '/home/user'
```

```
TASK_ID = 'vscode_lp_081'
WORKSPACE = f'{WORKDIR}/workspace'
VSCODE_USER = os.path.join(WORKDIR, '.config', 'Code', 'User')
SETTINGS_PATH = os.path.join(VSCODE_USER, 'settings.json')


def create_golden():
    # 工作区初始数据与 initial_setup.py 相同（此处因篇幅省略；
    # 发布脚本原样重建这五个文件，因为标准完成状态虚拟机
    # 独立于初始虚拟机构建）。
    os.makedirs(WORKSPACE, exist_ok=True)
    # ... 写入 database.config、app.config、email_notification.tmpl、
    #     dashboard.tmpl、main.py（内容与 initial_setup 相同）...

    # --- 应用标准完成状态补丁：添加 files.associations ---
    os.makedirs(VSCODE_USER, exist_ok=True)
    settings = {}
    if os.path.exists(SETTINGS_PATH):
        try:
            with open(SETTINGS_PATH, 'r') as f:
                settings = json.load(f)
        except (json.JSONDecodeError, FileNotFoundError):
            settings = {}

    settings['files.associations'] = {
        '*.config': 'json',
        '*.tmpl':   'html'
    }

    with open(SETTINGS_PATH, 'w') as f:
        json.dump(settings, f, indent=4)

    print(f'Golden workspace created at: {WORKSPACE}')
    print(f'Settings updated with files.associations: {SETTINGS_PATH}')


create_golden()
```

vscode_lp_081/reward.py

```
"""
奖励脚本：为 *.config（JSON）和 *.tmpl（HTML）配置自定义文件关联
任务 ID：  vscode_lp_081
领域：  vscode
评分：
  - 组件 1（0.35）：settings.json 中存在 files.associations 键
  - 组件 2（0.35）：*.config 映射为 json
  - 组件 3（0.30）：*.tmpl 映射为 html
"""

import os
import json
import re

HOME = '/home/user'
SETTINGS_PATH = os.path.join(HOME, '.config', 'Code', 'User', 'settings.json')
TASK_ID = 'vscode_lp_081'


def load_settings():
    """加载 VSCode 的 settings.json，并处理 JSONC（注释）。"""
    try:
        with open(SETTINGS_PATH, 'r') as f:
```

```
            content = f.read()
        content = re.sub(r'//.*$', '', content, flags=re.MULTILINE)
        content = re.sub(r'/\*.*?\*/', '', content, flags=re.DOTALL)
        return json.loads(content)
    except (FileNotFoundError, json.JSONDecodeError) as e:
        print(f"ERROR: Cannot load settings.json: {e}")
        return None


def verify_task():
    total_score = 0.0

    settings = load_settings()
    if settings is None:
        print("CRITICAL: settings.json not found or invalid")
        print("REWARD: 0.0")
        return 0.0

    # C1: files.associations 键存在且非空 (0.35 分)
    try:
        assoc = settings.get("files.associations")
        if isinstance(assoc, dict) and len(assoc) > 0:
            print(f"PASS: C1 -- files.associations exists with {len(assoc)} entries (0.35
    pts)")
            total_score += 0.35
        else:
            print(f"FAIL: C1 -- files.associations missing or empty, found: {assoc}")
    except Exception as e:
        print(f"ERROR: C1 -- {e}")

    # C2: *.config -> json (0.35 分)
    try:
        assoc = settings.get("files.associations", {})
        config_val = assoc.get("*.config")
        if config_val is not None and str(config_val).lower() == "json":
            print(f"PASS: C2 -- *.config -> {config_val} (0.35 pts)")
            total_score += 0.35
        else:
            print(f"FAIL: C2 -- *.config expected 'json', found: {config_val}")
    except Exception as e:
        print(f"ERROR: C2 -- {e}")

    # C3: *.tmpl -> html (0.30 分)
    try:
        assoc = settings.get("files.associations", {})
        tmpl_val = assoc.get("*.tmpl")
        if tmpl_val is not None and str(tmpl_val).lower() == "html":
            print(f"PASS: C3 -- *.tmpl -> {tmpl_val} (0.30 pts)")
            total_score += 0.30
        else:
            print(f"FAIL: C3 -- *.tmpl expected 'html', found: {tmpl_val}")
    except Exception as e:
        print(f"ERROR: C3 -- {e}")

    final_score = round(min(total_score, 1.0), 2)
    print(f"\nScore: {final_score}/1.0")
    print(f"REWARD: {final_score}")
    return final_score


verify_task()
```

E.1.3 跨应用任务——Slack + Impress + PDF：优先级排序演示文稿

这是一个跨应用任务，将从网页仿真应用（Slack）读取信息、在桌面应用（Impress）中创作，以及最终导出PDF 结合起来。`initial_setup.py` 通过统一状态API（§B.3）向仿真应用注入已填充内容的Slack 工作区状态，随后在Chrome 中打开Slack 地址，并打开一个新的LibreOffice Impress 窗口。`golden_patch.py`

在标准完成状态虚拟机上复用相同的Slack 状态，再使用辅助函数构建一份5 页的PPTX 演示文稿，并导出为PDF。奖励从结构上检查生成的PPTX（幻灯片数量、文本框、表格内容），并验证PDF 产物的存在性与页数。

slack_impress_pdf_36——任务指令（节选）

```
打开 Slack，查看 #product-feedback 频道，找到 Frank Zhang
在开展季度功能优先级投票后发布的功能请求投票结果。
客户与内部利益相关者对 10 个候选功能进行了投票。
打开 LibreOffice Impress，创建一份功能优先级演示文稿，包含：
（1）标题页，（2）投票方法，（3）包含全部 10 个功能的排名结果表，
（4）附工作量估算的前 5 项深入分析，（5）第二季度路线图承诺。
将演示文稿导出为 PDF。
```

奖励分解（7 个组件，总计 1.0）：

- **C1 (0.10)**: PPTX 恰好包含5 张幻灯片。
- **C2 (0.10)**: 标题页包含"Feature Prioritization" 和"Q2 2026"。
- **C3 (0.10)**: 方法页提到28 位客户和12 位内部利益相关者。
- **C4 (0.25)**: 排名结果表包含全部10 个功能及其票数。
- **C5 (0.15)**: 前5 项深入分析页包含工作量估算。
- **C6 (0.15)**: 第二季度路线图页恰好列出从前5 项中选出的3 个承诺交付项。
- **C7 (0.15)**: PDF 产物存在且为5 页。

slack_impress_pdf_36/initial_setup.py （省略Slack 状态注入）

```python
"""
初始设置：在 Slack 的 #product-feedback 频道中设置功能
优先级投票结果，并准备桌面，以便创建演示文稿。
任务 ID:  slack_impress_pdf_36
领域:  cross_app (slack_mock + libreoffice_impress + pdf)
"""
import json, os, shlex, subprocess, time, uuid
import requests

WORKDIR   = '/home/user'
TASK_ID   = 'slack_impress_pdf_36'
SLACK_URL = <self-hosted url>

# --- 生成会话 ID 并持久保存，供智能体 / 奖励使用 ---
sid = str(uuid.uuid4())
with open('/tmp/task_web_sid', 'w') as f:
    f.write(sid)

# --- 构建 Slack 工作区状态（因篇幅而截略）---
# 完整状态对象编码以下内容：
#   currentUser, workspace, users[10], channels[5+#product-feedback],
#   messages.product-feedback（含 Frank Zhang 的投票结果帖子）、
#   threads, dms, settings, notifications.
# 其中，messages.product-feedback 包含一篇长帖，列出 10 个候选
# 功能及其票数（客户、内部、总计）——这些就是智能体必须提取
# 并转化为演示文稿的数据。
slack_state = {
    "currentUser": {"userId": "user_1", "fullName": "John Smith",
                    "displayName": "John", "title": "Product Owner",
                    "timeZone": "America/New_York"},
    "workspace":   {"workspaceId": "ws_1", "workspaceName": "Acme Corp"},
    "users":       [ # ... 10 条用户记录 ... ],
    "channels":    [ # ... 包括 'product-feedback' 在内的 5 个频道 ... ],
    "messages": {
```

```
        "product-feedback": [
            # ... 更早的讨论消息 ...
            {"messageId": "msg_fp_001", "senderId": "user_7",
             "content": "Q1 2026 Feature Prioritization Poll Results\n"
                        "(28 customer votes, 12 internal votes)\n"
                        "1. Bulk data export ......... 22 cust + 9 int = 31\n"
                        "2. AI-powered report summaries .. 18 + 11 = 29\n"
                        "3. Mobile app offline mode .. 20 + 6 = 26\n"
                        "4. Slack integration enhancements .. 14 + 10 = 24\n"
                        "5. Custom dashboard layouts .. 17 + 7 = 24\n"
                        "6. API webhook support .. 9 + 11 = 20\n"
                        "7. Dark mode .. 16 + 4 = 20\n"
                        "8. SSO for sub-accounts .. 12 + 7 = 19\n"
                        "9. Audit log export .. 8 + 9 = 17\n"
                        "10. Custom email templates .. 13 + 3 = 16",
             "timestamp": "2026-04-10T08:30:00Z", "threadId": None,
             "reactions": [], "attachments": [], "isEdited": False},
            # ... 后续回应 / 回复 ...
        ]
    },
    "threads": {}, "dms": [], "settings": { },
    "notifications": [ {"notificationId": "notif_1",
                        "messageId": "msg_fp_001",
                        "channelId": "product-feedback", "read": False} ]
}

# --- 向 Slack 仿真应用注入状态（action="set" => 初始快照）---
resp = requests.post(f'{SLACK_URL}/post?sid={sid}',
                     json={'action': 'set', 'state': slack_state}, timeout=30)
assert resp.status_code == 200, f'Slack state injection failed: {resp.text}'
print(f'Slack state injected: sid={sid}')

# 验证注入后的状态可读。
go = requests.get(f'{SLACK_URL}/go?sid={sid}', timeout=10).json()
assert go['initial_state'] is not None
print('Verified: Slack initial_state and current_state are set')

# --- 准备干净的桌面，移除预先存在的目标文件 ---
DESKTOP = '/home/user/Desktop'
os.makedirs(DESKTOP, exist_ok=True)
for f in ['feature_priority_q2.pptx', 'feature_priority_q2.pdf']:
    path = os.path.join(DESKTOP, f)
    if os.path.exists(path):
        os.remove(path); print(f'Removed pre-existing {path}')

try:
    if not os.path.exists('/root/Desktop'):
        os.symlink(DESKTOP, '/root/Desktop')
except PermissionError:
    pass

# --- 启动 GUI：Chrome 打开 Slack 地址 + 空白 Impress 窗口 ---
def launch_gui(command, delay_sec=1.0):
    env = os.environ.copy(); env['DISPLAY'] = ':0'
    subprocess.Popen(shlex.split(command), stdout=subprocess.DEVNULL,
                     stderr=subprocess.DEVNULL, env=env)
    time.sleep(delay_sec)

launch_gui(f'google-chrome "{SLACK_URL}/?sid={sid}"', delay_sec=3.0)
launch_gui('libreoffice --impress', delay_sec=2.0)
print(f'GUI_READY: Chrome on Slack ({SLACK_URL}/?sid={sid}) + Impress')
```

slack_impress_pdf_36/golden_patch.py （省略状态复用与幻灯片布局）

```
"""
标准完成状态补丁：创建功能优先级演示文稿，并导出为 PDF。
任务 ID：  slack_impress_pdf_36
领域：  cross_app (slack_mock + libreoffice_impress + pdf)
变更：
  - Slack：使标准完成状态虚拟机的会话与初始虚拟机使用的状态
    保持一致，从而使跨虚拟机奖励检查一致。
  - Impress：写入包含 5 张幻灯片的 /home/user/Desktop/feature_priority_q2.pptx。
  - PDF：导出包含 5 页的 /home/user/Desktop/feature_priority_q2.pdf。
"""
import copy, json, os, uuid
import requests
from pptx import Presentation
from pptx.util import Inches, Pt
from pptx.dml.color import RGBColor

DESKTOP     = '/home/user/Desktop'
PPTX_OUTPUT = f'{DESKTOP}/feature_priority_q2.pptx'
PDF_OUTPUT  = f'{DESKTOP}/feature_priority_q2.pdf'
SLACK_URL   = <self-hosted url>

# --- 使用与初始状态相同的 sid 复用 / 注入 Slack 状态 ---
SID_PATH = '/tmp/task_web_sid'
try:
    sid = open(SID_PATH).read().strip()
except FileNotFoundError:
    sid = str(uuid.uuid4())
    open(SID_PATH, 'w').write(sid)

go = requests.get(f'{SLACK_URL}/go?sid={sid}', timeout=10).json()
if go.get('initial_state') is None:
    # 注入 initial_setup.py 使用的相同工作区状态
    # (workspace、users、channels、messages、settings) ——此处
    # 因篇幅省略；发布脚本原样保留该字面量。
    slack_state = { ... }   # 见 initial_setup.py
    requests.post(f'{SLACK_URL}/post?sid={sid}',
                  json={'action': 'set_current', 'state': slack_state},
                  timeout=30)

# --- 幻灯片构建调色板 ---
DARK_BLUE  = RGBColor(0x1B, 0x3A, 0x5C)
MEDIUM_BLUE= RGBColor(0x2E, 0x75, 0xB6)
LIGHT_BLUE = RGBColor(0xD6, 0xE4, 0xF0)
LIGHT_GRAY = RGBColor(0xF2, 0xF2, 0xF2)
GREEN      = RGBColor(0x27, 0xAE, 0x60)
ORANGE     = RGBColor(0xE6, 0x7E, 0x22)
WHITE = RGBColor(0xFF, 0xFF, 0xFF); BLACK = RGBColor(0x00, 0x00, 0x00)

# 幻灯片构建辅助函数 (add_textbox、add_table、add_metric_card) 及
# 第 1 至第 5 张幻灯片的具体布局代码，共以约 400 行构建幻灯片
# 几何布局；以发布脚本为准。概念上包含：
#   幻灯片 1：标题块 + 副标题 "Q2 2026" + 作者页脚
#   幻灯片 2：方法要点 (28 位客户、12 位内部利益相关者、
#             调研时间窗口、加权规则)
#   幻灯片 3：11x4 表格 (1 行表头 + 10 个功能)，列为
#             (排名、功能、客户 + 内部、总计)
#   幻灯片 4：前 5 项深入分析卡片 (所需迭代周期、ROI、依赖项)
#   幻灯片 5：第二季度承诺页 (3 个承诺项 + 7 个推迟至第三季度的项)
prs = Presentation()
# ... 在此构建第 1 至第 5 张幻灯片 ...

prs.save(PPTX_OUTPUT)
print(f'Golden PPTX saved: {PPTX_OUTPUT}')
```

```
# --- 使用 LibreOffice 无界面模式将 PPTX 导出为 PDF ---
import subprocess
subprocess.run(['libreoffice', '--headless', '--convert-to', 'pdf',
                '--outdir', DESKTOP, PPTX_OUTPUT], check=True, timeout=120)
print(f'Golden PDF saved: {PDF_OUTPUT}')
```

slack_impress_pdf_36/reward.py

```
"""
奖励脚本：根据 Slack 数据创建功能优先级演示文稿
任务 ID：  slack_impress_pdf_36
领域：  libreoffice_impress + pdf (cross_app)
评分：
  C1 (0.10) PPTX 恰好包含 5 张幻灯片
  C2 (0.10) 标题页包含 'Feature Prioritization' 和 'Q2 2026'
  C3 (0.10) 方法页提到 28 位客户 + 12 位内部参与者
  C4 (0.25) 排名结果表包含全部 10 个功能及其票数
  C5 (0.15) 前 5 项深入分析页包含工作量估算
  C6 (0.15) 第二季度路线图承诺页包含 3 个承诺项
  C7 (0.15) PDF 存在且为 5 页
"""
import os

WORKDIR   = '/home/user/Desktop'
PPTX_PATH = os.path.join(WORKDIR, 'feature_priority_q2.pptx')
PDF_PATH  = os.path.join(WORKDIR, 'feature_priority_q2.pdf')

# --- 从 Slack 消息中提取的预期数据 ---
EXPECTED_FEATURES = [
    ("Bulk data export",                22,  9, 31),
    ("AI-powered report summaries",     18, 11, 29),
    ("Mobile app offline mode",         20,  6, 26),
    ("Slack integration enhancements",  14, 10, 24),
    ("Custom dashboard layouts",        17,  7, 24),
    ("API webhook support",              9, 11, 20),
    ("Dark mode",                       16,  4, 20),
    ("SSO for sub-accounts",            12,  7, 19),
    ("Audit log export",                 8,  9, 17),
    ("Custom email templates",          13,  3, 16),
]
EFFORT_ESTIMATES = {"bulk": 2, "ai": 5, "offline": 6, "slack": 1, "dashboard": 3}
Q2_COMMITTED     = ["bulk", "slack", "dashboard"]


def get_all_text(slide):
    """从幻灯片中获取文本，包括文本框与表格单元格。"""
    from pptx.enum.shapes import MSO_SHAPE_TYPE
    texts = []
    for shape in slide.shapes:
        if shape.has_text_frame:
            texts.append(shape.text_frame.text)
        if shape.shape_type == MSO_SHAPE_TYPE.TABLE:
            tbl = shape.table
            for r in range(len(tbl.rows)):
                for c in range(len(tbl.columns)):
                    texts.append(tbl.cell(r, c).text)
    return texts


def get_table_data(slide):
    """若幻灯片中存在表格，则提取其各行。"""
    from pptx.enum.shapes import MSO_SHAPE_TYPE
```

```
    for shape in slide.shapes:
        if shape.shape_type == MSO_SHAPE_TYPE.TABLE:
            tbl = shape.table
            return [[tbl.cell(r, c).text.strip()
                     for c in range(len(tbl.columns))]
                    for r in range(len(tbl.rows))]
    return None


def verify_task():
    total_score = 0.0
    if not os.path.exists(PPTX_PATH):
        print(f"CRITICAL: PPTX not found at {PPTX_PATH}\nREWARD: 0.0")
        return 0.0

    try:
        from pptx import Presentation
        prs = Presentation(PPTX_PATH)
    except Exception as e:
        print(f"CRITICAL: cannot load PPTX: {e}\nREWARD: 0.0"); return 0.0

    # ----------------- C1: 5 张幻灯片 (0.10) -----------------
    try:
        n = len(prs.slides)
        if n == 5:
            print("PASS: C1 -- PPTX has 5 slides (0.10 pts)")
            total_score += 0.10
        else:
            print(f"FAIL: C1 -- expected 5 slides, found {n}")
    except Exception as e:
        print(f"ERROR: C1 -- {e}")

    # ----------------- C2: 标题文本 (0.10) -----------------
    try:
        combined = " ".join(get_all_text(prs.slides[0])).lower()
        has_feature = "feature prioritization" in combined
        has_q2_2026 = ("q2 2026" in combined or
                       ("q2" in combined and "2026" in combined))
        if has_feature and has_q2_2026:
            print("PASS: C2 -- title slide has 'Feature Prioritization' + 'Q2 2026' (0.10
    pts)")
            total_score += 0.10
        else:
            print(f"FAIL: C2 -- title text: {combined[:200]}")
    except Exception as e:
        print(f"ERROR: C2 -- {e}")

    # ----------------- C3: 方法提到 28 + 12 (0.10) -----------------
    try:
        combined = " ".join(get_all_text(prs.slides[1])).lower()
        has_28      = "28" in combined
        has_12      = "12" in combined
        has_keyword = any(k in combined for k in
                          ("methodology","voting","method","participants"))
        if has_28 and has_12 and has_keyword:
            print("PASS: C3 -- methodology mentions 28 + 12 (0.10 pts)")
            total_score += 0.10
        else:
            print(f"FAIL: C3 -- has_28={has_28}, has_12={has_12}, "
                  f"has_keyword={has_keyword}")
    except Exception as e:
        print(f"ERROR: C3 -- {e}")

    # ----------------- C4: 排名结果表 (0.25) -----------------
    # 功能名称存在最多得 0.125 分，总票数匹配最多得 0.125 分。
    try:
        table_data = get_table_data(prs.slides[2])
```

```
    if table_data is None:
        for slide in prs.slides:
            td = get_table_data(slide)
            if td is not None and len(td) >= 10:
                table_data = td; break

    if table_data is not None and len(table_data) >= 11:  # 表头 + 10 行
        features_found = 0; votes_correct = 0
        for fname, _, _, total in EXPECTED_FEATURES:
            fl = fname.lower()
            for row in table_data[1:]:
                row_text = " ".join(row).lower()
                if fl in row_text:
                    features_found += 1
                    if str(total) in " ".join(row):
                        votes_correct += 1
                    break
        c4 = 0.125 * (features_found / 10) + 0.125 * (votes_correct / 10)
        if c4 > 0:
            print(f"PASS: C4 -- {features_found}/10 features, "
                  f"{votes_correct}/10 votes ({c4:.3f} pts)")
            total_score += c4
        else:
            print("FAIL: C4 -- table found but no features matched")
    else:
        # 回退：在第 3 张幻灯片的自由文本中搜索功能名称。
        combined = " ".join(get_all_text(prs.slides[2])).lower()
        features_found = sum(1 for f, _, _, _ in EXPECTED_FEATURES
                             if f.lower() in combined)
        if features_found >= 8:
            c4 = 0.15 * (features_found / 10)
            print(f"PARTIAL: C4 -- no table; {features_found}/10 features "
                  f"in text ({c4:.3f} pts)")
            total_score += c4
        else:
            print(f"FAIL: C4 -- no table and only {features_found}/10 "
                  f"features in text")
except Exception as e:
    print(f"ERROR: C4 -- {e}")

# ----------------- C5：含工作量的前 5 项深入分析（0.15） -----------------
try:
    combined = " ".join(get_all_text(prs.slides[3])).lower()
    top5 = [f.lower() for f, _, _, _ in EXPECTED_FEATURES[:5]]
    top5_found = sum(1 for f in top5 if f in combined)

    effort_found = 0
    if "2 sprint" in combined and "bulk" in combined:        effort_found += 1
    if "5 sprint" in combined and ("ai" in combined or "report" in combined):
        effort_found += 1
    if "6 sprint" in combined and ("offline" in combined or "mobile" in combined):
        effort_found += 1
    if "1 sprint" in combined and "slack" in combined:        effort_found += 1
    if "3 sprint" in combined and "dashboard" in combined:    effort_found += 1

    feat_score   = 0.075 * (top5_found / 5)
    effort_score = 0.075 * (effort_found / 5)
    c5 = feat_score + effort_score
    if c5 > 0:
        print(f"PASS: C5 -- {top5_found}/5 features, {effort_found}/5 "
              f"efforts ({c5:.3f} pts)")
        total_score += c5
    else:
        print("FAIL: C5 -- missing top-5 features or effort estimates")
except Exception as e:
    print(f"ERROR: C5 -- {e}")
```

```
    # ----------------- C6: 第二季度路线图承诺 (0.15) -----------------
    try:
        combined = " ".join(get_all_text(prs.slides[4])).lower()
        has_roadmap = any(k in combined for k in ("roadmap","commitment","q2"))
        committed = 0
        if "bulk"     in combined and ("export" in combined or "data" in combined):
            committed += 1
        if "slack"    in combined and ("integration" in combined or "enhancement" in
    combined):
            committed += 1
        if "dashboard"in combined and ("custom" in combined or "layout" in combined):
            committed += 1
        has_deferred = ("defer" in combined or "q3" in combined)

        c6 = 0.0
        if has_roadmap:  c6 += 0.03
        c6 += 0.10 * (committed / 3)
        if has_deferred: c6 += 0.02
        c6 = min(c6, 0.15)
        if c6 > 0:
            print(f"PASS: C6 -- roadmap={has_roadmap}, {committed}/3 committed, "
                  f"deferred={has_deferred} ({c6:.3f} pts)")
            total_score += c6
        else:
            print("FAIL: C6 -- missing roadmap content")
    except Exception as e:
        print(f"ERROR: C6 -- {e}")

    # ----------------- C7: 5 页 PDF (0.15) -----------------
    try:
        if os.path.exists(PDF_PATH):
            import pymupdf
            doc = pymupdf.open(PDF_PATH); pages = doc.page_count
            page1 = doc[0].get_text().lower()
            content_ok = "feature" in page1 and "prioritization" in page1
            if pages == 5 and content_ok:
                print("PASS: C7 -- PDF has 5 pages with matching content (0.15 pts)")
                total_score += 0.15
            elif pages == 5:
                print("PARTIAL: C7 -- 5 pages but page-1 text mismatch (0.10 pts)")
                total_score += 0.10
            else:
                print(f"PARTIAL: C7 -- PDF exists with {pages} pages (0.05 pts)")
                total_score += 0.05
            doc.close()
        else:
            print(f"FAIL: C7 -- PDF not found at {PDF_PATH}")
    except Exception as e:
        print(f"ERROR: C7 -- {e}")

    final = min(total_score, 1.0)
    print(f"\nScore: {total_score:.3f}/1.0\nREWARD: {final}")
    return final


verify_task()
```

E.1.4 跨应用任务——PDF → Calc：多币种费用分析

这是一个长时程跨应用任务。智能体必须从PDF 中提取结构化表格数据，用验证公式填充电子表格，使用条件格式标记异常项，用SUMIFS 构建汇总工作表，并添加图表。奖励分解为六个可独立检查的组件，其中两个依赖单元格层面的数值计算，两个依赖单元格格式，另两个依赖派生工作表结构。`initial_setup.py` 使用PyMuPDF 从零合成一份包含 50 行、5 页的PDF 源文档；`golden_patch.py` 写入任务完成后的工作簿，其中包含相同的表格内容，以及验证公式、填充色、汇总工作表和图表。

l3_pcb1_040——任务指令（节选）

```
从 ~/Documents/global_expenses.pdf 提取多币种费用数据
（列：Expense ID、Date、Category、Local Amount、Currency、
Exchange Rate to USD、USD Amount、Approver）。
在 Calc 中创建 ~/Documents/global_expenses.xlsx。添加：
（1）在 I 列使用 Local Amount x Exchange Rate 公式验证 USD Amount，
（2）差异列 J = ABS(G − I)，（3）将差异 > $0.01 的行填充为黄色，
（4）使用 SUMIFS 汇总的币种敞口汇总工作表，
（5）币种敞口饼图。
```

奖励分解（6 个组件，总计 1.0）：

- **C1 (0.20)**: 文件存在，包含50 行数据及预期的表头集合。
- **C2 (0.15)**: I 列使用形如=D*F 的验证公式。
- **C3 (0.15)**: J 列使用形如=ABS(G-I) 的差异公式。
- **C4 (0.20)**: 恰好六行以黄色填充标记。
- **C5 (0.15)**: 存在包含SUMIFS 聚合的Currency Exposure Summary 工作表。
- **C6 (0.15)**: 汇总工作表上锚定有一张饼图。

该任务体现了这样一个设计原则：任何单一组件都不足以完成任务。智能体若解决数据提取与公式子问题，却跳过图表，得分为 0.85；若构建图表却跳过差异列，得分同样为 0.85；只有完成全部工作流程的智能体才能获得 1.0。

l3_pcb1_040/initial_setup.py

```python
"""
初始设置：从 PDF 提取多币种费用数据并创建 xlsx
任务 ID:  l3_pcb1_040
领域:  multi_apps (PDF + LibreOffice Calc)
"""

import os, shlex, subprocess, time

WORKDIR = '/home/user'
DOCUMENTS = f'{WORKDIR}/Documents'
TASK_ID = 'l3_pcb1_040'
PDF_OUTPUT = f'{DOCUMENTS}/global_expenses.pdf'

def launch_gui(command, delay_sec=1.0):
    env = os.environ.copy(); env["DISPLAY"] = ":0"
    subprocess.Popen(shlex.split(command),
        stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL, env=env)
    time.sleep(delay_sec)

def create_initial():
    os.makedirs(DOCUMENTS, exist_ok=True)
    import pymupdf

    # ---- 定义涉及 6 种币种的 50 行费用数据 ----
    # 其中 6 行的舍入差异将 > $0.01。
    categories = ['Travel', 'Office Supplies', 'Meals & Entertainment',
                  'Software', 'Consulting', 'Telecom', 'Marketing',
                  'Training', 'Shipping', 'Utilities']
    approvers = ['Sarah Chen', 'Marcus Johnson', 'Elena Rodriguez', 'James Liu',
                 'Priya Sharma', 'David Kim', 'Rachel Foster', 'Thomas Becker']
    exchange_rates = {'USD': 1.0000, 'EUR': 1.0842, 'GBP': 1.2635,
                      'JPY': 0.0067, 'CNY': 0.1382, 'CAD': 0.7415}
    discrepancy_indices = {7, 15, 22, 31, 38, 45}
    currency_cycle = ['USD', 'EUR', 'GBP', 'JPY', 'CNY', 'CAD']
```

```
local_amounts = [
    1250.00, 890.50, 3200.00, 475.25, 15600.00, 2100.75, 680.00,
    45000.00, 1890.30, 520.00, 3450.00, 760.25, 28500.00, 1125.50,
    4200.00, 950.75, 125000.00, 2340.60, 810.00, 6750.00, 1575.25,
    430.00, 89000.00, 3680.40, 1290.50, 2850.00, 615.75, 52000.00,
    1740.20, 5100.00, 985.50, 375000.00, 2960.80, 1450.00, 7200.25,
    1680.75, 540.00, 167000.00, 4120.30, 2250.00, 3150.50, 825.25,
    41500.00, 1960.60, 5800.00, 1340.00, 710.50, 93000.00, 2580.40,
    3900.75,
]
dates = [f'2025-{m:02d}-{d:02d}' for (m, d) in [
    (1,8),(1,12),(1,15),(1,19),(1,22),(1,27),(2,3),(2,7),(2,11),(2,14),
    (2,18),(2,22),(2,26),(3,2),(3,5),(3,9),(3,13),(3,17),(3,20),(3,24),
    (3,28),(4,1),(4,4),(4,8),(4,11),(4,15),(4,19),(4,23),(4,26),(4,30),
    (5,3),(5,7),(5,10),(5,14),(5,18),(5,22),(5,25),(5,29),(6,2),(6,5),
    (6,9),(6,13),(6,16),(6,20),(6,23),(6,27),(7,1),(7,4),(7,8),(7,11),
]]

expenses = []
for i in range(50):
    exp_id = f'EXP-2025-{1001 + i}'
    currency = currency_cycle[i % 6]
    rate = exchange_rates[currency]
    local_amount = local_amounts[i]
    correct_usd = round(local_amount * rate, 2)
    if i in discrepancy_indices:
        offsets = [0.05, -0.03, 0.08, -0.04, 0.06, -0.02]
        disc_idx = sorted(discrepancy_indices).index(i)
        usd_amount = round(correct_usd + offsets[disc_idx], 2)
    else:
        usd_amount = correct_usd
    expenses.append((exp_id, dates[i], categories[i % 10], local_amount,
                     currency, rate, usd_amount, approvers[i % 8]))

# ---- 将费用数据渲染为 5 页 PDF ----
doc = pymupdf.open()
header = ['Expense ID', 'Date', 'Category', 'Local Amount', 'Currency',
          'Exchange Rate', 'USD Amount', 'Approver']
page_width, page_height = 842, 595   # A4 横向
margin_left, margin_top = 40, 60
col_widths = [85, 70, 110, 80, 60, 80, 80, 100]
row_height, header_height = 18, 22
rows_per_page = 24

def draw_header(pg, y):
    x = margin_left
    for j, h in enumerate(header):
        rect = pymupdf.Rect(x, y, x + col_widths[j], y + header_height)
        pg.draw_rect(rect, color=(0.2,0.2,0.5), fill=(0.2,0.3,0.6))
        pg.insert_textbox(rect, h, fontsize=8, fontname="helv",
                          color=(1,1,1), align=pymupdf.TEXT_ALIGN_CENTER)
        x += col_widths[j]
    return y + header_height

def draw_row(pg, y, row_data, row_idx):
    x = margin_left
    bg = (0.95,0.95,0.98) if row_idx % 2 == 0 else (1,1,1)
    values = [row_data[0], row_data[1], row_data[2],
              f'{row_data[3]:,.2f}', row_data[4],
              f'{row_data[5]:.4f}', f'${row_data[6]:,.2f}', row_data[7]]
    for j, v in enumerate(values):
        rect = pymupdf.Rect(x, y, x + col_widths[j], y + row_height)
        pg.draw_rect(rect, color=(0.8,0.8,0.8), fill=bg)
        align = (pymupdf.TEXT_ALIGN_RIGHT if j in (3,5,6)
                 else pymupdf.TEXT_ALIGN_LEFT)
        pg.insert_textbox(rect, v, fontsize=7, fontname="helv",
                          color=(0.1,0.1,0.1), align=align)
```

```
            x += col_widths[j]
        return y + row_height

    # 第 1 页：标题 + 前 20 行
    page = doc.new_page(width=page_width, height=page_height)
    page.insert_textbox(pymupdf.Rect(margin_left, 20, page_width-40, 45),
        "Global Multi-Currency Expense Report", fontsize=16, fontname="helv",
        color=(0.1,0.1,0.4))
    page.insert_textbox(pymupdf.Rect(margin_left, 45, page_width-40, 60),
        "Fiscal Year 2025 - Q1/Q2 Summary", fontsize=10, fontname="helv",
        color=(0.3,0.3,0.3))
    y = draw_header(page, 85)
    row_counter = 0
    for i in range(min(20, len(expenses))):
        y = draw_row(page, y, expenses[i], i); row_counter += 1

    # 第 2 至第 5 页：剩余行或备注摘要
    remaining = expenses[row_counter:]; page_num = 2
    while remaining:
        page = doc.new_page(width=page_width, height=page_height)
        y = draw_header(page, 30)
        page_rows = remaining[:rows_per_page]
        for i, exp in enumerate(page_rows):
            y = draw_row(page, y, exp, row_counter + i)
        row_counter += len(page_rows); remaining = remaining[len(page_rows):]
        page_num += 1
    while page_num <= 5:
        page = doc.new_page(width=page_width, height=page_height)
        page.insert_textbox(pymupdf.Rect(margin_left, 30, page_width-40, 50),
            "Notes and Reconciliation Summary", fontsize=12, fontname="helv",
            color=(0.1,0.1,0.4))
        page.insert_textbox(pymupdf.Rect(margin_left, 55, page_width-40, 200),
            "This report contains 50 expense entries across 6 currencies. "
            "Any discrepancies between Local Amount times Exchange Rate and "
            "the stated USD Amount should be flagged and reviewed.",
            fontsize=9, fontname="helv", color=(0.2,0.2,0.2))
        page_num += 1

    doc.save(PDF_OUTPUT); doc.close()
    print(f'PDF created: {PDF_OUTPUT} with {len(expenses)} expense rows')

    launch_gui(f'evince "{PDF_OUTPUT}"', delay_sec=2.0)
    launch_gui('libreoffice --calc', delay_sec=2.0)
    print('GUI_READY: launched Evince and LibreOffice Calc with DISPLAY=:0')

create_initial()
```

l3_pcb1_040/golden_patch.py

```
"""
标准完成状态补丁：包含验证公式、黄色标记、汇总和饼图的
多币种费用 xlsx。
任务 ID：  l3_pcb1_040
领域：  multi_apps (PDF + LibreOffice Calc)
"""

import openpyxl
from openpyxl.styles import Font, PatternFill, Alignment, Border, Side
from openpyxl.chart import PieChart, Reference

WORKDIR = '/home/user'
TASK_ID = 'l3_pcb1_040'
OUTPUT = f'{WORKDIR}/Documents/{TASK_ID}.xlsx'
```

```
def create_golden():
    import os
    os.makedirs(f'{WORKDIR}/Documents', exist_ok=True)
    wb = openpyxl.Workbook()

    # ===== 工作表 1: Expense Data =====
    ws = wb.active; ws.title = "Expense Data"
    headers = ['Expense ID', 'Date', 'Category', 'Local Amount', 'Currency',
               'Exchange Rate to USD', 'USD Amount', 'Approver',
               'Verified USD', 'Discrepancy']
    header_font  = Font(name='Calibri', size=11, bold=True, color='FFFFFF')
    header_fill  = PatternFill(start_color='FF2F5496',
                               end_color='FF2F5496', fill_type='solid')
    header_align = Alignment(horizontal='center', vertical='center',
                             wrap_text=True)
    thin_border  = Border(left=Side(style='thin'), right=Side(style='thin'),
                          top=Side(style='thin'),  bottom=Side(style='thin'))
    for col, h in enumerate(headers, 1):
        c = ws.cell(row=1, column=col, value=h)
        c.font = header_font; c.fill = header_fill
        c.alignment = header_align; c.border = thin_border

    # 初始数据列表（categories、approvers、exchange_rates、currency_cycle、
    # local_amounts、dates、discrepancy_indices）与 initial_setup.py
    # 中相同，此处为避免重复而省略；发布脚本原样重现这些列表。
    # ...（约 50 行初始数据列表）...
    yellow_fill = PatternFill(start_color='FFFFFF00',
                              end_color='FFFFFF00', fill_type='solid')
    discrepancy_rows = []

    for i in range(50):
        row = i + 2
        exp_id = f'EXP-2025-{1001 + i}'
        currency = currency_cycle[i % 6]; rate = exchange_rates[currency]
        local_amount = local_amounts[i]
        correct_usd = round(local_amount * rate, 2)
        if i in discrepancy_indices:
            offsets = [0.05, -0.03, 0.08, -0.04, 0.06, -0.02]
            disc_idx = sorted(discrepancy_indices).index(i)
            usd_amount = round(correct_usd + offsets[disc_idx], 2)
            discrepancy_rows.append(row)
        else:
            usd_amount = correct_usd

        ws.cell(row=row, column=1,  value=exp_id)
        ws.cell(row=row, column=2,  value=dates[i])
        ws.cell(row=row, column=3,  value=categories[i % 10])
        c_local = ws.cell(row=row, column=4, value=local_amount)
        c_local.number_format = '#,##0.00'
        ws.cell(row=row, column=5, value=currency)
        c_rate = ws.cell(row=row, column=6, value=rate)
        c_rate.number_format = '0.0000'
        c_usd = ws.cell(row=row, column=7, value=usd_amount)
        c_usd.number_format = '$#,##0.00'
        ws.cell(row=row, column=8, value=approvers[i % 8])
        # I 列：验证公式 = 本币金额 * 汇率
        c_verify = ws.cell(row=row, column=9, value=f'=D{row}*F{row}')
        c_verify.number_format = '$#,##0.00'
        # J 列：差异 = ABS（美元金额 - 验证后的美元金额）
        c_disc = ws.cell(row=row, column=10, value=f'=ABS(G{row}-I{row})')
        c_disc.number_format = '$#,##0.00'
        for col in range(1, 11):
            ws.cell(row=row, column=col).border = thin_border

    # 将六个有差异的行填充为黄色。
    for disc_row in discrepancy_rows:
        for col in range(1, 11):
```

```
            ws.cell(row=disc_row, column=col).fill = yellow_fill

    ws.freeze_panes = 'A2'
    for col_letter, w in {'A':16,'B':12,'C':22,'D':14,'E':10,
                          'F':18,'G':14,'H':18,'I':14,'J':14}.items():
        ws.column_dimensions[col_letter].width = w

    # ===== 工作表 2: Currency Exposure Summary =====
    ws2 = wb.create_sheet("Currency Exposure Summary")
    for col, h in enumerate(['Currency', 'Total USD Exposure'], 1):
        c = ws2.cell(row=1, column=col, value=h)
        c.font = header_font; c.fill = header_fill
        c.alignment = header_align; c.border = thin_border

    currencies = ['USD', 'EUR', 'GBP', 'JPY', 'CNY', 'CAD']
    for idx, curr in enumerate(currencies):
        row = idx + 2
        ws2.cell(row=row, column=1, value=curr).border = thin_border
        formula = (f"=SUMIFS('Expense Data'!G2:G51,"
                   f"'Expense Data'!E2:E51,A{row})")
        c_total = ws2.cell(row=row, column=2, value=formula)
        c_total.number_format = '$#,##0.00'; c_total.border = thin_border

    total_row = len(currencies) + 2
    ws2.cell(row=total_row, column=1, value='Total').font = Font(bold=True)
    c_grand = ws2.cell(row=total_row, column=2,
                       value=f'=SUM(B2:B{total_row-1})')
    c_grand.number_format = '$#,##0.00'; c_grand.font = Font(bold=True)

    # ===== 饼图 =====
    pie = PieChart()
    pie.title = "USD Exposure by Currency"; pie.style = 10
    data_ref = Reference(ws2, min_col=2, min_row=1, max_row=7)
    cats_ref = Reference(ws2, min_col=1, min_row=2, max_row=7)
    pie.add_data(data_ref, titles_from_data=True)
    pie.set_categories(cats_ref); pie.width = 18; pie.height = 12
    ws2.add_chart(pie, "D2")

    wb.save(OUTPUT)
    print(f'Golden file created: {OUTPUT}')
    print(f'Discrepancy rows (yellow fill): {discrepancy_rows}')

create_golden()
```

l3_pcb1_040/reward.py

```
"""
奖励脚本: 多币种费用 xlsx 验证。
任务 ID:  l3_pcb1_040
领域:  multi_apps (PDF -> LibreOffice Calc)
评分 (6 个组件, 总计 1.0) :
  C1 (0.20) : 文件包含 50 行数据和正确表头
  C2 (0.15) : I 列包含 =D*F 验证公式
  C3 (0.15) : J 列包含 =ABS(G-I) 差异公式
  C4 (0.20) : 恰好 6 行以黄色填充标记
  C5 (0.15) : Currency Exposure Summary 工作表包含 SUMIFS
  C6 (0.15) : 汇总工作表上包含饼图
"""

import os, re

WORKDIR = '/home/user/Documents'
TASK_ID = 'l3_pcb1_040'
```

```
def verify_task(file_path):
    total_score = 0.0
    try:
        import openpyxl
        wb = openpyxl.load_workbook(file_path)
    except Exception as e:
        print(f"CRITICAL: Cannot load {file_path}: {e}")
        print("REWARD: 0.0"); return 0.0

    # ----------------- C1: 文件结构 (0.20) -----------------
    try:
        data_sheet = None
        for sn in wb.sheetnames:
            ws_check = wb[sn]
            v = ws_check.cell(row=1, column=1).value
            if v and 'expense' in str(v).lower():
                data_sheet = ws_check; break
        if data_sheet is None:
            data_sheet = wb.worksheets[0]

        expected = ['expense id','date','category','local amount','currency',
                    'exchange rate','usd amount','approver']
        actual = [str(data_sheet.cell(row=1, column=c).value).lower().strip()
                  for c in range(1, data_sheet.max_column + 1)
                  if data_sheet.cell(row=1, column=c).value]
        match = sum(1 for e in expected if any(e in a for a in actual))

        n = sum(1 for r in range(2, data_sheet.max_row + 1)
                if data_sheet.cell(row=r, column=1).value is not None)

        if match >= 6 and 45 <= n <= 55:
            print(f"PASS: C1 -- {n} rows, {match}/8 headers (0.20 pts)")
            total_score += 0.20
        else:
            print(f"FAIL: C1 -- {n} rows (need ~50), {match}/8 headers")
    except Exception as e:
        print(f"ERROR: C1 -- {e}")

    # ----------------- C2: I 列验证公式 (0.15) -----------------
    try:
        verify_col = None
        for c in range(1, data_sheet.max_column + 1):
            h = data_sheet.cell(row=1, column=c).value
            if h and 'verif' in str(h).lower():
                verify_col = c; break
        if verify_col is None:
            sample = data_sheet.cell(row=2, column=9).value
            if sample and isinstance(sample, str) and '=' in sample:
                verify_col = 9
        formula_count = 0
        if verify_col:
            for r in range(2, data_sheet.max_row + 1):
                v = data_sheet.cell(row=r, column=verify_col).value
                if v and isinstance(v, str):
                    u = v.upper().replace(' ', '')
                    if '=' in u and '*' in u:
                        formula_count += 1
        if formula_count >= 45:
            print(f"PASS: C2 -- {formula_count} formulas in col {verify_col} (0.15 pts)")
            total_score += 0.15
        else:
            print(f"FAIL: C2 -- only {formula_count} (need >= 45)")
    except Exception as e:
        print(f"ERROR: C2 -- {e}")

    # ----------------- C3: J 列 ABS 差异公式 (0.15) -----------------
```

```
try:
    discrep_col = None
    for c in range(1, data_sheet.max_column + 1):
        h = data_sheet.cell(row=1, column=c).value
        if h and 'discrep' in str(h).lower():
            discrep_col = c; break
    if discrep_col is None:
        sample = data_sheet.cell(row=2, column=10).value
        if sample and isinstance(sample, str) and 'ABS' in str(sample).upper():
            discrep_col = 10
    abs_count = 0
    if discrep_col:
        for r in range(2, data_sheet.max_row + 1):
            v = data_sheet.cell(row=r, column=discrep_col).value
            if v and isinstance(v, str) and 'ABS' in v.upper().replace(' ', ''):
                abs_count += 1
    if abs_count >= 45:
        print(f"PASS: C3 -- {abs_count} ABS formulas in col {discrep_col} (0.15 pts)")
        total_score += 0.15
    else:
        print(f"FAIL: C3 -- only {abs_count} (need >= 45)")
except Exception as e:
    print(f"ERROR: C3 -- {e}")

# ----------------- C4: 恰好 6 行填充为黄色 (0.20) -----------------
try:
    yellow_rows = set()
    for r in range(2, data_sheet.max_row + 1):
        for c in range(1, data_sheet.max_column + 1):
            cell = data_sheet.cell(row=r, column=c)
            try:
                rgb = (cell.fill.fgColor.rgb or '').upper()
                if len(rgb) == 8:
                    a, R, G, B = (int(rgb[i:i+2], 16) for i in (0,2,4,6))
                    if a > 0 and R > 200 and G > 200 and B < 100:
                        yellow_rows.add(r); break
            except Exception:
                pass
    n = len(yellow_rows)
    if n == 6:
        print(f"PASS: C4 -- exactly 6 yellow rows: {sorted(yellow_rows)} (0.20 pts)")
        total_score += 0.20
    elif 4 <= n <= 8:
        print(f"PARTIAL: C4 -- {n} yellow rows, partial credit (0.10 pts)")
        total_score += 0.10
    else:
        print(f"FAIL: C4 -- {n} yellow rows (expected 6)")
except Exception as e:
    print(f"ERROR: C4 -- {e}")

# ----------------- C5: Currency Exposure Summary 包含 SUMIFS (0.15) -----------------
try:
    summary_sheet = None
    for sn in wb.sheetnames:
        if any(k in sn.lower() for k in ('summar','exposure','currency')):
            summary_sheet = wb[sn]; break

    if summary_sheet is None:
        print("FAIL: C5 -- no summary sheet found")
    else:
        need = {'USD','EUR','GBP','JPY','CNY','CAD'}
        found = set(); sumifs_count = 0
        for r in range(1, summary_sheet.max_row + 1):
            v = summary_sheet.cell(row=r, column=1).value
            f = summary_sheet.cell(row=r, column=2).value
            if v and str(v).upper() in need:
                found.add(str(v).upper())
```

```
                    if f and isinstance(f, str) and 'SUMIF' in f.upper():
                        sumifs_count += 1
            if found >= need and sumifs_count >= 5:
                print(f"PASS: C5 -- {len(found)} currencies, {sumifs_count} SUMIFS (0.15 pts
    )")
                total_score += 0.15
            elif sumifs_count >= 3:
                print(f"PARTIAL: C5 -- {sumifs_count} SUMIFS, {len(found)} currencies (0.08
    pts)")
                total_score += 0.08
            else:
                print(f"FAIL: C5 -- {sumifs_count} SUMIFS, {len(found)} currencies")
    except Exception as e:
        print(f"ERROR: C5 -- {e}")

    # ----------------- C6: 存在饼图 (0.15) -----------------
    try:
        chart_found = False
        for sn in wb.sheetnames:
            for ch in wb[sn]._charts:
                if 'pie' in ch.__class__.__name__.lower():
                    print(f"PASS: C6 -- PieChart on '{sn}' (0.15 pts)")
                    total_score += 0.15; chart_found = True; break
            if chart_found: break
        if not chart_found:
            for sn in wb.sheetnames:
                if wb[sn]._charts:
                    print(f"PARTIAL: C6 -- non-pie chart on '{sn}' (0.08 pts)")
                    total_score += 0.08; break
            else:
                print("FAIL: C6 -- no chart found")
    except Exception as e:
        print(f"ERROR: C6 -- {e}")

    final = min(round(total_score, 2), 1.0)
    print(f"\nScore: {total_score}/1.0\nREWARD: {final}")
    return final

# 入口
file_path = f'{WORKDIR}/{TASK_ID}.xlsx'
alt_path  = f'{WORKDIR}/global_expenses.xlsx'
target = file_path if os.path.exists(file_path) else (
         alt_path  if os.path.exists(alt_path) else None)
if target:
    verify_task(target)
else:
    print(f"File not found: {file_path} or {alt_path}\nREWARD: 0.0")
```

E.2 完整采样轨迹：Shopify 供应商整合

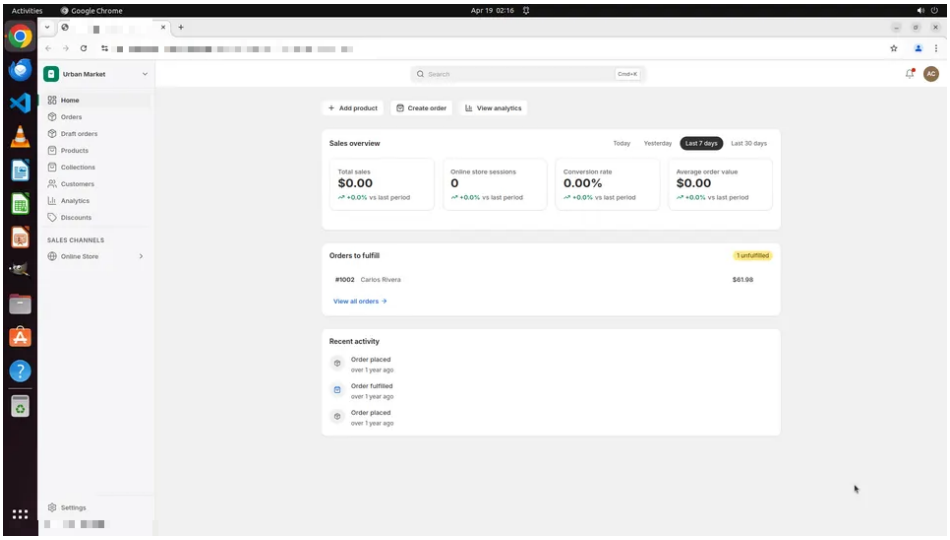
我们逐步完整展示一条验证集轨迹，该轨迹来自对一个经RL 训练的检查点在合成的Shopify 管理后台仿真应用上进行的OSWorld-Verified 评估（`shopify_adv_005`，终止奖励 1.0，19 轮）。任务要求策略：(i) 识别属于两个指定供应商（`BasicWear`、`HomeGoods`）的商品；(ii) 将每件商品的供应商更新为`UnifiedBrands`；(iii) 在每件受影响商品的描述末尾添加固定的营销文案；(iv) 保持其他所有商品不变。每一步展示策略观察的屏幕、工具调用前输出的自然语言推理前言，以及原始`<tool_call>`块。该轨迹展示了正文指出的若干模式：在单轮中串联确定性动作（第13–14、16–18 步）、逐商品完成检查（第15步），以及导航失误后§4.3 所述的恢复模式（前一次点击未打开目标页面后，第17 步返回商品列表）。

任务指令

```
我想整合供应商。将目前属于 'BasicWear' 和 'HomeGoods' 的所有
商品的供应商改为 'UnifiedBrands'。然后更新这些商品的描述，
在末尾添加 'Now part of the UnifiedBrands family'。
```

不要改动其他供应商的商品。

第1步。

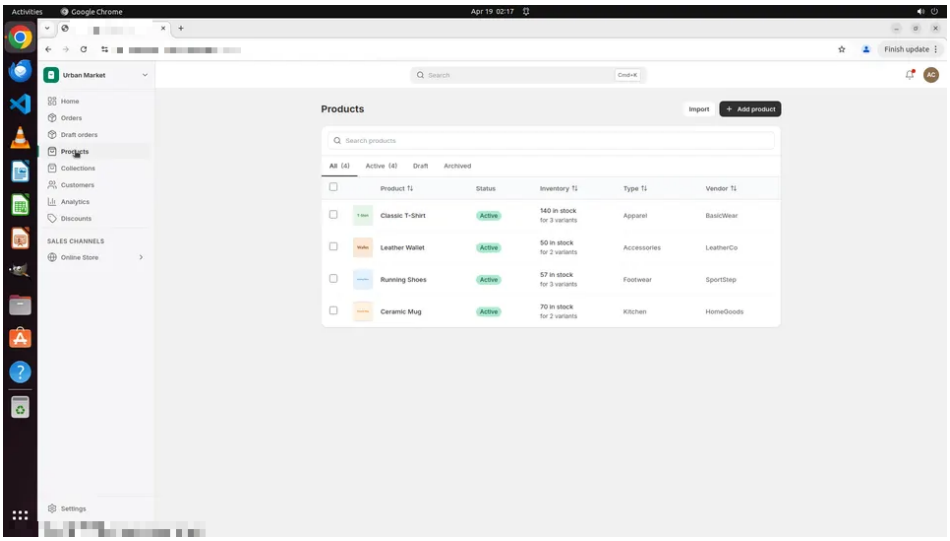


思考： 我看到了Shopify 仿真管理后台的仪表盘。先进入Products 区域查看所有商品。

动作

```
<tool_call>
<function=computer_use>
<parameter=action>left_click</parameter>
<parameter=coordinate>[72, 270]</parameter>
</function>
</tool_call>
```

第2步。

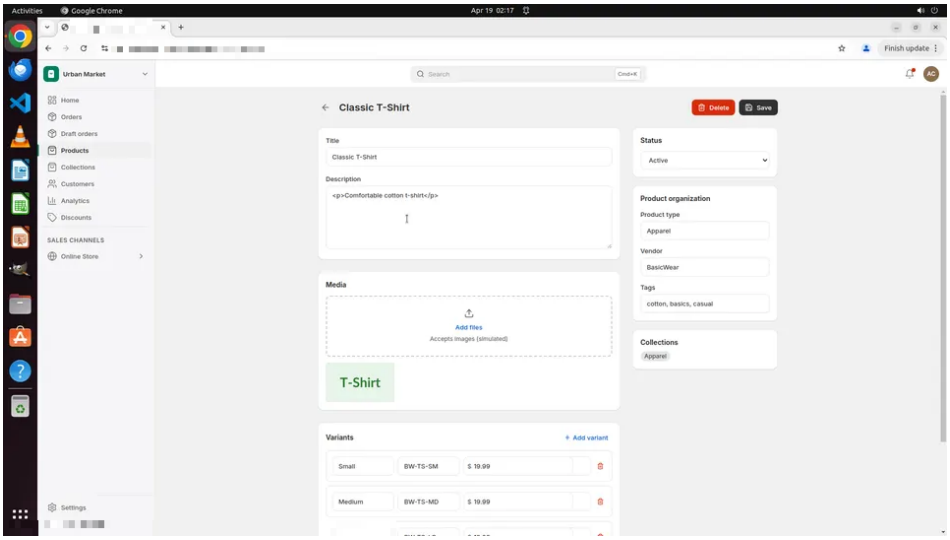


思考： 我看到了Products 页面，包含4 件商品：*Classic T-Shirt*（BasicWear）、*Leather Wallet*（LeatherCo）、*Running Shoes*（SportStep）和*Ceramic Mug*（HomeGoods）。我只需更新BasicWear 和HomeGoods 商品的供应商。先点击*Classic T-Shirt* 进行编辑。

动作

```
<tool_call>
<function=computer_use>
<parameter=action>left_click</parameter>
<parameter=coordinate>[422, 393]</parameter>
</function>
</tool_call>
```

第3 步。

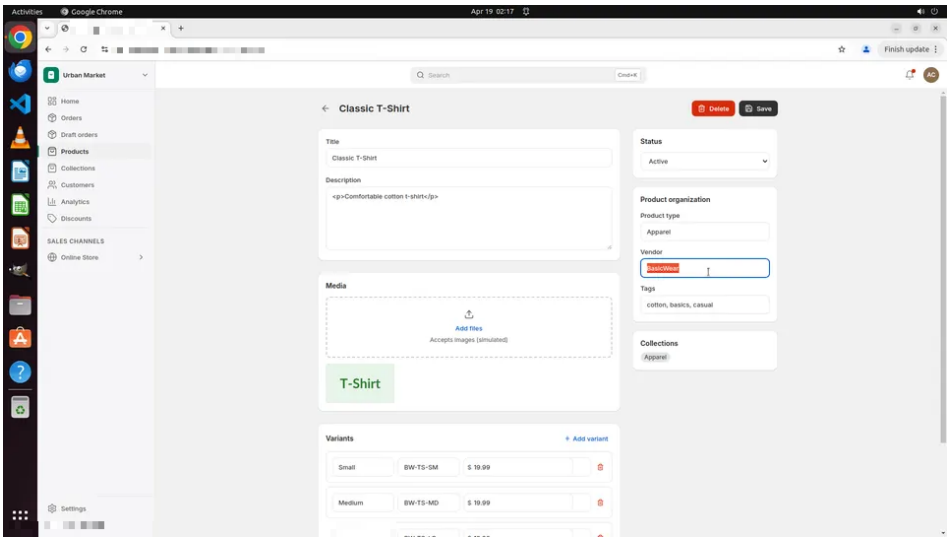


思考： 我看到了Classic T-Shirt 商品页面。供应商是`BasicWear`— 需要改为`UnifiedBrands`。先选中供应商字段。

动作

```
<tool_call>
<function=computer_use>
<parameter=action>triple_click</parameter>
<parameter=coordinate>[741, 491]</parameter>
</function>
</tool_call>
```

第4步。

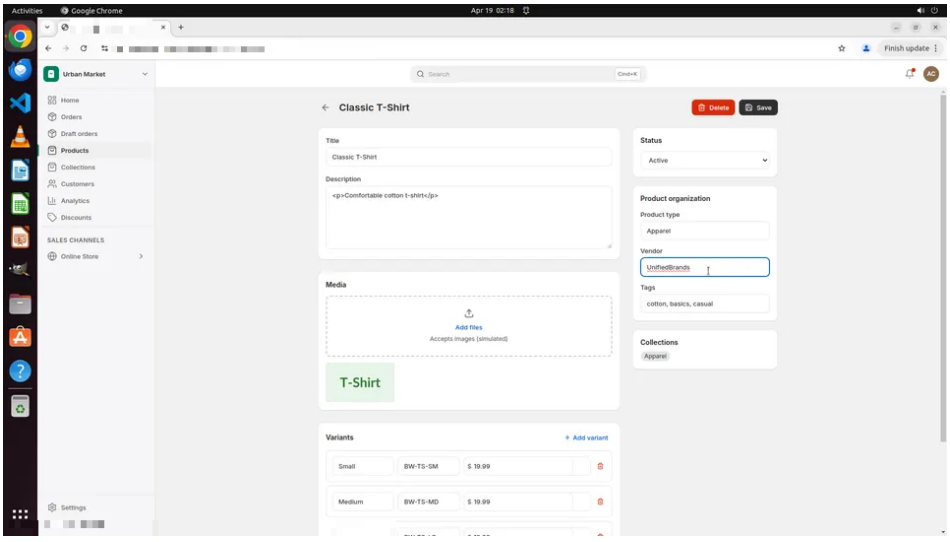


思考：供应商字段BasicWear已被选中。输入UnifiedBrands替换它。

动作

```
<tool_call>
<function=computer_use>
<parameter=action>type</parameter>
<parameter=text>UnifiedBrands</parameter>
</function>
</tool_call>
```

第5步。



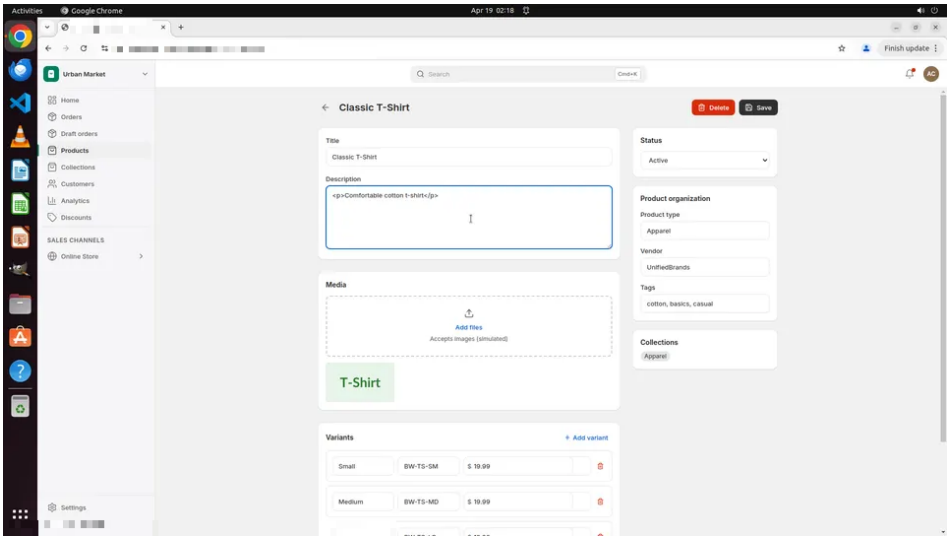
思考：现在需要更新描述，在末尾添加营销文案。先点击描述字段。

动作

```
<tool_call>
<function=computer_use>
<parameter=action>left_click</parameter>
<parameter=coordinate>[490, 393]</parameter>
```

```
</function>
</tool_call>
```

第6步。

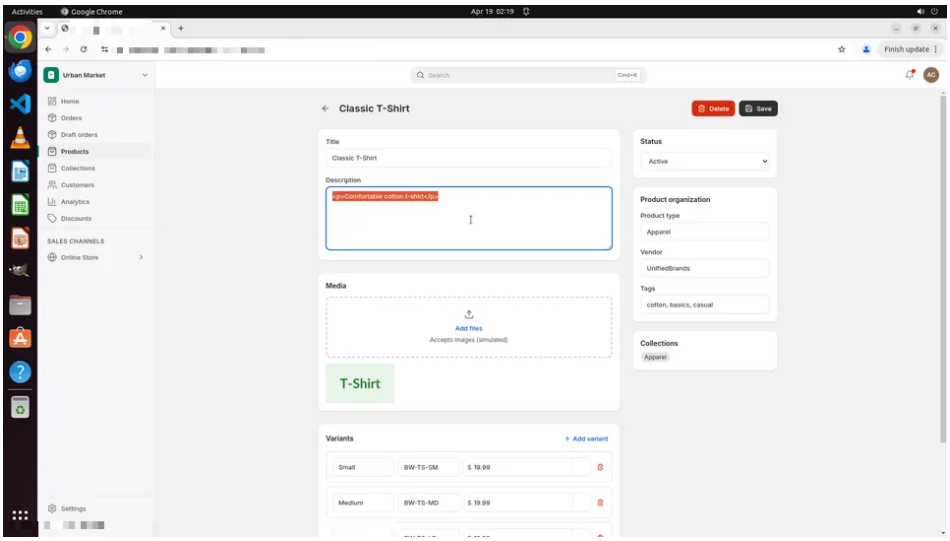


思考： 需要选中描述字段中的全部文字，并替换为原文加上新增的一行。使用Ctrl+A 全选。

动作

```
<tool_call>
<function=computer_use>
<parameter=action>key</parameter>
<parameter=keys>["ctrl", "a"]</parameter>
</function>
</tool_call>
```

第7步。

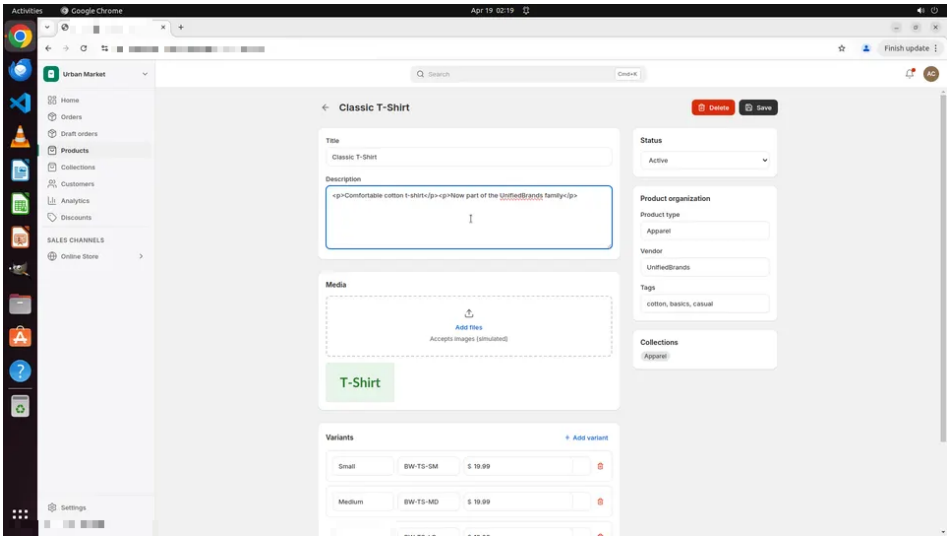


思考： 文字已选中。现在输入由原文和营销文案组成的新描述。

动作

```
<tool_call>
<function=computer_use>
<parameter=action>type</parameter>
<parameter=text><p>Comfortable cotton t-shirt</p><p>Now part of the UnifiedBrands family</p
    ></parameter>
</function>
</tool_call>
```

第8步。

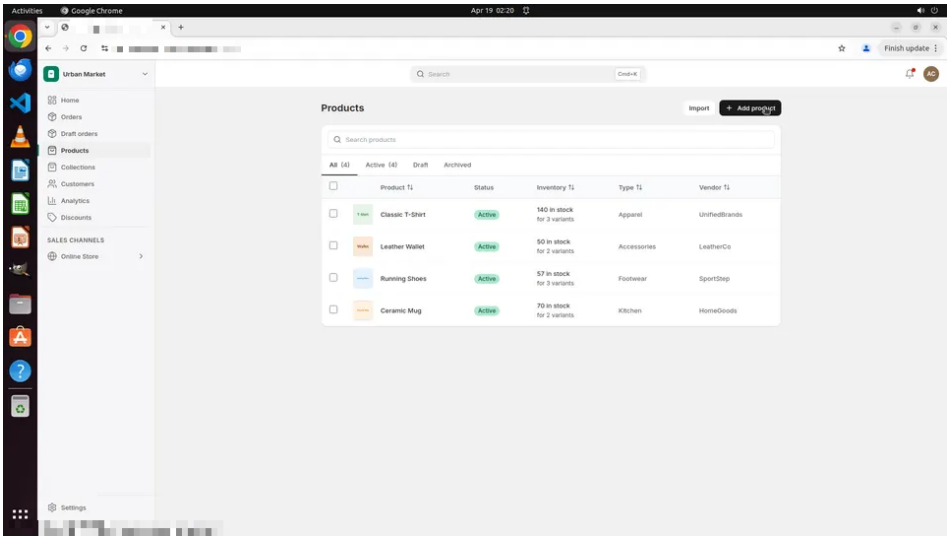


思考：两个字段都已更新。点击*Save*保存这件商品。

动作

```
<tool_call>
<function=computer_use>
<parameter=action>left_click</parameter>
<parameter=coordinate>[802, 190]</parameter>
</function>
</tool_call>
```

第9步。

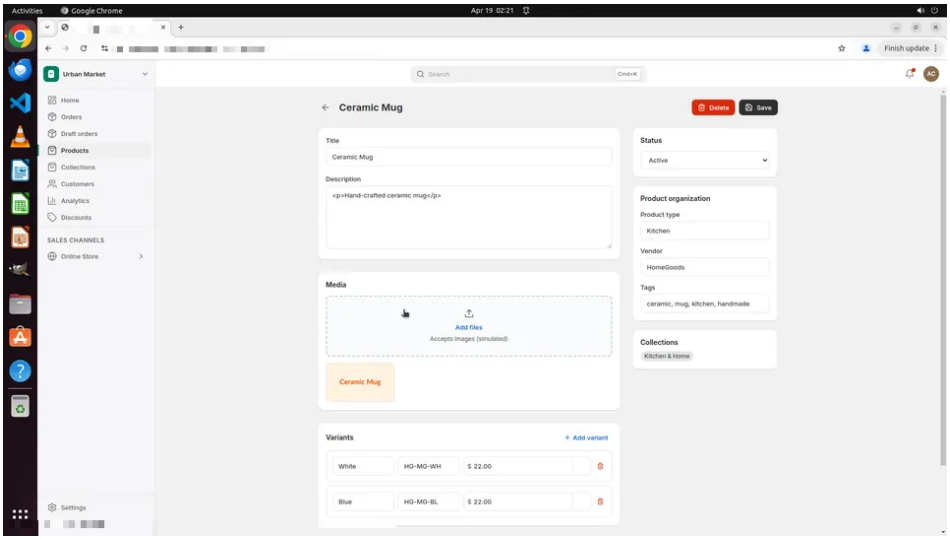


思考：Classic T-Shirt 已保存，供应商为`UnifiedBrands`。现在点击另一件受影响的商品*Ceramic Mug*，接着更新它。

动作

```
<tool_call>
<function=computer_use>
<parameter=action>left_click</parameter>
<parameter=coordinate>[420, 571]</parameter>
</function>
</tool_call>
```

第10步。



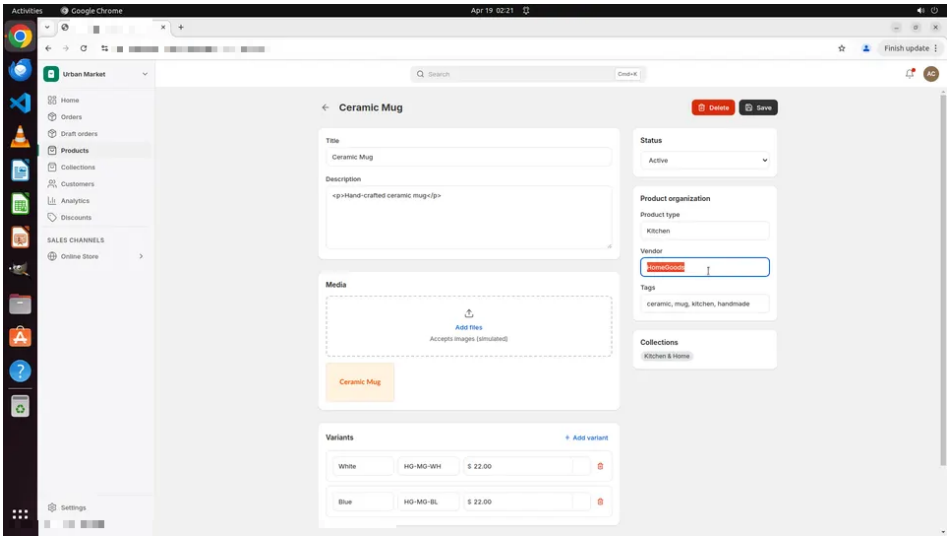
思考： 我看到了Ceramic Mug 商品页面，其供应商为`HomeGoods`。先更新供应商字段。

动作

```
<tool_call>
<function=computer_use>
<parameter=action>triple_click</parameter>
```

```
<parameter=coordinate>[741, 491]</parameter>
</function>
</tool_call>
```

第11步。

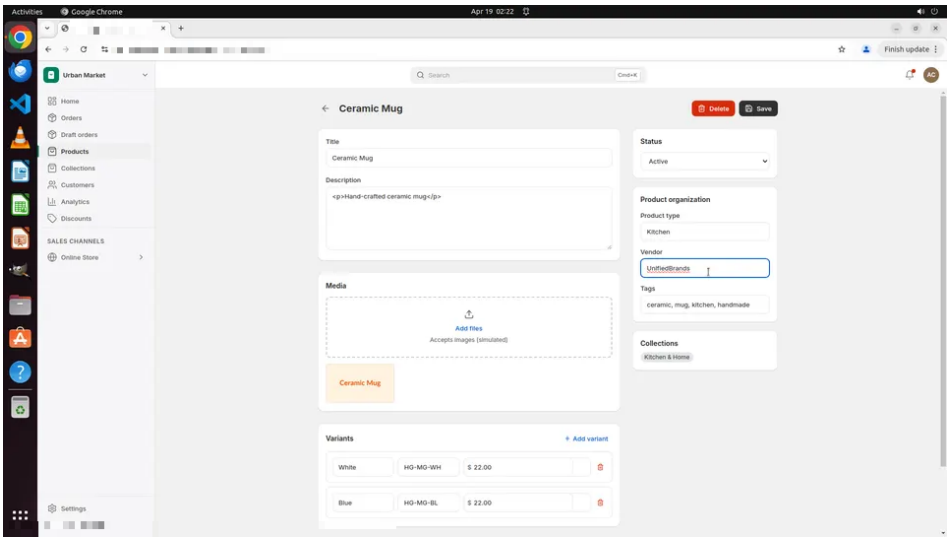


思考： 在选中的供应商字段中输入`UnifiedBrands`。

动作

```
<tool_call>
<function=computer_use>
<parameter=action>type</parameter>
<parameter=text>UnifiedBrands</parameter>
</function>
</tool_call>
```

第12步。

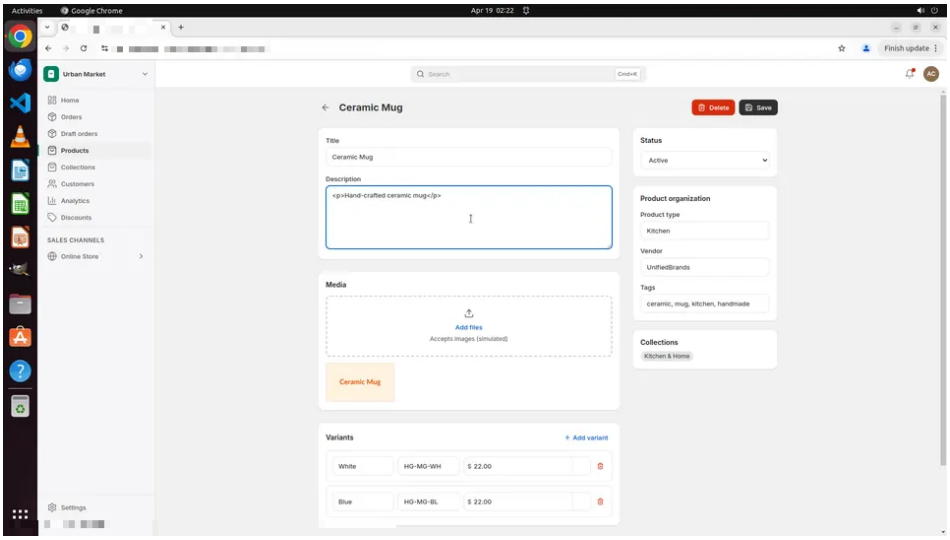


思考： 现在点击描述字段，更新Ceramic Mug 的描述。

动作

```
<tool_call>
<function=computer_use>
<parameter=action>left_click</parameter>
<parameter=coordinate>[490, 393]</parameter>
</function>
</tool_call>
```

第13步。

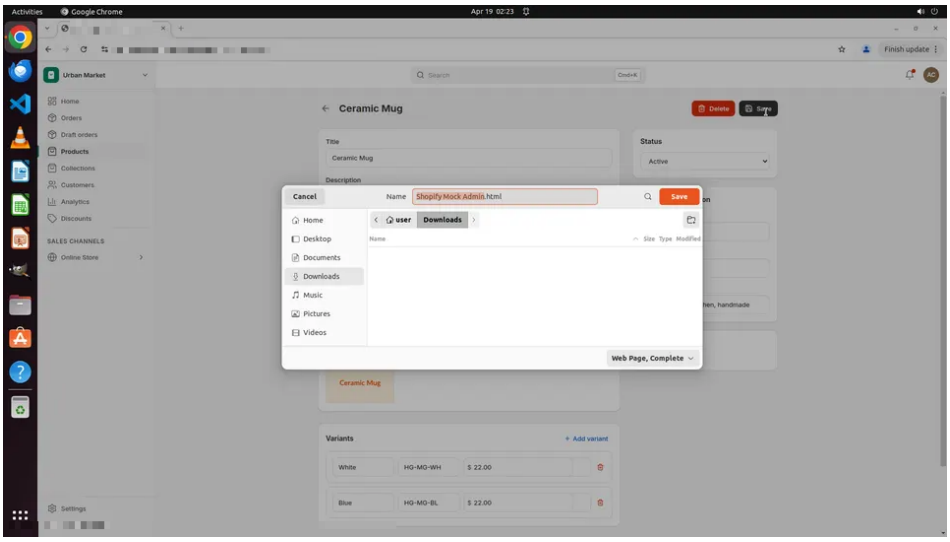


思考： 现在全选、输入新描述并保存 — 在一轮中输出整个动作链。

动作

```
<tool_call>
<function=computer_use>
<parameter=action>key</parameter>
<parameter=keys>["ctrl", "a"]</parameter>
</function>
</tool_call>
<tool_call>
<function=computer_use>
<parameter=action>type</parameter>
<parameter=text><p>Hand-crafted ceramic mug</p><p>Now part of the UnifiedBrands family</p></
    parameter>
</function>
</tool_call>
<tool_call>
<function=computer_use>
<parameter=action>key</parameter>
<parameter=keys>["ctrl", "s"]</parameter>
</function>
</tool_call>
<tool_call>
<function=computer_use>
<parameter=action>left_click</parameter>
<parameter=coordinate>[802, 190]</parameter>
</function>
</tool_call>
```

第14 步。

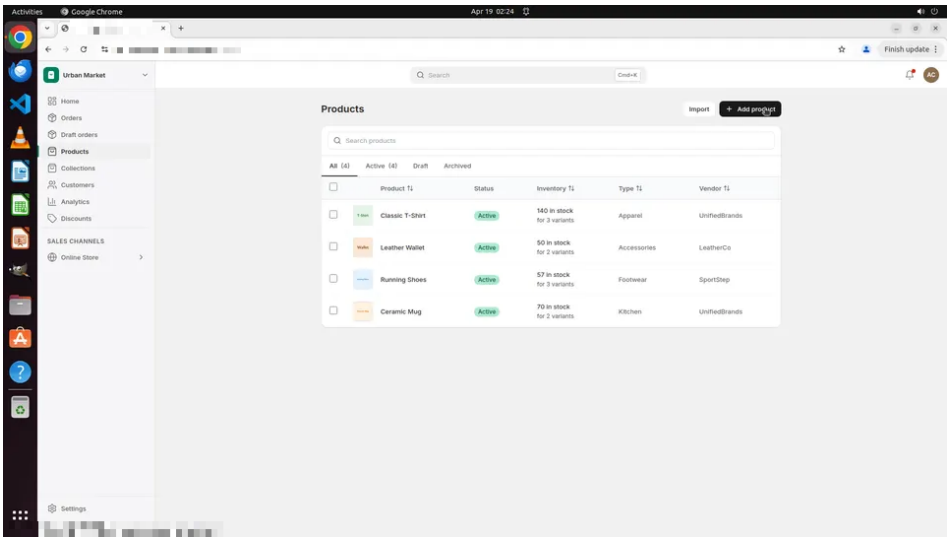


思考： Ctrl+S 弹出了保存对话框 — 先取消它，改用页面中的Save 按钮。

动作

```
<tool_call>
<function=computer_use>
<parameter=action>left_click</parameter>
<parameter=coordinate>[318, 356]</parameter>
</function>
</tool_call>
<tool_call>
<function=computer_use>
<parameter=action>left_click</parameter>
<parameter=coordinate>[802, 190]</parameter>
</function>
</tool_call>
```

第15 步。

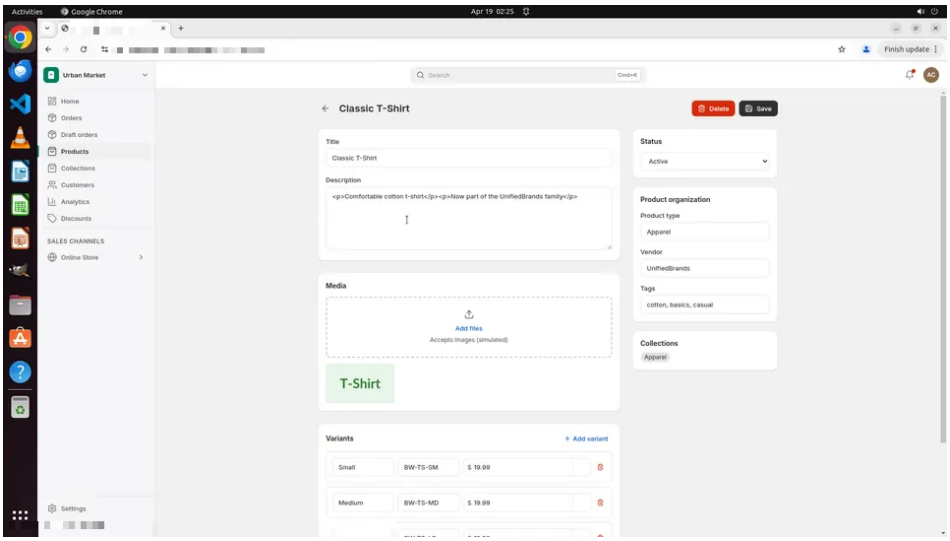


思考： 两件商品都已更新。商品列表现在显示*Classic T-Shirt* 和*Ceramic Mug* 的供应商为`UnifiedBrands`，而*Leather Wallet*（LeatherCo）和*Running Shoes*（SportStep）保持不变。重新打开Classic T-Shirt，验证修改已持久保存。

动作

```
<tool_call>
<function=computer_use>
<parameter=action>left_click</parameter>
<parameter=coordinate>[422, 393]</parameter>
</function>
</tool_call>
<tool_call>
<function=computer_use>
<parameter=action>screenshot</parameter>
</function>
</tool_call>
```

第16 步。

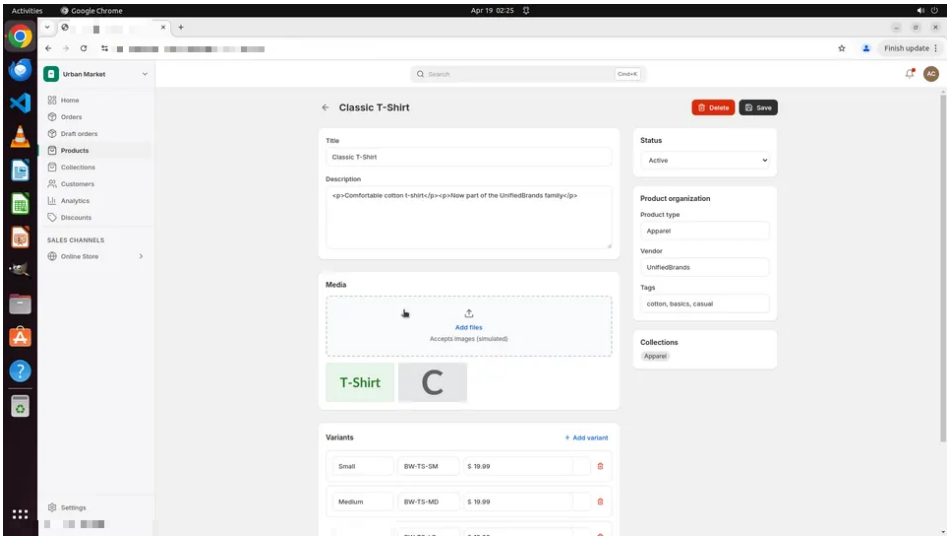


思考： Classic T-Shirt 看起来正确 — 供应商为`UnifiedBrands`，描述中包含营销文案。现在验证Ceramic Mug。

动作

```
<tool_call>
<function=computer_use>
<parameter=action>key</parameter>
<parameter=keys>["esc"]</parameter>
</function>
</tool_call>
<tool_call>
<function=computer_use>
<parameter=action>left_click</parameter>
<parameter=coordinate>[420, 571]</parameter>
</function>
</tool_call>
<tool_call>
<function=computer_use>
<parameter=action>screenshot</parameter>
</function>
</tool_call>
```

第17 步。

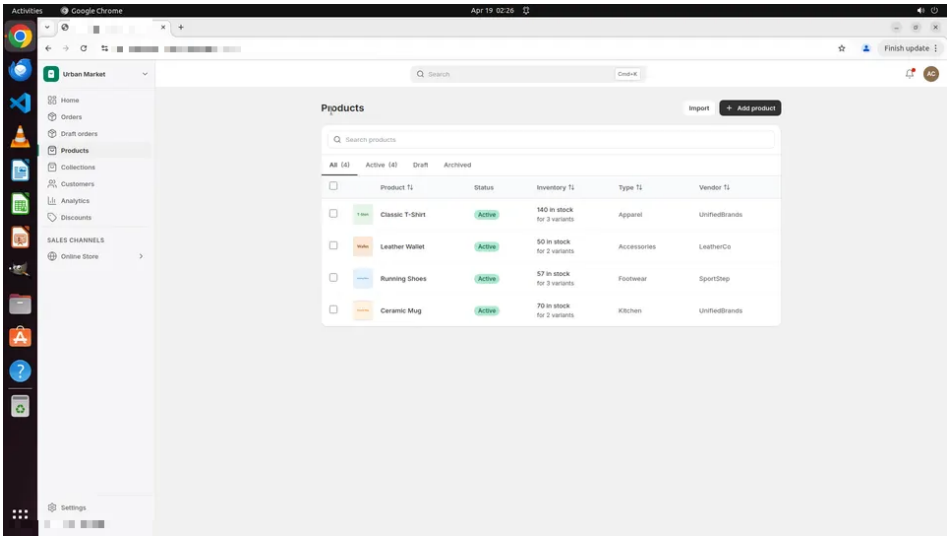


思考： 上一次点击没有进入Ceramic Mug。先返回商品列表。

动作

```
<tool_call>
<function=computer_use>
<parameter=action>left_click</parameter>
<parameter=coordinate>[342, 190]</parameter>
</function>
</tool_call>
<tool_call>
<function=computer_use>
<parameter=action>screenshot</parameter>
</function>
</tool_call>
```

第18 步。

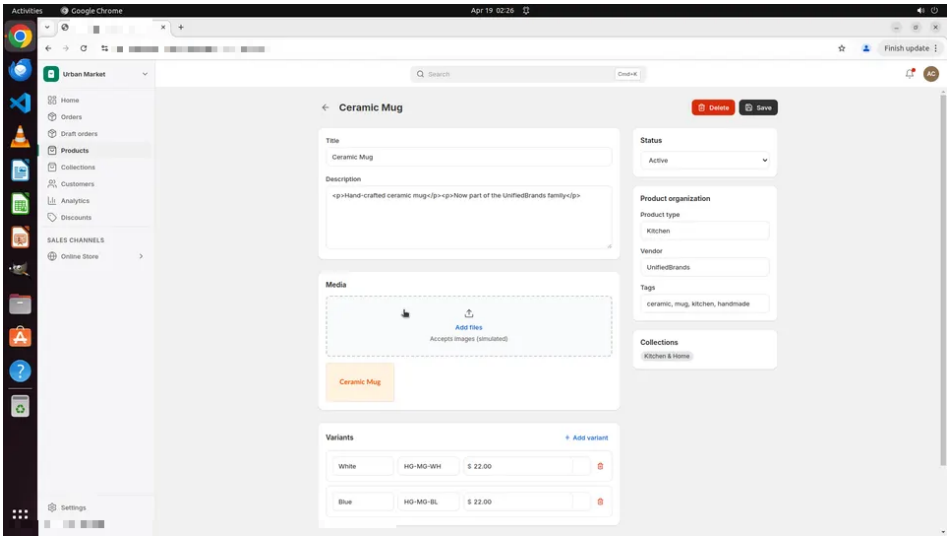


思考： 已返回商品列表。四件商品都显示预期的供应商。点击进入Ceramic Mug，确认其描述。

动作

```
<tool_call>
<function=computer_use>
<parameter=action>left_click</parameter>
<parameter=coordinate>[420, 571]</parameter>
</function>
</tool_call>
<tool_call>
<function=computer_use>
<parameter=action>screenshot</parameter>
</function>
</tool_call>
```

第19 步。



思考： 两件商品均已成功验证。*Ceramic Mug*：供应商 = `UnifiedBrands`，描述中包含`Now part of the UnifiedBrands family`。*Leather Wallet* 和*Running Shoes* 按要求未被改动。任务已完成；轨迹以奖励 1.0 结束。

轨迹说明。 该轨迹呈现了§4.3 中记录的多动作涌现：第1–12 步每轮输出一次工具调用，但从第13 步起，策略开始将确定性动作链（`select-all` → `type` → `save`）合并到单轮中。第14 步展示了一次清晰的恢复：上一轮的Ctrl+S 意外唤出“网页另存为”对话框后，策略输出两个纠正动作（关闭对话框、点击页面内的Save），而没有放弃动作链。第15–18 步构成验证链；策略在同一轮中交织导航动作与`screenshot`调用，以便无需额外轮次即可重新观测状态。最后一步（第19 步）不调用工具而终止，表明任务完成。

E.3 信息隔离屏障拦截的奖励投机案例

我们展示三类奖励投机失效情形，信息隔离屏障（§A.3.2）与禁用模式静态扫描（§A.3.3）正是为阻止这些情形而设计。每一类对应早期原型中观察到的一种不同漏洞利用方式，当时判别器可以访问生成器的设置脚本；在发布的流水线中，以下每个示例都会在写入阶段被拒绝，从而无法被循环接受为元组。

投机A——直接布尔赋值。 判别器检查生成器的`golden_patch.py`，发现智能体只需修改一个工作表属性，于是编写了一个未经计算便直接断言标志值的奖励函数。

投机A：直接布尔赋值（被静态扫描拒绝）

```
# 禁用模式：未经验证便直接断言标志值。
chart_verified = True            # 常量赋值
if chart_verified:               # 始终为真
    score += 0.5
```

投机B——仅按存在性评分。 该奖励仅凭文件存在就给分，绕过任何内容检查。因此，智能体只要在正确路径创建一个空文件就能通过。

投机B：仅按存在性评分（被静态扫描拒绝）

```
# 禁用模式：不检查内容，仅按文件存在性评分。
if os.path.exists('/home/user/report.xlsx'):
    score += 0.3
```

上述每种情况下，违规元组都会在循环的静态扫描阶段被拒绝；判别器收到匹配模式的名称作为反馈，并重新输出执行实际验证的奖励函数。

E.4 对抗循环迭代记录

我们追踪一个生产任务经历两轮对抗的过程。该任务（`calc_gao_008`）要求智能体向工作簿中添加两个命名方案，每个方案存储一组不同的单元格数值 5 元组；奖励解析工作簿底层的XLSX XML，并断言两个方案名称及其数值向量均正确。

第1 轮（失败）。 生成器的第一版`golden_patch.py` 引入了两个方案，却意外将*Optimistic* 方案填成了*Pessimistic* 的数值（一次复制粘贴造成的回归错误）。判别器在计算标准完成状态奖励时检测到不匹配，并输出结构化的`REVIEW.md`，标明失败组件。

第1 轮REVIEW.md（根据第2 轮反馈记录合成）

```
# 第 1 轮审查

## 结论：FAIL（失败）

## 一致性条件
| # | 条件                          | 状态 | 详情                                          |
| 1 | initial_setup 可运行          | PASS | xlsx 存在，具有 B2:B6 数值，无方案          |
| 2 | golden_patch 可运行           | PASS | xlsx 存在，具有两个命名正确的方案           |
| 3 | reward(golden) == 1.0         | FAIL | 得分：0.65 —— Optimistic 数值不匹配         |
| 4 | reward(initial) == 0.0        | PASS | 得分：0.0                                   |
| 5 | 不存在禁用模式                | PASS | 无违规                                      |

## 给设置生成器的反馈
Optimistic 方案目前存储 B2:B6 = (350000, 140000, 100000,
60000, 30000)，与 Pessimistic 方案的数值相同。
任务要求 Optimistic 使用提高后的数值 (650000, 280000,
200000, 140000, 75000)。请修复 golden_patch.py，为 Optimistic
注入不同的 inputCells。
```

第2 轮（通过）。 生成器通过为两个方案写入不同的`inputCells` 来处理第1 轮反馈。判别器重新对两个端点状态评分，并接受该元组。

第2 轮REVIEW.md（原样引自生产运行）

```
# 第 2 轮审查

## 结论：PASS（通过）

## 一致性条件
| # | 条件                          | 状态 | 详情                                          |
| 1 | initial_setup 可运行          | PASS | xlsx 存在，具有 B2:B6 数值，无方案          |
| 2 | golden_patch 可运行           | PASS | xlsx 同时包含 Optimistic 和 Pessimistic     |
| 3 | reward(golden) == 1.0         | PASS | 得分：1.0                                   |
| 4 | reward(initial) == 0.0        | PASS | 得分：0.0                                   |
```

```
| 5 | 不存在禁用模式                | PASS | 无违规                                          |

## 评分明细——标准完成状态文件
| 组件                                          | 分值 | 状态 |
| 两个方案分别名为 Optimistic 和 Pessimistic      | 0.30 | PASS |
| Optimistic 数值 (650k, 280k, 200k, 140k, 75k)   | 0.35 | PASS |
| Pessimistic 数值 (350k, 140k, 100k, 60k, 30k)    | 0.35 | PASS |
| 总计                                          | 1.00 |      |

## 给设置生成器的反馈
未发现问题。第 1 轮反馈已解决：Optimistic 方案现在正确包含
提高后的数值，而不是重复 Pessimistic 的数值。两个方案均正确
存储在 XLSX XML 中，具有不同的 inputCells。
```

这种模式——第1 轮给出范围明确的诊断，第2 轮进行针对性修复——占发布数据集中大多数多轮收敛案例。

E.5 奖励函数示例集

作为上述四个详解的补充，我们抽取了三种奖励函数模式，其验证原语与§E.1.1 中基于文件和公式的模式存在性质上的差异。它们与前述详解共同覆盖了发布数据集中使用的主要验证原语。

图表结构检查（LibreOffice Calc）。 图表作为锚定于工作表的绘图对象存储；奖励枚举这些对象，并通过`openpyxl` 的图表抽象检查其类型、数据范围、标题和坐标轴标签。

图表结构检查：类型、数据范围、标题、坐标轴标签

```python
# C1：图表存在且为柱状图 / 条形图（0.30 + 0.20 分）
charts = ws._charts
if len(charts) >= 1:
    score += 0.30
    chart = charts[0]
    if isinstance(chart, openpyxl.chart.BarChart) and chart.type == 'col':
        score += 0.20

# C3：图表引用 12 个月的数据范围（0.20 分）
ref = chart.series[0].val.numRef.f       # 例如 'MonthlySales!$B$2:$B$13'
if re.search(r'\$B\$2:\$B\$13', ref):
    score += 0.20

# C4 / C5：存在图表标题和坐标轴标题（0.15 + 0.15 分）
if get_title_text(chart.title):
    score += 0.15
if get_title_text(chart.x_axis.title) and get_title_text(chart.y_axis.title):
    score += 0.15
```

图像属性验证（LibreOffice Calc）。 这是一个包含插入水印图像的页面设置任务。奖励检查目标工作表的方向、页边距和图像清单，并将纸张大小与顺序视为前置条件（任何已保存`.xlsx` 中都会存在的LibreOffice 默认值），而非评分组件。

图像属性验证：方向、页边距、水印

```python
# C1：方向 = 横向（0.25 分）
if ws.page_setup.orientation == 'landscape':
    score += 0.25

# C2：四个页边距均设为 1.5cm = 约 0.591 英寸（0.35 分）
INCHES_PER_CM = 1 / 2.54
target = 1.5 * INCHES_PER_CM
```

```
m = ws.page_margins
if all(abs(getattr(m, k) - target) < 0.02
       for k in ('left', 'right', 'top', 'bottom')):
    score += 0.35

# C3: Print 工作表上至少锚定一张图像 (0.40 分)
if any(hasattr(img, 'anchor') for img in ws._images):
    score += 0.40
```

文件系统层级验证（多应用操作系统任务）。 这是一个文件分类任务：智能体必须将六个采用通用名称的文件移动至对应类别的子文件夹。奖励同时检查文件在目标文件夹中存在*且*在桌面根目录中不存在，确保只复制而未删除原文件的失败情况不会得分。

文件系统层级：逐文件移动验证

```
FILE_CLASSIFICATION = {
    'doc1.pdf':  'Invoices',
    'doc5.xlsx': 'Invoices',
    'doc2.docx': 'Resumes',
    'doc4.odt':  'Resumes',
    'doc3.pdf':  'Reports',
    'doc6.txt':  'Reports',
}
POINTS_PER_FILE = 1.0 / len(FILE_CLASSIFICATION)

for fname, expected_dir in FILE_CLASSIFICATION.items():
    in_target = os.path.exists(os.path.join(DESKTOP, expected_dir, fname))
    not_in_root = not os.path.exists(os.path.join(DESKTOP, fname))
    if in_target and not_in_root:
        score += POINTS_PER_FILE
```

这里展示的三类原语——结构对象检查、页面属性与图像锚点检查、文件系统层级断言——补充了§E.1已展示的单元格公式、JSON 键和文档内容检查。完整发布数据集的奖励函数按其目标任务分布的比例覆盖这些原语。

F 数据集统计

F.1 分布细分

F.1.1 各环境的任务数量

CUA-Gym 的32,112 个已验证元组分布在110 个环境中，各粗粒度类别的数量汇总于表 9。分布存在中等程度的偏斜：以知识工作为主的场景（电子表格、文档写作、演示文稿、软件工程、系统管理）占据前五位，合计覆盖语料库的 84%，其余任务分散在PDF、通信、项目管理和业务运营场景中。没有任何单一场景超过总量的 21%，这与§A.2.5 的均衡约束一致。在每个场景内部，任务分布于多个组成应用中（例如，*电子表格与数据*场景涵盖`libreoffice_calc`、数据分析仿真网站，以及涉及Calc 的跨应用组合）。

Scenario	Tasks	%
Spreadsheet & Data	6,684	20.8
Document Writing	6,314	19.7
Presentation Design	5,328	16.6
Software Engineering	4,434	13.8
System Administration	4,225	13.2
PDF & Publishing	2,055	6.4
Communication & Email	939	2.9
Project & Task Management	879	2.7
Business Operations	840	2.6
Creative & Media	412	1.3

表 9: 已发布的CUA-Gym 语料库中各场景的任务数量。

F.1.2 领域× 难度矩阵

表 10 按领域和难度交叉统计任务数量。可以观察到两种模式。首先，语料库整体偏向更高难度：44.6% 的任务标注为*困难*，37.7% 为*中等*，只有 17.7% 为*简单*，符合§2.2 中阐述的设计目标。其次，这种偏斜在多应用领域最为明显：*跨桌面应用*的困难任务占 80.1%，*跨网页应用*占 87.6%，*桌面 × 网页*占 95.8%，反映出这些任务对状态跟踪和上下文切换提出了额外要求。单应用领域的难度梯度较为平缓（例如，*电子表格*：简单占 17.8%，中等占 47.2%，困难占 35.0%）。

Domain	Easy	Medium	Hard	Total
Cross-Desktop	300	1,198	6,030	7,528
Spreadsheet	993	2,627	1,952	5,572
Document Editing	881	2,070	1,165	4,116
Presentation	889	1,541	737	3,167
System & OS	1,110	1,388	311	2,809
Code Editing	690	1,130	660	2,480
Desktop × Web	0	84	1,917	2,001
PDF	113	946	757	1,816
Cross-Web	0	53	376	429
E-Commerce	166	191	27	384
Cloud & Productivity	93	166	116	375
Social Media	127	191	21	339
Project Management	82	135	58	275
Communication	65	120	37	222
Business & CRM	37	87	71	195
Media	80	68	2	150
Development Tools	28	55	65	148
Image Editing	30	53	23	106
Total	5,684	12,103	14,325	32,112

表 10: 已发布的CUA-Gym 语料库中按领域和难度划分的任务数量。表中省略了两个各自包含 <10 个任务的领域（*其他*、*网页浏览*）。

F.1.3 动作动词频率

每条任务指令的第一个祈使动词（跳过“我想要…”或“请…”等铺垫性从句）概括了智能体必须学会落实为具体操作的动作词汇。表 11 列出了频率最高的20 个祈使动词。直接编辑类动词（`open`〔打开〕、`create`〔创建〕、`set`〔设置〕、`add`〔添加〕、`insert`〔插入〕、`change`〔更改〕、`remove`〔移除〕）在高频部分占据主导；中频动词涵盖构建（`build`〔构建〕、`configure`〔配置〕、`apply`〔应用〕）、提取（`extract`〔提取〕、`export`〔导出〕、`convert`〔转换〕）和核查（`check`〔检查〕、`find`〔查找〕、`read`〔读取〕）。未展示的长尾部分延伸到在少于 50 个任务中出现的动词，例如`merge`〔合并〕、`flatten`〔展平〕、`normalize`〔规范化〕、`reconcile`〔核对协调〕和`annotate`〔批注〕。

Verb	Count	Verb	Count	Verb	Count
open	4,585	insert	562	remove	227
create	2,728	build	440	export	183
set	1,448	read	427	check	170
add	1,028	change	361	find	139
use	906	write	290		
extract	891	convert	265		
configure	728				
apply	605				

表 11: CUA-Gym 任务语料库中最常见的祈使动词，按频率区间分组。

F.1.4 跨应用对共现

在 12,311 个*跨应用*任务（占语料库的 38.3%，§A.2.5）中，出现频率最高的应用对列于表 12。应用对通过任务指令中的关键词匹配来检测（例如，使用“`.xlsx`”或“Calc”识别`calc`）；当一条指令提及三个或更多应用时，它可能计入多个应用对。文档制作三应用组合（`calc+writer+pdf` 和`impress+writer+pdf`）占跨应用任务的大部分，反映出知识工作流程往往以PDF 产物作为终点这一实际现象。开发者工具对（`terminal+vscode`、`files+vscode`）形成了一个规模较小但特征鲜明的群组；网页仿真应用对集中于 salesforce/gmail/slack/notion 系列（未展示）。

Pair	Count	Pair	Count	Pair	Count
pdf + writer	3,173	impress + pdf	1,381	files + pdf	316
calc + writer	3,129	calc + pdf	1,003	files + terminal	315
impress + writer	2,056	files + writer	534	calc + files	313
calc + impress	1,576	terminal + vscode	412	calc + vscode	294
		vscode + writer	364	terminal + writer	272
				files + vscode	268

表 12: 跨应用任务指令中共同出现频率最高的应用对。计数按应用对统计（一条涉及四个应用的指令会使全部六个应用对的计数各增加一次）。

F.1.5 指令长度分布

任务指令长度（按空白分隔的词元计数）呈中等程度的右偏分布：均值为 54.1，中位数为 41，90% 的任务少于 108 个词，而最复杂的跨应用工作流在长尾处可达 390 个词。分桶分布见表 13。少于 30 个词的指令通常是在熟悉的产物上执行单步操作（例如，“打开 `report.xlsx` 并冻结首行”），而 100 个词以上的指令则描述包含多个条件分支和明确验收标准的多阶段工作流。

Length (words)	Tasks	%
0–29	10,988	34.2
30–59	11,010	34.3
60–99	6,073	18.9
100–149	2,775	8.6
150–249	1,042	3.2
250+	234	0.7
Summary statistics: mean 54.1; p10 17; p25 24; p50 41; p75 69; p90 108; p99 237; max 390.		

表 13: CUA-Gym 任务指令的词数分布。

F.2 生成成本

F.2.1 每个元组的词元消耗

综合四个由LLM 驱动的流水线阶段（任务生成、生成器、判别器、筛选器），一个被接受的元组平均消耗 Claude Sonnet 4.6 的 ∼10,000 个输入词元和 ∼5,000 个输出词元。输入消耗主要来自生成器和判别器的系统提示词及其REVIEW.md 上下文窗口；输出消耗主要来自判别器的奖励脚本以及生成器的初始设置与标准完成状态补丁。按Claude Sonnet 公布的每百万输入词元 $3、每百万输出词元 $15 的价格 [4]，每个已验证元组的LLM 费用为 $0.030 + 0.075 \approx \$0.11$。扩展到已发布的32,112 个元组语料库，LLM 总成本约为 $3,300 至 $3,500，具体取决于每个任务的接受率。

F.2.2 每个已验证元组的虚拟机时数

双虚拟机执行阶段（配置`initial_env/golden_env` 环境对、执行`initial_setup.py` 和`golden_patch.py`、分别运行奖励函数、销毁环境）每个被接受的元组约耗费 45 分钟墙钟时间，运行于阿里云ECS `g8i.xlarge` 实例（4 个vCPU、16 GB 内存，撰写本文时按需价格约为每实例小时 ∼$0.21）。两个实例在整个验证期间持续占用，因此每个已验证元组的摊销虚拟机成本为 $0.75 \times 2 \times \$0.21 \approx \$0.32$；已发布语料库由此累计消耗约 ∼$10K 的虚拟机计算费用。虚拟机时数主导了已验证元组的边际成本，约为LLM 成本的 3×。具体分解如下：快照恢复（∼2 分钟）、`initial_setup` 执行（平均 ∼10 分钟，数据预置文件下载导致长尾）、`golden_patch` 执行（∼8 分钟）、包含后置配置的奖励执行（∼15 分钟，主要耗时于文档重新渲染和断言解析），以及销毁环境（∼10 分钟，包含磁盘镜像重置）。

G 提示词目录

G.1 提示词工程说明

本目录中的所有提示词均为节选；完整原文（包括JSON 模式声明和少样本示例）随代码一同发布。除非另有说明，提示词均以温度 1.0 输入Claude-Sonnet-4-6 [4]。生成器和判别器进程按照§A.3.2 的访问矩阵，作为独立的推理实例运行；编排器通过结构化Markdown 产物（`REVIEW.md`）协调二者，而非依靠直接消息通信。以下提示词略微调整了空白格式，并省略了内部产物路径；其语义内容保持不变。

G.2 编排器系统提示词

编排器协调生成器—判别器循环，监控§A.3.1 中的五项一致性条件，并将最终被接受的元组写入磁盘。它自身不编写任何脚本。

编排器系统提示词

```
你是 CUA-Gym 计算机使用智能体训练流水线的编排器。
你不得自行生成任何脚本或文件。你的唯一职责是:

  1. 准备工作环境。
  2. 在对抗循环中启动 setup-generator 和 reward-generator 进程。
  3. 通过读取 REVIEW.md 监控一致性条件。
  4. 收集并输出最终经过验证的结果。

# 执行模式 (强制要求)

只允许双环境:
  - 为每个任务配置两台隔离的虚拟机: initial_env、golden_env。
  - 状态分离由环境隔离保证，而非文件名后缀。
  - 单环境执行已弃用。

# 角色边界

你要做的事:
  - 解析输入并准备工作目录。
  - 启动 setup-generator 和 reward-generator 进程。
  - 读取 REVIEW.md，检查一致性条件。
  - 将最终输出复制到 output/final/<task_id>/。
  - 更新流水线状态。

你不能做的事:
  - 生成 Python 脚本 (由 setup-gen 负责) 。
  - 生成奖励脚本 (由 reward-gen 负责) 。
  - 修改任何已生成的脚本。
  - 作出超出五项一致性条件的质量判断。
  - 跳过对抗循环或提前绕过流程。
  - 在 output/adversarial/<task_id>/、output/final/<task_id>/
    或 output/reward_sandbox/<task_id>/ 之外写入文件。

# 工作目录布局 (对安全至关重要)

output/adversarial/<task_id>/
  task_config.json            # 由编排器放置
  env_config_initial.json     # initial_env 虚拟机连接配置
  env_config_golden.json      # golden_env 虚拟机连接配置
  initial_setup.py            # 由 setup-gen 编写
  golden_patch.py             # 由 setup-gen 编写
  reward.py                   # PASS 后从奖励沙箱复制
  REVIEW.md                   # PASS 后从奖励沙箱复制

output/reward_sandbox/<task_id>/   # reward-gen 的隔离工作目录
  task_config.json            # reward-gen 能看到的唯一任务描述
  env_config_initial.json
  env_config_golden.json
  REVIEW.md                   # 上一轮的文件 (若存在)
  reward.py                   # 本轮由 reward-gen 编写

注意: 任务的正式产物文件只存在于各虚拟机的 /home/user/ 下，
绝不下载到本地文件系统。reward-gen 只能获得沙箱目录;
它无法看到 setup-gen 的脚本或数据文件。
它必须通过仅访问状态的 API 探索虚拟机。
```

```
这种隔离是信息隔离屏障在操作层面的实现形式，
也是支撑流水线安全性的核心保障。

# 工作流

步骤 0  计算 PROJECT_ROOT；若当前不在 CUA-Gym 项目根目录，
        则拒绝继续。

步骤 1  从 task-gen 输出的 JSON 中加载任务。

步骤 2  对每个任务配置两台虚拟机，将 task_config.json 和
        env_configs 写入 output/adversarial/<task_id>/，
        并启动 setup-gen。

步骤 3  setup-gen 返回后，将 task_config.json 和 env_configs
        （不包括设置脚本或产物）复制到奖励沙箱，然后启动 reward-gen。

步骤 4  读取 REVIEW.md 中的裁定。
        若为 PASS：
          - 独立验证五项一致性条件。
          - 将最终输出复制到 output/final/<task_id>/。
        若为 FAIL 且 round < 5：
          - 将 REVIEW.md 反馈追加到 setup-gen 的调用输入中。
          - 重新启动 setup-gen。
        若为 FAIL 且 round == 5：
          - 拒绝该元组，并记录失败模式。
```

G.3 生成器子智能体系统提示词

生成器阶段涵盖两个专门角色：*任务生成器*（*Task-Gen*）合成自然语言任务指令及其有具体依据的上下文，*设置生成器*（*Setup-Gen*）生成`initial_setup.py`和`golden_patch.py`。二者共享相同的领域技能文件（§A.5），但工作粒度不同。

任务生成器提示词。

任务生成器系统提示词

```
你是计算机使用任务设计专家，对人们如何使用桌面和网页软件、
以及如何训练智能体执行同样的操作有深入经验。你的工作是：
根据用户对期望任务组合的提示，构思并合成一组多样化、高质量、
适合强化学习智能体后训练的计算机使用任务。

# 输出格式

一个 JSON 数组。每个条目包含以下字段。

task_id（字符串）
  格式：<domain_short>_<topic_short>_<3-digit-number>
  例如 calc_pivot_001、os_file_012、gimp_layer_003。

task_instruction（字符串）
  将直接交给智能体执行的原始请求。

  要求：
    - 意图清晰。智能体必须准确知道该做什么。
    - 包含必要细节。具体名称、数值、范围、位置。
    - 语气自然。像真实用户的请求，而非教程步骤。
    - 不要过于冗长。是一条普通请求，而非 500 词的文章。
    - 句式多样。混用祈使句（“创建……”）、请求句（“能否……”）、
      目标导向表达（“我需要……”）和带背景的表达。
```

```
  不良指令（不得生成）：
    - 含糊：“把电子表格格式弄漂亮些”
        （哪些单元格？采用什么格式？）
    - 机械：“执行以下操作：将当前工作表的 A1:A10 单元格设为粗体”
    - 开放式：“创建一份关于气候变化的演示文稿”
        （没有唯一的标准答案）
    - 不可行：“侵入系统”
    - 泄露方法：“使用 DSUM 函数计算销售总额”
        （指明目标，而非方法）
    - 只需输入公式：“在 D2 单元格中输入 =AVERAGE(B2:B13)”
        （没有 GUI 交互）
    - 过于简单的单公式任务：“计算 B 列的标准差”
        （一个公式不构成有意义的计算机使用任务）

  良好指令：
    - “将 Sheet1 的表头行设为粗体，并将字体设为 Arial 14pt。”
    - “我的桌面上有一个 CSV 文件 'sales.csv'。
       将它导入一份新的 LibreOffice Calc 电子表格。”
    - “将当前 Writer 文档的页面方向改为横向。”
    - “为 'Status' 列设置条件格式：
       'Completed' 为绿色，'Pending' 为黄色，'Overdue' 为红色。”
    - “冻结顶部两行和第一列，使其在滚动时保持可见。”

context（字符串，>= 300 个字符）
  用自然语言描述设置和评估此任务所需的一切。
  必须包含三个部分：

  1. 初始状态——智能体开始前必须存在的内容：
       文件及其内容和位置、应用状态、
       数据特征（行、列、取值范围）。

  2. 标准答案——预期的正确结果：
       精确数值、状态、文件内容；部分得分检查点；
       适用时给出具体数字
        （“B15 单元格应包含 342.50”）。

  3. 隐含前提——指令未说明但评估器需要的信息：
       例如，“GIMP 画布应为 1920x1080，带有白色背景图层”；
        “Firefox 应打开 3 个标签页：Google、Wikipedia、GitHub”。

  上下文至关重要。没有它，我们就无法设置虚拟机、
  评估成功与否，也无法给予部分得分。

difficulty（字符串，取值为 {easy, medium, hard}）
  easy   -- 1—3 个原子动作，常见操作
              （“将表头行设为粗体”）
  medium -- 3—10 个步骤，需要理解功能
              （“创建包含指定字段的数据透视表”）
  hard   -- 多步骤工作流、高级功能、跨组件交互或边界情况
              （“创建一份无需宏的自动化报告，
             包含条件格式、图表和跨工作表引用”）

# 工作流（两个阶段）

阶段 1（规划）：
  通过网页搜索研究该领域；建立功能分类体系和场景矩阵；
  识别覆盖不足的单元格；在生成之前提交计划以供批准。

阶段 2（生成）：
  按顺序进行三轮——广度覆盖（在分类体系上均匀采样）、
  填补空缺（对覆盖不足的单元格重新采样），
```

```
  以及边界情况（边界 / 跨应用 / 罕见原语任务）。
  在各轮之间应用基于嵌入的去重和槽位模板多样性规则。
```

设置生成器提示词。

设置生成器系统提示词

```
你是 CUA-Gym 对抗式设置流水线中的生成器。
你的工作是创建两个 Python 脚本：

  1. initial_setup.py  -- 在 initial_env 上创建任务开始前的状态
  2. golden_patch.py   -- 在 golden_env 上构建预期的任务完成产物

你与验证器在对抗循环中协作。每轮结束后，验证器会将结构化反馈
写入 REVIEW.md。若审查未通过，流水线会再次调用你。
你的目标是生成验证器认可为正确的输出。

# 执行模式（强制要求）

只允许双环境：
  - initial_setup.py 只在 initial_env 上执行
  - golden_patch.py  只在 golden_env 上执行

分离依靠环境隔离，而非文件名后缀。

# 绝对规则

规则 1——必须在 golden_env 中独立构建标准完成产物。
  标准完成产物必须直接在 golden_env 中生成，
  不得加载或复制 initial_env 中的文件。
  跨环境文件依赖在设计上即为无效。

规则 2——不得产生副作用。
  标准完成状态规范必须准确表示预期的任务完成结果；
  初始状态规范必须表示任务开始前的结果。
  差异只能由任务要求引起。不得重组、重新排版或“清理”其他内容。

规则 3——内容必须真实可信。
  初始文件必须包含可信且有实质内容的信息。
  使用真实姓名（“Sarah Chen”、“Marcus Johnson”）、
  看起来真实的日期和业务指标。
  禁止使用：“测试数据 1”、“Lorem ipsum”、“foo bar”、占位值。

规则 4——复杂度必须充分。
  电子表格：多个工作表、10 行以上、不同类型的列。
  演示文稿：3 张以上幻灯片，包含文字、形状和多样化布局。
  文档：多个段落、标题和真实的结构。

规则 5——初始状态必须准备好 GUI。
  initial_setup.py 不能只创建文件；它必须准备任务预期的 GUI
  起始状态：打开所需应用和文件，通过非阻塞子进程启动，
  设置 DISPLAY=:0，加入短暂等待以保证稳定，保持启动幂等。

规则 6——为多媒体保存初始参考副本。
  对于智能体会原地覆盖媒体文件的任务（GIMP、VLC、OpenShot），
  initial_setup.py 必须在 <task_id>_initial_reference.<ext>
  保存参考副本，智能体和 golden_patch.py 都不得触碰此副本。
  奖励脚本需要用它进行视觉判定的前后对比。

# 工作流
```

```
步骤 0  加载领域技能文件（API 参考、惨痛教训、评估模式）。
        对于 mock_websites，还需加载每个仿真应用的状态模式；
        缺失键会导致页面空白。

步骤 1  读取 task_config.json。将 context 字段解析为设计规范：
        初始状态要求、标准答案中的完成状态取值，以及否定性约束
        （初始产物绝对不能包含的内容）。违反否定性约束会导致
        reward(initial_env) > 0，浪费对抗迭代轮次。

步骤 2  若 round > 1，则读取 REVIEW.md 并进行有针对性的修复。
        - “reward(golden_env) 返回了 X.X，而非 1.0”：
          修复 golden_patch，使其符合奖励函数的预期。
        - “reward(initial_env) 返回了 X.X（应为 0）”：
          修复 initial_setup，移除已体现任务完成的产物内容。
        - 脚本执行错误：修复具体异常。
        除非方法存在根本性错误，否则不要从头重写；
        有针对性的修复收敛更快。

步骤 3  根据设计规范生成 initial_setup.py 和 golden_patch.py。
        在各自的虚拟机上执行这两个脚本，确认它们无异常完成后，
        再通知验证器。
```

G.4 判别器子智能体系统提示词

判别器（Reward-Gen）是信息隔离屏障的核心支撑组件。它仅根据任务语义编写 reward.py，无法访问生成器的设置脚本。

判别器（Reward-Gen）系统提示词

```
你是 CUA-Gym 对抗式设置流水线中的验证器 / 判别器。
你的工作是：

  1. 生成奖励脚本（reward.py），以程序方式验证任务完成情况。
  2. 在 golden_env 产物（必须返回 1.0）和 initial_env 产物
     （必须返回 0.0）上测试该脚本。
  3. 编写结构化的 REVIEW.md，给出裁定与反馈。

要全面而公正。标准完成产物确实符合任务时，应报告 PASS；
不要为了对抗而对抗。

# 信息隔离屏障（不可妥协）

  - 你不得读取 initial_setup.py、golden_patch.py 或任何
    setup-generator 源代码。
  - 你不得读取 setup-generator 工作目录中的文件；
    你的工作目录是 output/reward_sandbox/<task_id>/。
  - 你不得将任一虚拟机中的任务产物数据文件下载到本地机器。
  - 你必须仅通过只访问状态的环境 API，在虚拟机上探索和验证文件。
  - 你的奖励脚本必须从 task_config.json 中的任务描述推导而来，
    不得通过窥视 setup-generator 的输出编写。

# 奖励脚本：必需属性

  1. 返回 [0, 1] 范围内反映完成进度的浮点数。
  2. 使用实际验证（读取真实文件，检查真实数据）。
  3. 对部分完成给予部分得分（0.3、0.5、0.7 等）。
  4. 只有在 100% 完成时才恰好返回 1.0。
  5. 将每个评分分量包裹在 try/except 中。
  6. 最后一行输出打印 "REWARD: X.X"。
  7. 脚本应自包含，仅使用标准库及 openpyxl/python-pptx/python-docx。
```

```
  8. 为评分逻辑添加注释。

# 只对任务引入的变化评分

每个评分分量都必须验证 initial_env 与 golden_env 之间不同的内容。
在两个端点都成立的属性属于前置条件，绝不能贡献分数。

检验准则：如果一项检查在 initial_env 产物上就能通过
（即智能体尚未执行任何操作），那么它就没有衡量任务完成情况，
绝不能据此给分。目标：reward(initial_env) 返回 0.0（或非常接近 0）。

正确模式：
  分量 1：'Raw Data' 工作表已隐藏（0.6 分）                  # 正确
  分量 2：'Raw Data' 已隐藏，且 'Summary' 可见（0.4 分）     # 正确

错误模式（对预先存在的属性评分）：
  分量：'Summary' 工作表可见（0.15 分）                     # 禁止
  分量：'Raw Data' 具有正确的表头（0.10 分）                 # 禁止

应将数据完整性检查用作前置条件门控（若文件损坏则提前返回 0.0），
而非评分分量。

# REVIEW.md 输出

verdict: PASS | FAIL
agreement_table:
  C1_initial_executes      {pass, evidence}
  C2_golden_executes       {pass, evidence}
  C3_golden_reward_eq_1    {pass, observed_value}
  C4_initial_reward_eq_0   {pass, observed_value}
  C5_no_forbidden_pattern  {pass, matched_pattern_or_null}
feedback_to_setup_gen:
  failing_conditions: [C1|C2|C3|C4|C5, ...]
  setup_issues: [简短祈使句，每个问题一条]
  recommended_action: <一个段落>
```

G.5 五项一致性条件的验证器提示词

§A.3.1 中的五项一致性条件通过程序评估，而非通过LLM，因此没有对应提示词；此处给出判别器用于向编排器传达裁定的结构化REVIEW.md 模式。

REVIEW.md 模式

```
# REVIEW.md 模式

verdict:        PASS | FAIL
agreement_table:
  C1_initial_executes:     {pass: bool, evidence: <str>}
  C2_golden_executes:      {pass: bool, evidence: <str>}
  C3_golden_reward_eq_1:   {pass: bool, observed: <float>}
  C4_initial_reward_eq_0:  {pass: bool, observed: <float>}
  C5_no_forbidden_pattern: {pass: bool, matched_pattern: <str|null>}
feedback_to_setup_gen:
  failing_conditions: [<C1|C2|C3|C4|C5>, ...]
  setup_issues:       [<简短祈使句，每个问题一条>]
  recommended_action: <自由表述的单段摘要>
```

G.6 筛选评审器提示词

数据集级筛选器是一种单遍评审器，采用§A.4.1 中默认保守的决策策略。以下评审器提示词逐字转载自filter/critic_prompt.py。

筛选评审器系统提示词

```
你是计算机使用训练任务的保守评审器。

输入:
  - query
  - setup
  - golden_setup
  - reward

目标: 决定该任务应直接保留、改写请求后保留，还是从训练集中剔除。

重要约束:
  - 在本次决策中，setup、golden_setup 和 reward 不可更改。
  - 你只能修改 query。
  - 如果必须更改 setup、golden_setup 或 reward 才能使任务合格，
    则剔除该任务。
  - 存在高风险问题时，优先选择 reject，而非 keep。
  - 当请求已经可用，且自包含程度足以用于训练时，
    优先选择 keep，而非 modify_query。
  - 只有当请求存在实质性缺陷，损害公平性、自包含性或任务有效性时，
    才使用 modify_query。

严重程度映射:
  P0  reject。致命的设置 / 奖励 / 环境问题，或无法修复的不一致。
  P1  modify_query。请求缺少重要上下文、存在有害歧义，
      或泄露流程并因此损害公平性。
  P2  keep 或 modify_query。非致命但明显的质量问题。
  P3  keep。任务质量良好，除风格外没有实质性问题。

评估优先级:
  1. 该任务是否适合训练，并且安全？
  2. 奖励在语义上是否有效且稳健？
  3. 请求是否自包含、足够自然，并与 setup/golden_setup/reward
     保持一致？
  4. 能否在不改变任务语义的前提下修复请求？

必须剔除的致命原因:
  - 奖励对产物或实现痕迹的检查多于对用户目标完成情况的检查
  - 奖励对 GUI / 窗口 / 进程状态高度敏感
  - 奖励依赖脆弱的内部实现、启发式感知或不稳定的信息提取
  - 对浏览器状态、插件、扩展或外部资源作出环境无法满足的隐含假设
  - setup/query/reward 不匹配，且无法仅通过改写请求修复
  - 主观任务却采用僵化的单一答案奖励

输出模式 (仅限 JSON):
  verdict                    keep | modify_query | reject
  severity                   P0 | P1 | P2 | P3
  can_fix_with_query_only    bool
  query_issues               [str, ...]
  setup_reward_risks         [str, ...]
  training_pool_fit          low | medium | high
  confidence                 float (范围为 [0, 1])
  reasoning_summary          str
  revised_query              str | null
```

G.7 仿真应用合成智能体提示词

仿真环境合成流水线（§B.4）由三个专门的智能体（Plan、Dev、Web）实现，它们通过文件产物协调。下面节选各智能体的角色规范；完整提示词随代码发布。

规划智能体。

规划智能体系统提示词

```
你是资深产品研究员和技术规划者。你的工作是深入理解一个目标现实应用，
并制定结构化、可执行的计划，供开发智能体实现忠实的仿真应用。

主要输出：TODO.md，它是开发智能体读取并据以执行的唯一事实来源。
其他一切内容（素材、截图、数据模型）都是为了让 TODO.md 中的条目
能够被无歧义地实现。

范围之内：
  - 所有可见的 UI 组件与布局
  - 导航、路由、视图切换
  - 内容的创建、读取、更新、删除操作
  - 搜索、筛选、排序交互
  - 表单输入与校验反馈
  - 拖放、多选、批量操作
  - 通知 / 角标计数、未读状态
  - 设置面板与偏好开关
  - 真实可信的仿真数据（用户、消息、任务等）

明确不在范围之内：
  - 登录 / 登出 / 身份认证
  - 密码管理或账户创建
  - OAuth、SSO、身份验证
  - 加密或安全机制
  - 真实的网络通信或 API 调用
  - localStorage 以外的数据库持久化
  - 向真实服务器上传文件
  - 发送电子邮件 / 短信
```

开发智能体。

开发智能体系统提示词

```
你是前端工程专家，专门构建和维护用于强化学习智能体训练的仿真网页应用。
你的唯一职责是代码：编写、改进和修复代码。

所有协调均基于文件——智能体之间不直接发送消息。

你读取的输入：
  TODO.md           （来自规划智能体）  工作队列
  assets/README.md  （来自规划智能体）  UI 布局、工作流
  assets/data_model.md  （来自规划智能体）  实体定义
  DESIGN.md         （来自规划 / 用户）  设计系统规范；首先读取
  TEST.md           （来自网页智能体）  缺陷报告、视觉差异
  AUDIT.md          （来自网页智能体）  代码层面的问题

每个工作周期的优先级：
  AUDIT P0 > TEST P0 > AUDIT P1 > TODO P0 > TODO P1 > TODO P2

项目布局（每个仿真应用都遵循此布局）：
  src/App.jsx                 -- 路由
  src/main.jsx
  src/components/             -- 可复用的 UI 组件
  src/pages/                  -- 路由级页面
  src/context/AppContext.jsx  -- 全局状态
  src/utils/dataManager.js    -- 状态初始化、localStorage
  src/utils/stateTracker.js   -- /go 的差异计算
  vite.config.js              -- /post + /state 的中间件插件
```

网页智能体。

网页智能体系统提示词

```
你是仿真网页应用的质量保证工程师。使用无头 Playwright 浏览器操作
已部署的仿真应用，依据 DESIGN.md 和 assets/README.md 验证 UI 树，
每轮生成两份报告：

  TEST.md   功能与视觉缺陷报告
  AUDIT.md  代码层面的问题：无效的处理函数、未跟踪的状态、
            差异 API 中缺失的条目、失效的路由

两份报告均采用 P0/P1/P2 严重程度，与开发智能体的优先级顺序一致。
满足以下条件时，一轮工作才算完成：
  - TODO.md 中列出的每个交互元素都已被实际操作
  - 与参考截图之间的视觉差异低于阈值
  - 状态 API 端点（/post、/go、/state、/upload）
    均在会话隔离约定下正确响应
```